

Search, Hidden Information & Card-Game Self-Play

A reader's edition of five papers for the MTG Pauper agent

How to use this

Part I retells each paper in plain language, in the order that builds best: Gumbel search first (the core of the search-target plan), then ReBeL (search done properly when the opponent's cards are hidden), PerfectDou (a training trick for the value head), action masking (a sanity check for the current PPO code), and finally a Hearthstone agent trained by self-play, the closest published analogue to this project.

Part II contains the original papers with their reference lists removed, for the equations, figures and appendices.

Numbered notes at the bottom of each page explain RL and game-theory terms for a reader with a statistics / causal-inference background. Watch for flagged *false friends*: words that mean something different here than in statistics.

Contents

1. Gumbel AlphaZero / MuZero
2. ReBeL
3. PerfectDou
4. Invalid action masking
5. Hearthstone self-play

Gumbel MuZero: Search That Still Helps With Two Simulations

Ivo Danihelka, Arthur Guez, Julian Schrittwieser, David Silver (2022), "Policy Improvement by Planning with Gumbel", *International Conference on Learning Representations (ICLR 2022)*

In one paragraph

AlphaZero-style agents improve themselves by running a small look-ahead search at every move and then training their policy network to imitate what the search recommended. That loop only works if the search's recommendation is reliably *better* than the raw network. With a generous search budget it is; with a tiny budget (a handful of simulations) the standard recipe can actually make things worse, and learning stalls. This paper replaces AlphaZero's hand-tuned search heuristics with a construction that comes with a guarantee: sample a few candidate actions *without replacement* using Gumbel noise, evaluate them, pick the winner by "noise + network logit + scaled value", and train the network toward a target distribution built from those values. Whenever the action values are evaluated correctly, the result is provably at least as good as the network's own policy, no matter how few simulations are spent. Empirically, the new agent learns 9x9 Go and Ms. Pac-Man from as few as 2 simulations per move where the original fails below about 16, and it matches the original at large budgets.

Why it matters for the MTG agent

- **Search is expensive in your engine.** Every simulation needs a `fork()` that replays the whole game history. Gumbel search is designed precisely for the regime where you can afford only 2 to 16 simulations per decision, and it still delivers a target that is guaranteed (under correct values) not to be worse than the current policy.
- **A ready-made "search-improved policy target".** The completed-Q target $\pi' = \text{softmax}(\text{logits} + \sigma(\text{completedQ}))$ is a full distribution over legal actions, so it can be added to your PPO loss as a KL distillation term. It trains every action, not just the one that was played.
- **Action masking fits naturally.** Illegal actions simply get logit $-\infty$; the Gumbel trick then never samples them, and the number of candidates becomes $\min(n, \text{number of legal actions})$.
- **Hidden information caveat.** The guarantee assumes the values of the sampled actions are correct. Your determinized search (re-sampling the opponent's hand) gives noisy and possibly biased values. The paper's own partially observable test shows that keeping the network's prior weighted in (a small value scale) protects you from over-trusting those values.
- **Open-source reference.** DeepMind released the search as the `mctx` JAX library, which you can read or port rather than reimplement from the equations.

Primer: how AlphaZero's search works

The paper assumes you know AlphaZero, so here is the minimum. AlphaZero has one neural network with two outputs for any game position s : a policy¹ $\pi(a|s)$, a guess of which move is good, and a value² $v(s)$, a guess of who is winning.

Before each real move, AlphaZero runs Monte-Carlo tree search³ (MCTS). The current position is the root⁴ of a tree. One simulation⁵ walks down the tree, picks a move at each node, reaches a position not yet in the tree, asks the network for π and v there, and propagates v back up, updating the average value and the visit count $N(a)$ of every move on the path.

The PUCT rule

At each node, the move chosen during a simulation maximises a score that trades off "looks good so far" against "the network likes it and we have not tried it much":

$$\text{score}(a) = q(a) + c \cdot \pi(a) \cdot \sqrt{(1 + \sum_b N(b)) / (1 + N(a))}$$

The first term is the current average value of move a . The second is an exploration bonus⁶: proportional to the network's prior probability, growing slowly with total visits at the node, and shrinking as a itself gets visited. This is PUCT⁷. Unvisited moves get a q of zero.

After all simulations, AlphaZero (a) plays a move sampled in proportion to the root visit counts and (b) trains the policy head to match those normalised visit counts and the value head to match the eventual game result. Training data comes from self-play⁸. To make sure the root search tries unlikely moves, AlphaZero mixes Dirichlet noise⁹ into π at the root.

The whole scheme is a form of policy iteration¹⁰: search acts as the improvement operator, the network as a compressed memory of past improvements. If the search output is ever worse than the network, the loop trains the network toward something worse.

1. A policy is just a conditional distribution of actions given state, $\pi(a|s)$. The network outputs it as a vector of probabilities over all moves.

2. The value $v(s)$ is the expected final game outcome (say +1 win, -1 loss) from position s if play continues under the current policy. A Q-value $q(s,a)$ is the same expectation but conditional on first taking action a .

3. MCTS: build a tree of possible futures starting from the current position, expanding it one leaf at a time and keeping running averages of outcomes at every node. "Monte-Carlo" here refers to averaging sampled evaluations, not to an MCMC chain.

4. The root node is the position where the real decision has to be made. Every other node in the tree is a hypothetical future position.

5. One simulation = one trip from the root down to a new leaf, one network evaluation there, and one update of the statistics along the path. Its cost is roughly one network call plus one step of the game engine (for you, one `fork()`).

6. Exploration means spending effort on options that are not currently estimated to be best, so that the estimate can be corrected if it is wrong. The bonus shrinks as an option accumulates visits, like a confidence interval narrowing with sample size.

7. "Predictor + Upper Confidence bound applied to Trees". It descends from UCB algorithms for bandits, which pick the option with the highest optimistic estimate (mean plus an uncertainty bonus); PUCT scales that bonus by the network's prior.

8. The agent plays games against copies of itself, so the opponent improves as the agent does. Your masked-PPO setup already does this.

9. Random perturbation drawn from a Dirichlet distribution and mixed into the prior π at the root. It is a heuristic for exploration with no guarantee attached.

10. Alternate between evaluating the current policy and producing a better one from that evaluation. Here the search does both locally at each position, and the network is then fitted to the result.

1. What the paper changes

The authors list five places where AlphaZero relies on heuristics that work well with hundreds of simulations but can fail with few:

1. **Which root actions to explore:** Dirichlet noise is replaced by sampling actions without replacement via Gumbel noise.
2. **How to spend simulations at the root:** PUCT is replaced by Sequential Halving, a bandit algorithm built for picking a single best option.
3. **Which move to play:** instead of sampling from visit counts, play the single winner of the search.
4. **What to train the policy toward:** instead of visit counts, a distribution built from the search's value estimates.
5. **How to choose moves deeper in the tree:** instead of PUCT, a deterministic rule that makes visit counts track an improved policy.

The same changes apply to MuZero¹¹, giving "Gumbel AlphaZero" and "Gumbel MuZero".

2. Background: the Gumbel tricks

The Gumbel distribution¹² gives a neat way to sample from a categorical distribution. Suppose the policy is defined by logits¹³. Draw an independent standard Gumbel value $g(a)$ for every action and take

$$A = \operatorname{argmax}_a [g(a) + \operatorname{logit}(a)]$$

Then A is an exact draw from π . Statisticians may recognise this as the random-utility derivation of the multinomial logit model: if each option's utility is a fixed part plus i.i.d. Gumbel error, the chance that option a has the highest utility is exactly its softmax probability.

The Gumbel-Top- k trick extends this: keep the same noise and take the top n actions instead of just the top one. The result is a sample of n distinct actions drawn sequentially without replacement from π (draw one, renormalise over the rest, draw again). The paper writes this set as $\operatorname{argtop}(g + \operatorname{logits}, n)$.

3. Planning at the root

3.1 The problem as a bandit

Strip the problem to its core: at the root there are k actions, each with an unknown value $q(a)$. You may "visit" actions n times, then you must commit to one final action A_{n+1} . This is a bandit¹⁴ with a predictor (the policy network's prior).

11. MuZero is AlphaZero with a learned simulator: instead of calling the real game engine inside search, it uses a network that predicts the next hidden state and reward. False friend: "model" in model-based RL means this transition simulator, not a statistical model of the data in the usual sense.

12. The standard Gumbel distribution has CDF $\exp(-\exp(-x))$. It is the limiting distribution of the maximum of many i.i.d. light-tailed variables, which is why it appears in extreme-value theory.

13. Unnormalised log-probabilities. The probabilities are obtained by softmax: $\pi(a) = \exp(\operatorname{logit}(a)) / \sum_b \exp(\operatorname{logit}(b))$. Adding a constant to all logits changes nothing.

14. A multi-armed bandit is a repeated choice among options with unknown payoffs, where each trial reveals information about the option tried. Think of sequential experimental design for picking the best treatment arm.

Crucially, only the final choice matters. Nothing above the root depends on how good the intermediate test visits were. So the right objective is simple regret¹⁵, not cumulative regret. AlphaZero's PUCT was designed for the latter, which is a mismatch.

The goal is a policy improvement¹⁶:

$$\mathbb{E}[q(A_{n+1})] \geq \sum_a \pi(a) q(a)$$

The right-hand side is the expected value of simply acting with the network. The left is the expected value of what the search chooses. This becomes hard when k is larger than n : 19x19 Go has 362 moves and the paper tests budgets as low as 2.

3.2 A counterexample to the obvious heuristic

The tempting rule "evaluate the n most probable actions and pick the best of them" can be worse than the network. Take three actions with values $q = (0, 0, 1)$ and prior $\pi = (0.5, 0.3, 0.2)$. The network's own expected value is 0.2 (it plays the good action 20% of the time). With $n = 2$, the heuristic only ever looks at actions 0 and 1, both worth 0, so the search's expected value is 0. The search has made the agent strictly worse. Randomly generated q and π vectors turn up plenty of similar failures.

3.3 Planning with Gumbel

The fix has two steps that share the same noise vector g :

1. Pick the candidates: $A_{\text{top}n} = \text{argtop}(g + \text{logits}, n)$. This is sampling n actions from π without replacement (less variance than sampling with replacement, and no wasted duplicate visits).
2. Evaluate those n actions, then choose $A_{n+1} = \text{argmax over } A_{\text{top}n} \text{ of } [g(a) + \text{logit}(a) + \sigma(q(a))]$, where σ is any increasing function.

Why does this guarantee improvement? Without the $\sigma(q)$ term, the argmax over the candidate set is exactly the Gumbel-Max draw from π (the overall top of $g + \text{logits}$ is always in the candidate set). Adding an increasing function of q can only shift the choice toward a higher-valued candidate. So for every realisation of g , the chosen action's value is at least that of the plain policy sample; take expectations and the inequality follows. Reusing the same g in both steps is what makes the comparison pathwise; fresh noise in step 2 would double-count the prior and break the argument.

Keeping the logits in the final score also matters: it lets prior knowledge temper noisy value estimates, which the authors note is useful under partial observability¹⁷.

15. Simple regret: how much worse your final pick is than the best option. Cumulative regret: total loss summed over every trial along the way. PUCT descends from methods that minimise cumulative regret, which spend most trials on the current favourite.

16. A procedure yields policy improvement if the expected value of what it does is at least the expected value of just sampling from the current policy. It is the monotonicity property that makes self-improvement loops safe.

17. Partially observable: the agent does not see the full state (hidden cards in MTG, or off-screen information in some video games). Its observations are then evidence about the state rather than the state itself.

Worked example (MTG flavour). You hold five legal options: pass, play a land, cast Lightning Bolt at the opponent's creature, cast Bolt at their face, cast a creature. Logits from the network: (1.0, 2.0, 1.5, 0.0, 0.5). With budget $n = 2$, draw five Gumbel values, add them to the logits, and keep the top two; say this gives "play land" and "Bolt the creature". Fork twice, roll out or evaluate each, and get estimated values 0.55 and 0.70. The final score of each is its Gumbel-perturbed logit plus $\sigma(\text{value})$. "Bolt the creature" wins unless its perturbed logit was far below "play land". Two forks, and the expected outcome is guaranteed not to be below just sampling from the network, provided those two values were accurate.

3.4 Noisy values: Sequential Halving

In real search, a visit gives only a noisy estimate $\hat{q}(a)$ of the value, so visiting promising actions more than once helps. The paper splits the budget in two ways: first sample m candidates (with $m \leq k$) via Gumbel-Top-k, then use a simple-regret bandit algorithm to divide the n simulations among them.

They chose Sequential Halving¹⁸ because it was easier to tune than alternatives.

The ranking criterion inside each halving round is again $g(a) + \text{logit}(a) + \sigma(\hat{q}(a))$. For example, with $n = 200$ and $m = 16$ there are 4 rounds of about 50 simulations: 16 actions get about 3 visits each, then 8 get about 6, then 4 get about 12, then the final 2 get about 25. The last one standing is played.

The concrete value transform is

$$\sigma(\hat{q}(a)) = (c_{\text{visit}} + \max_b N(b)) \cdot c_{\text{scale}} \cdot \hat{q}(a)$$

with values normalised to $[0, 1]$. The multiplier grows as the most-visited action accumulates visits: early on, the prior dominates; as estimates firm up, values dominate. This is the familiar Bayesian pattern of the likelihood overtaking the prior as data accumulate, though here it is a design choice rather than a derived posterior. Settings of $c_{\text{visit}} = 50$ and $c_{\text{scale}} = 1$ worked across budgets in board games.

4. Learning an improved policy

Once the search has run, what should the network be trained toward? The simplest option is to treat the chosen action as a label: $\text{loss} = -\log \pi(A_{n+1})$. That works, but throws away the values of the other evaluated actions.

The richer option first **completes** the value vector. Visited actions keep their estimated value; unvisited ones are filled in with the policy's own expected value $v_\pi = \sum_a \pi(a)q(a)$, which in practice comes from the value head. Filling in this way says "we have no evidence this action is better or worse than average", giving unvisited actions zero advantage¹⁹. Then build

$$\pi' = \text{softmax}(\text{logits} + \sigma(\text{completedQ}))$$

and train with $\text{KL}(\pi' \parallel \pi)$, i.e. cross-entropy toward π' . Intuitively: start from the network's logits, nudge up actions that searched better than average, nudge down those that searched worse, leave the

18. A best-arm identification algorithm: split the budget into $\log_2(m)$ rounds; in each round give every surviving option the same number of trials, then discard the worse half. It has no tuning constants that depend on the problem.

19. Advantage = $q(a) - v(s)$: how much better action a is than the policy's average. It is the same centred quantity that drives PPO updates.

rest in proportion. The appendix proves this nudged distribution has expected value at least v_π . The argument reduces to one visited action at a time: moving probability toward an action is beneficial exactly when its value exceeds the average of the others, which (when its prior is below 1) is equivalent to exceeding v_π .

This step is a form of distillation²⁰, and the construction is also an instance of regularised policy optimisation²¹.

A practical detail: the raw value head gave a worse fill-in than a mixed estimate that interpolates between the value head and the prior-weighted average of the values actually observed, with the observed part gaining weight as visits accumulate. The authors call it unsophisticated; it is a consistent estimator, and it helped.

5. Planning deeper in the tree

Below the root, the authors also replace PUCT. At every interior node they build the same kind of improved policy π' from completed values. Sampling the next move from π' would add noise to the value estimates being backed up, so instead they choose deterministically, picking the move whose visit share would bring the empirical visit distribution closest (in squared error) to π' . After algebra this becomes

choose $\operatorname{argmax}_a [\pi'(a) - N(a) / (1 + \sum_b N(b))]$

That is: visit the action whose share of visits lags furthest behind its target probability. It is like systematic or quota sampling rather than simple random sampling, which reduces variance for the same budget. At the root they keep the Gumbel randomness, because it makes different games try different actions while preserving the guarantee.

6. Related work in brief

Earlier uses of Sequential Halving in tree search lacked a principled way to fold in a policy prior; here $g + \text{logits}$ plays that role. Earlier analysis had also shown that AlphaZero's visit counts only approximate a regularised policy optimisation, with large error at small budgets. Sampling with replacement, as in Sampled MuZero, is a natural but higher-variance alternative, tested here as an ablation²².

7. Experiments

Agents compared: standard MuZero; Gumbel MuZero (Gumbel root plus completed-Q policy loss); a variant sampling candidates *with* replacement; two variants that use regularised policy optimisation directly inside search with a TRPO-style or MPO-style KL penalty²³; and Full Gumbel MuZero (also

20. Training a network to reproduce the outputs of a slower, better procedure (here, the search), so that the improvement is available at the cost of a single forward pass next time.

21. Choosing a new policy that maximises expected value minus a penalty for straying from the old one (often a KL divergence). The finite value scale σ plays the role of that penalty's strength.

22. An ablation removes or swaps one component of a system to measure its contribution, holding the rest fixed. It is the engineering analogue of a single-factor intervention.

23. TRPO and MPO are policy-optimisation methods that differ mainly in which direction the KL penalty is taken, $\text{KL}(\text{old} \parallel \text{new})$ versus $\text{KL}(\text{new} \parallel \text{old})$. PPO is a simplified relative of TRPO.

changing non-root nodes). Playing strength is reported as Elo²⁴, and training budgets are measured in millions of frames (positions generated).

9x9 Go

Each agent trains with $n = 2, 4, 16, 32$ or 200 simulations per move and is evaluated with 800. The headline result: standard MuZero does not learn at all with 16 or fewer simulations, while Gumbel MuZero learns reliably even at $n = 2$. At 200 simulations all methods are similar. The with-replacement variant is worse at small budgets, confirming that sampling without replacement matters. The TRPO and MPO variants only worked because they too used completed values.

Ablations show that the completed-Q loss beats the simple "label = chosen action" loss, although the simple loss would be adequate for many uses. Sampling only $m = 4$ candidates slowed learning by concentrating the budget too narrowly; the default is $m = \min(n, 16)$. The deterministic non-root rule gave only a small extra gain. Results were insensitive to c_{visit} once it was at least 50. Keeping AlphaZero's habit of exploratory moves (sampling proportional to visit counts) in the first 30 moves of training games still helped.

The payoff is compute: with fewer simulations, self-play needs far fewer accelerators. Training-step throughput rose about 6-fold at $n = 32$ and about 24-fold at $n = 4$, compared to $n = 200$.

19x19 Go and chess

At the full scale of 400 simulations, Gumbel MuZero matched or slightly exceeded MuZero on 19x19 Go, and Gumbel AlphaZero matched AlphaZero on chess. So the new machinery costs nothing where the old one already worked.

Atari

On Ms. Pac-Man, MuZero fails with 4 or fewer simulations; Gumbel MuZero learns from 2. Atari has only 18 actions, so with $n \leq 18$ there is no halving at all: each candidate is visited once and the Gumbel rule picks the winner. That isolates the Gumbel step itself as the source of the improvement.

Because reward scales differ wildly across games, values are normalised by the range seen in the tree and a smaller $c_{\text{scale}} = 0.1$ is used, which bounds how far π' can move from π . On Beam Rider, used as a partially observable example, large value scales hurt performance: trusting noisy values too much and the network's prior too little was harmful. This is the result most relevant to a hidden-information card game.

8. Conclusion

One principle, provable improvement whenever values are right, replaces five of AlphaZero's heuristics. The result learns from two simulations per move, loses nothing at large budgets, and is cheaper to experiment with.

24. A rating where the difference between two players maps to an expected win probability via a logistic curve: a 400-point gap means about 10:1 odds. Here 1000 is anchored to a fixed reference Go program.

Key takeaways

- AlphaZero's search can produce a target worse than the network itself when simulations are scarce; then self-improvement stalls.
- Gumbel-Top-k sampling of candidates, plus choosing the winner by Gumbel + logit + scaled value with the *same* noise, guarantees improvement for any budget, given correct values.
- At the root, optimise simple regret (quality of the final pick) with Sequential Halving rather than PUCT's cumulative-regret logic.
- Train toward $\pi' = \text{softmax}(\text{logits} + \sigma(\text{completedQ}))$, filling unvisited actions with the state value; this trains all actions and is itself a provable improvement.
- For the MTG agent: a budget of 2 to 16 forks per decision can already yield a useful KL distillation target for PPO; mask illegal actions via $-\infty$ logits; keep the value scale modest because determinized values over hidden hands are noisy and biased, and the guarantee only holds for accurate values.

ReBeL: Self-Play and Search When Cards Are Hidden

Noam Brown, Anton Bakhtin, Adam Lerer, Qucheng Gong (2020), *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games*, NeurIPS 2020

In one paragraph

AlphaZero-style programs learn a value function through self-play and use it for lookahead search, both in training and in play. This recipe breaks when players hold hidden information, because a position's worth then depends on how each player would act with every hand they might hold. ReBeL redefines the "state" as the public history plus the probability distribution, known to both players, over everyone's possible private hands. Under that definition the game behaves like a perfect-information game, so the AlphaZero loop can run. The authors prove convergence to a Nash equilibrium in two-player zero-sum games. They also show a simple way to make test-time search safe against exploitation. ReBeL beats strong poker bots and a top human professional at heads-up no-limit hold'em, and it does so with little poker-specific engineering.

Why it matters for the MTG agent

- **It is the principled version of determinized search.** Searching each re-sampled opponent hand as if the cards were face-up ignores the opponent's beliefs. ReBeL shows why that is unsound: an action's value depends on the opponent's policy across *all* hands they might hold.
- **Search-improved policy targets.** The policy network learns to imitate the average policy found by search, which is exactly plan (2). Value targets are bootstrapped from search, and the paper names the bias/variance trade-off this involves.
- **Beliefs as inputs.** The value network takes a distribution over hidden hands as input. That will not scale to MTG hands, but it argues for an opponent head (plan 1) whose output is belief-like, a distribution over what the opponent holds or will do.
- **Unsafe search is exploitable.** If search best-responds to one assumed opponent policy, the result can be punished. This matters as soon as the opponent head feeds into search.
- **Cost.** ReBeL re-solves a small tree hundreds of times per decision and evaluates every leaf on every iteration. With a `fork()` that replays the whole history, building the subgame becomes the bottleneck.

1. Why AlphaZero's recipe fails with hidden information

In chess every position has one correct value: the result under perfect play from there. A program with a learned approximation of that value can search¹ a few moves ahead and choose well. AlphaZero learns the value by self-play² and searches with it during both training and play.

1. Build a small tree of possible future moves, score the positions at the bottom with a learned value function, and pick the best choice within the tree.

2. Training an agent against copies of itself, so the data come from the agent's own current behaviour instead of a fixed dataset.

The paper's opening example shows the failure. Take Rock-Paper-Scissors, but any game involving Scissors pays double. The optimal mix is Rock 40%, Paper 40%, Scissors 20%, and against that mix every action earns zero. Suppose player 1 does one-move lookahead using these equilibrium values. Every move scores zero, so the search has nothing that leads it to 40/40/20. What Rock is worth depends on how often you play Rock.

So with hidden information, a position defined by what has happened so far has no single value. Its value depends on the policies³ that led there. A big bet in poker means one thing from an opponent who bluffs often and another from one who never bluffs.

2. Related work

DeepStack, an earlier poker program, already valued belief distributions. But it trained on randomly generated situations, which the authors compare to learning Go from boards with scattered stones. Pluribus instead precomputed a few "blueprint" strategies and let players choose among them at the search leaves, which needs good blueprints computed without search. ReBeL keeps DeepStack's belief-valued states but learns their values through self-play.

3. Histories, infostates, public states

- **History** h : the complete true sequence of states and actions, hidden cards included.
- **Infostate**⁴ s_i : what player i actually knows.
- **Public state** s_{pub} : what everyone sees. In MTG: battlefield, graveyards, life totals, the visible action log.

In the paper's dice example, each player rolls two dice and shows one. Player 2's infostate fits 6 true histories, and the public state fits 36. The solution concept is **Nash equilibrium**⁵, and the setting is **two-player zero-sum**⁶ games. A **depth-limited subgame** is a small tree running a few actions ahead, with its leaves scored by a value function.

4. From world states to public belief states

The referee thought experiment

Picture a game where each player is dealt one secret card. Now change it so that nobody sees their own card and a referee holds them all. On your turn you announce a full plan, e.g. "with an Ace raise 80%, with a 7 fold 60%", and so on for every card. The referee samples your action from the plan for your real card. Since the plans are public, everyone can update the probability of each card with Bayes' rule.

3. A policy $\pi(a|s)$ is a conditional distribution over actions given what the player knows. With hidden information, optimal policies are usually randomized.

4. Everything one player has observed: the public history plus their own private cards. Many true histories map to one infostate, much as the observed data leave latent variables unresolved.

5. A pair of policies in which neither player gains by changing only their own policy. In two-player zero-sum games an equilibrium policy guarantees at least the game's value against any opponent.

6. Two players, one's gain is the other's loss. This makes the equilibrium value unique, so "optimal" does not depend on who the opponent is. One-on-one MTG qualifies.

Strategically, the two games are identical. The second one, though, contains no private information. Everything that matters is common knowledge⁷. It is a perfect-information game whose states contain probability vectors and whose actions are whole plans.

The public belief state

A state of this transformed game is a **public belief state**⁸ (PBS), written β . In hold'em each player has 1,326 possible two-card hands, so a PBS carries 2,652 probabilities alongside the board and pot. Computing these beliefs exactly assumes both policies are known. That holds in self-play, where both sides run the same network, but not against a real opponent. Section 6 addresses this.

The value of a PBS is an expectation over the true situations it allows:

$$V_i^\pi(\beta) = \sum_h p(h \mid \beta) \cdot v_i^\pi(h)$$

Here $p(h \mid \beta)$ is the belief-implied probability of true history h , and $v_i^\pi(h)$ is player i 's expected payoff from h under policies π . The key fact: in two-player zero-sum games every PBS has a **unique** equilibrium value. Raw positions do not have this property, and it is what makes value learning well-defined.

Tree search over a continuous belief space is hopeless. However, solving a two-player zero-sum subgame is a convex problem⁹. So ReBeL runs an iterative solver on the ordinary card-level tree and never branches over beliefs.

Per-hand values

The solver needs a value for every possible hand inside a PBS:

$$v_i(s_i \mid \beta) = \max_{\pi_i} \sum_{h \in H(s_i)} p(h \mid s_i, \beta_{-i}) \cdot v_i(h)$$

In words: holding hand s_i , average over the opponent's possible hands weighted by the beliefs, with the opponent playing equilibrium and you picking your best policy. Theorem 1 shows these per-hand values are supergradients¹⁰ of the PBS value, so they carry the same information. The network $\hat{v}$ takes a PBS and outputs one value per possible hand for each player.

5. The ReBeL loop

1. Start at the initial PBS: uniform beliefs, nothing public yet.
2. Build a depth-limited subgame. In poker it runs to the end of the current betting round.
3. Run T iterations of an equilibrium solver, scoring the leaves with $\hat{v}$ on every iteration.
4. Store the root's per-hand values as a value target, and optionally the subgame's average policy as a policy target.
5. Sample a leaf, make its PBS the new root, and repeat until the game ends.

7. Everyone knows it, everyone knows that everyone knows it, and so on without end.

8. The public history plus, for each player, a distribution over the infostates they might be in. In effect it is the posterior over all hidden information given the public data and both players' known policies.

9. An optimization with no spurious local optima, so step-by-step improvement methods reach the global solution.

10. The concave counterpart of a subgradient: a slope vector whose tangent plane lies on or above the function. Here it says how the PBS value changes as belief mass moves toward each hand.

5.1 The solver

The default solver is **counterfactual regret minimization**¹¹ in its depth-limited form, CFR-D. On iteration t , CFR proposes policies π^t . The leaf beliefs are what Bayes' rule gives under π^t , and $\hat{v}$ scores the leaves under those beliefs. CFR then updates its regrets¹².

Because the policies change, the leaf beliefs and leaf values change every iteration. This is how ReBeL captures the Rock-Paper-Scissors effect that fixed leaf values miss. CFR's guarantee applies to the *average* policy, so the value target is the root value averaged over the T iterations. An appendix variant, CFR-AVG, scores leaves under the running average policy, which keeps network inputs in a narrower, more realistic region. Fictitious play¹³ is tested as an alternative.

5.2 Generating training data

The network must be accurate on beliefs seen during *all* solver iterations. So the next root is sampled under the policy of a randomly chosen iteration. For exploration¹⁴, one player acts uniformly at random 25% of the time. Theorem 2 shows that, with a perfect function approximator, value error at any reachable PBS shrinks like $C/\sqrt{T}$, inherited from CFR's convergence rate.

The targets are **bootstrapped**¹⁵, since each comes from a search scored by the current network. Training on actual game outcomes is the alternative, and the authors frame the choice as bias versus variance.

5.3 The policy network

A policy network $\Pi(\beta)$ learns to predict the solver's average policy at every PBS in the subgame. It is not needed for correctness. Its role is to warm-start¹⁶ the solver and to keep value-network inputs realistic. This is the pattern behind search-improved policy targets: search acts as the teacher, and the network imitates it.

Determinization versus ReBeL in MTG. The opponent holds up two Islands with three cards in hand. Determinized search samples one world with Counterspell and one without, plays each face-up, and averages. In the first world the opponent always counters. In the second they never hold mana up as a bluff. Neither world captures the real play: holding Islands up whether or not Counterspell is in hand, to keep you guessing. ReBeL's state includes the belief "35% Counterspell", and its solver sets the opponent's policy for every possible hand jointly, so bluffing falls out of the solution.

11. CFR: an iterative solver that shifts probability toward actions that would have done better than the current policy. Its average policy converges to a Nash equilibrium. False friend: "counterfactual" only means "valued as if I had played to reach this point". It has nothing to do with potential outcomes or structural causal models.

12. Per action, the cumulative amount it would have earned beyond what the current policy earned. Positive regret means "do this more often".

13. A classic solver in which each player repeatedly best-responds to the opponent's average policy so far. It converges in two-player zero-sum games, usually more slowly than CFR.

14. Deliberately taking non-preferred actions so the data cover states the current policy rarely reaches.

15. In RL: training a value estimate on targets that themselves use current value estimates. This is unrelated to bootstrap resampling. It has lower variance than waiting for the game result, and it is biased while the network is wrong.

16. Start an iterative method from a good guess so it converges in fewer steps, like initializing an MCMC chain near the mode.

6. Test time without knowing the opponent's policy

Against a real opponent, the true PBS is unknown, yet search must start from one. The natural fix is **unsafe search**¹⁷: solve the opponent's earlier decision, assume they played that solution, and best-respond. In the RPS variant the solver might produce Rock 40.01%, Paper 39.99%, Scissors 20%. The best response to that is pure Paper, which an informed opponent beats with pure Scissors. Earlier safe-search methods added constraints that cost strength, so competitive bots used unsafe search anyway.

ReBeL simply repeats its training procedure: pick a random solver iteration, assume both players used that iteration's policies, and pass down the beliefs they imply. Any single decision may look pure, for example 100% Rock. The randomness over iterations produces an equilibrium *in expectation*. Theorem 3 bounds the distance from equilibrium in terms of the value network's error and T . Linear CFR¹⁸ gives early iterations little weight, which lowers the risk of sampling a poor one.

7. Experimental setup

Quality is measured by **exploitability**¹⁹. The games are heads-up no-limit hold'em (HUNL); turn endgame hold'em (TEH), a reduced version small enough for exact exploitability; and small variants of Liar's Dice. Bets are limited to a few pot fractions, randomly perturbed during training, and unexpected sizes are added to the tree at play time. Both networks are plain MLPs. The value net uses a Huber loss²⁰ and outputs one value per hand. There is no card bucketing and no precomputed equities. Data generation was the bottleneck: one training machine was fed by up to 128 eight-GPU machines generating self-play.

8. Results

TEH. With 250 solver iterations per subgame, ReBeL matched the exploitability of roughly 125 iterations of tabular CFR²¹ on the full game. Top poker bots typically use 100 to 1,000. A value network trained on randomly sampled beliefs learned essentially nothing. Adding the policy network helped further. Self-play matters because it focuses training on beliefs that actually occur.

HUNL, in milli-big-blinds per hand²² ($\pm$ one SD):

Opponent	ReBeL's win rate
Slumbot (former champion bot)	+45 $\pm$ 5
BabyTartanian8 (former champion bot)	+9 $\pm$ 4
Local best response exploiter	+881 $\pm$ 94
Dong Kim, top pro, 7,500 hands	+165 $\pm$ 69

17. Assume the opponent played one particular computed policy, treat the resulting beliefs as true, and best-respond. An opponent who can anticipate this can exploit it.

18. A CFR variant that weights iteration t in proportion to t , so poor early iterations count little.

19. How much a perfect adversary who knows your policy would win against it. Zero means exact equilibrium. It is a worst-case measure, not an average win rate.

20. Quadratic for small errors and linear for large ones. It is the robust-regression M-estimator, which limits the pull of outliers.

21. "Tabular" means a separate stored number for every infostate, with no network and no generalization. It is exact but only feasible for games that fit in memory.

22. Thousandths of the big blind won per hand, the standard poker win-rate unit.

The human match used AIVAT²³ to tame poker's enormous luck. ReBeL averaged under 2 seconds per hand. In an appendix ablation, modified CFR-AVG beat BabyTartanian8 while plain CFR-D lost to it narrowly.

Liar's Dice. ReBeL with depth-2 subgames reached exploitability of about 0.015 to 0.04, against about 0.001 to 0.002 for full-game tabular CFR. ReBeL is less exact, but it keeps working after the game grows too large to tabulate.

9. Limitations

The network input holds a probability for every infostate consistent with the public state. That is fine for hold'em's 1,326 hands but impossible in games with little common knowledge. Magic, where a hidden hand of up to seven cards comes from a 60-card library, falls squarely in that class. Theorem 1 suggests a scalar PBS value is enough in principle, which offers a possible route around this. The guarantees also cover only two-player zero-sum games.

Key takeaways

- With hidden information, a position's value depends on the policies that led to it. Determinized, face-up search ignores this and can miss bluffing entirely.
- Make the state the public history plus the shared posterior over hidden information. In two-player zero-sum games, this belief state has a unique, learnable value.
- The loop: solve a small lookahead tree with CFR, score leaves with a value net that takes beliefs as input, train on search outputs (values, and optionally the average policy), then step to a sampled leaf.
- Training on random beliefs fails. Self-play focuses learning on beliefs that actually arise.
- Best-responding to one assumed opponent policy is exploitable. Using the beliefs from a randomly chosen solver iteration is cheap and provably safe.
- For MTG the exact method does not scale, but several principles carry over: search-derived policy targets, bootstrapped values with a bias/variance trade-off, belief-like opponent modelling, and fast subgame construction.

23. A control-variate estimator that subtracts the expected value of card luck and of the bot's own known randomization. It stays unbiased while cutting variance sharply.

PerfectDou: a critic that cheats, a player that doesn't

Guan Yang, Minghuan Liu, Weijun Hong, Weinan Zhang, Fei Fang, Guangjun Zeng, Yue Lin (2022), *PerfectDou: Dominating DouDizhu with Perfect Information Distillation*, NeurIPS 2022

In one paragraph

PerfectDou is a bot for DouDizhu, a three-player Chinese card game with hidden hands. Its central trick is simple: during training, the part of the network that *judges* positions (the critic) is allowed to see every player's cards, while the part that *chooses* moves (the policy) only ever sees what a real player would see. Add a dense training reward computed from everyone's hidden cards and richer card features. Trained with PPO self-play on a modest cluster, PerfectDou beats every earlier DouDizhu program, including the previous best (DouZero), while using roughly a tenth of the training samples. It does no search at test time and needs about 6 ms per move.

Why it matters for the MTG agent

- **Nearly free improvement to masked PPO.** The engine already knows the opponent's hand and library order. Feeding that to the value head only (never the policy) is a small code change. It needs no `fork()`, no determinization and no search, and the paper's ablation shows it measurably helps.
- **A baseline before search.** PerfectDou reaches the top with zero test-time search, so try this before paying for determinized search with history replay.
- **Dense reward from hidden information.** Their "distance to winning" reward is computed from all hands and is only used in training. An MTG version could be a clock or tempo estimate, written as a difference between consecutive turns so it does not change what the best policy is.
- **Legal-action scoring.** The policy embeds each legal move, joins it with the state embedding, and scores each pair. This is the same shape as a transformer over board objects with a pointer-style output, and it handles a variable number of legal moves naturally.
- **Opponent modelling.** PerfectDou does not predict opponent actions (related DouDizhu work that did found small gains). The privileged critic is a cheaper, complementary way to inject opponent knowledge.
- **Evaluation design.** Deals are replayed with seats swapped; for MTG, mirror deck and play/draw assignments.

1. The problem and the game

Game-playing programs built with reinforcement learning¹ have mastered Go, chess, poker and several video games. DouDizhu is harder in three specific ways.

- **Hidden information.** You see your own hand and every card played, but not what the others hold.
- **Mixed teams.** One player, the *Landlord*, plays against two *Peasants*, who win or lose together but cannot talk to each other.
- **A huge action space.** There are 27,472 distinct card combinations that can ever be played, and often hundreds are legal at once.

DouDizhu in five lines. A 54-card deck (two jokers included) is dealt 17 cards per player. The Landlord also takes three extra face-down "bottom" cards. Play goes around the table in rounds. The leader plays a combination: a single card, a pair, a trio, a run of five or more, a run of pairs, a trio with an attached spare card (the spare is called a *kicker*), and so on. Each following player must either pass or beat it with a higher combination of the same type. A *bomb* (four of a kind) or the *rocket* (both jokers) beats anything, and every bomb doubles the stakes. When two players in a row pass, whoever played last leads the next round. The first player to empty their hand wins for their side. Suits are irrelevant; only rank matters.

Greedy play loses: splitting a run to answer one trick can strand unplayable singletons, like spending removal early and having nothing for the real threat. And because bombs multiply the score, winning *often* differs from winning *points*. Hence two metrics: WP , the fraction of games won, and ADP , the average score margin per game, which is what they train for.

The previous best, DouZero, used Deep Monte-Carlo² learning and had recognisable weak spots.

2. Preliminaries: games as trees with blurry vision

The paper formalises the game as an imperfect-information extensive-form game³. The key object is the information set⁴ (info set). Two nodes in the same info set look identical to the acting player, so the player must act the same way at both.

A policy⁵ π_p for player p maps each info set to a probability vector over legal moves. Each player wants to maximise expected final payoff, $R_p = E_{Z \sim \pi} [r_p(Z)]$, where Z is the leaf the game ends in. The paper considers only the card-play phase and skips the bidding that decides who becomes Landlord. The

1. Learning a decision rule by trial and error: an agent acts, receives numerical rewards, and adjusts its behaviour to maximise expected total reward. No labelled "correct moves" are supplied.

2. A value-learning method: play whole games, then regress the final outcome on each (state, action) pair seen along the way. It is essentially an unbiased but high-variance Monte Carlo estimator of each action's value. Moves are picked by maximising the predicted value.

3. A game written as a tree: each node is a full history of moves, including chance moves like the deal. Each node has one player to act, and leaves carry payoffs. "Imperfect information" means some branches, such as other players' deals, are invisible to some players.

4. The set of tree nodes a player cannot tell apart given what they have observed. In poker, all deals consistent with your hole cards and the betting so far form one information set. A statistician can read it as the event you are conditioning on, with the hidden cards still random within it.

5. The agent's decision rule, written $\pi(a | s)$: a conditional probability distribution over actions given what the agent observes. Training means fitting the parameters of this conditional distribution.

path to any node therefore includes everyone's starting hand plus all moves, while each player's info set contains only their own hand plus the public moves.

3. Method: perfect information distillation

3.1 The idea

A player who could peek at every hand would play better. The question is how to exploit that "cheating" game in training without the final player depending on it.

Their answer is a framework called Perfect-Training-Imperfect-Execution (PTIE). It is a special case of centralised training, decentralised execution⁶, and it is built on an actor-critic⁷ learner.

Plain policy gradient⁸ updates the policy parameters θ in the direction

$$\nabla_{\theta} J = E_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a | s) \cdot Q^{\pi}(s, a)],$$

where $Q^{\pi}(s, a)$ is the action value⁹: the expected return of playing a in s and following π afterwards. In words, make moves that turned out well more likely, weighted by how well they turned out.

The observation behind PTIE is that only the actor is used at play time. The critic is training scaffolding. Nothing stops the critic from seeing the true node, meaning all hidden cards, as long as the actor does not. Write $D(h)$ for the "fully revealed" version of node h . The modified gradient is

$$\nabla_{\theta} J = E_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a | h) \cdot Q^{\pi}(D(h), a)].$$

The policy term still conditions only on what the player sees. The value term conditions on everything.

The authors call this distillation¹⁰, because the perfect information is "boiled down" into a policy that never sees it directly.

Why this is statistically sound (our gloss)

The paper argues informally. The point is worth making precise for a statistician. The policy's action is drawn from $\pi(a | \text{info set})$, so given the info set, the action is independent of the hidden cards. Averaging the perfect-information value $Q(D(h), a)$ over the hidden cards, weighted by how likely each deal is given what you see, gives back the ordinary info set value. That is just the law of total expectation. Sampled games provide that weighting automatically. The paper puts this as the policy receiving "an expected value estimation on each info set".

6. A common recipe in multi-agent learning: during training a central component may see all agents' observations; at deployment each agent acts only on its own observation. PTIE applies the same idea to hidden cards in a card game.

7. A learning setup with two learned functions: the actor (the policy, which picks moves) and the critic (a value function, which predicts how good a situation or move is). The critic exists only to produce better training signals for the actor.

8. Optimising the policy's parameters by stochastic gradient ascent on expected reward. The gradient estimator is the score-function (likelihood-ratio) estimator: increase the log-probability of actions in proportion to how well they turned out.

9. The expected total future reward if you take action a in state s and then follow policy π . It is a conditional expectation of the return given (s, a) , and the critic approximates it with a neural network.

10. In machine learning, "distillation" usually means training a student network to imitate a teacher network's outputs. Here the word is used loosely: knowledge from the all-seeing critic flows into the blind policy only through the training signal. No student copies a teacher.

There is a familiar analogy. In PPO the critic mostly serves as a baseline¹¹. A baseline that knows the hidden cards works like regression adjustment in a randomised trial with a strongly prognostic pre-treatment covariate. The "treatment" (the action) is randomised by the policy independently of that covariate given the info set. Adjusting for the covariate therefore costs no bias and can remove a lot of noise. In DouDizhu the deal explains a large share of the outcome, so the gain is large.

One caveat: the implicit "posterior" over hidden cards is induced by the current self-play opponents' actual play.

Self-play¹² reinforcement learning is known to approach a Nash equilibrium¹³ in some two-player settings. They state openly that no such guarantee exists for three players. PTIE is justified by its results, not by a theorem.

4. System design

4.1 Card representation

Each set of cards is a 12×15 binary matrix, with one column per rank (3 through 2, plus the two jokers). The first four rows count how many copies of each rank you hold, as in DouZero. The new part is eight extra rows that flag which combinations the cards can form: solo, pair, trio, bomb, rocket, run of singles, run of pairs, run of trios.

Spelling these out saves the network from rediscovering them; the MTG analogue is precomputed features like "can attack this turn" or "mana available after this land drop".

4.2 Two views of the same node

The policy gets the **imperfect features**: your hand, cards not yet seen, each player's played cards, the three bottom cards, the last 15 moves, each opponent's most recent move, card counts, bomb count, and whether you currently "control" the round. A last item is the minimum number of plays needed to empty your own hand. In total that is 23 card matrices plus 6 scalars.

The critic gets all of that plus the two other players' actual hands and their minimum-plays-to-empty, for 25 matrices plus 8 scalars. The policy's inputs are a strict subset of the critic's.

4.3 Networks and action scoring

The learner is PPO¹⁴ with GAE¹⁵. So the critic actually learns a state value V , and the policy is trained on the advantage¹⁶ $A = R - V$.

11. A quantity subtracted from the return in the policy-gradient estimator to cut variance. Because it does not depend on the action, it leaves the gradient's expectation unchanged. It is exactly a control variate.

12. Training an agent by having it play against copies (current or past) of itself, so the opposition improves as the agent does.

13. A set of strategies, one per player, such that no player can gain by changing only their own strategy. In two-player zero-sum games it is the "unexploitable" solution. With three players, as in DouDizhu, it is less well-behaved and carries no comparable guarantee.

14. Proximal Policy Optimization, the standard on-policy method. It repeatedly collects games, then takes several gradient steps that are clipped so the new policy's action probabilities cannot drift far from the old ones. The clipped quantity is an importance-sampling ratio between new and old policy.

15. Generalized Advantage Estimation: a way of estimating how much better an action was than expected, by blending one-step and multi-step return estimates with a decay weight λ . Small λ gives low variance but more bias from the critic; $\lambda = 1$ gives the unbiased but noisy Monte Carlo return. A classic bias-variance dial.

16. The advantage $A(s, a) = Q(s, a) - V(s)$: how much better this action is than the policy's average action in that state. It is the baseline-subtracted return.

The critic is a plain MLP¹⁷. It takes the shared imperfect encoding plus a separate encoding of the perfect-only features and outputs one number.

The policy first runs an LSTM¹⁸ over the move history to build a state embedding e_s . Each legal move a_i also gets its own embedding e_{a_i} . That embedding comes from its card matrix plus handy flags: is it a bomb, is it the largest possible play, does it match an opponent's remaining card count, how many plays would remain after it. The move probabilities are then

$$p(a_i) = \text{softmax}_i(f([e_s, e_{a_i}])) , \text{ for } i = 1 \dots N \text{ legal moves,}$$

where $[\cdot, \cdot]$ means concatenation and f is an MLP producing one score per pair. Softmax¹⁹ runs only over the legal moves, so illegal moves get zero probability by construction. This is masking done structurally.

This resembles target attention²⁰. For MTG, it is the natural output head for a transformer over board objects.

To keep the output layer manageable, they shrink the 27,472 raw moves to 621 abstract ones by merging moves that share the same main cards and differ only in kickers. This is an action abstraction²¹. After the policy picks an abstract move, a small heuristic chooses which spare cards to attach, preferring low cards that are used in few other combinations.

4.4 Perfect reward design

Final-score rewards are sparse²². PTIE allows a richer training reward, because a reward can depend on hidden cards as long as only the critic and the training targets use it.

The authors use an oracle²³: a dynamic-programming²⁴ solver that returns N , the minimum number of plays a player needs to empty their hand, a rough "distance to winning". Each side's position is then summarised as

$$\text{Adv}_t = N_t^{\text{Landlord}} - \min(N_t^{\text{Peasant 1}}, N_t^{\text{Peasant 2}}).$$

Positive values favour the Peasants. Taking the minimum over the two Peasants matters: their team is as close to winning as its closer member. That is what pushes Peasants to help each other.

17. Multi-layer perceptron: a stack of fully connected layers with nonlinearities. It is the simplest neural network and works like a flexible nonlinear regression.

18. Long short-term memory network: a recurrent neural network that reads a sequence step by step and keeps a running summary, here of the recent move history.

19. The function that turns a vector of real scores into probabilities: exponentiate each score and normalise. It has the same form as a multinomial-logit choice model.

20. An attention mechanism in which a query (here, the game state) is compared with a set of candidate items (here, the legal actions) to produce a score for each. This is the pointer-style output that transformer agents use to pick among a variable set of objects.

21. Replacing the true action set with a smaller set of representative actions, then mapping each choice back to a concrete action. It trades some precision for a much smaller learning problem.

22. A sparse reward is non-zero only rarely, here only at the end of a 30-plus-move game. That makes it hard to work out which of many moves deserve the credit or blame.

23. In this paper, an exact helper computation that uses information the player does not have. It is a dynamic-programming routine, not a learned model.

24. An algorithm that solves a problem by breaking it into overlapping subproblems and caching their answers. Here it finds the fewest plays needed to empty a hand.

The step reward is the change in this quantity, scaled by a constant l (set to 50):

$r_t = -1.0 \times (\text{Adv}_t - \text{Adv}_{t-1}) \times l$ for the Landlord, and $r_t = 0.5 \times (\text{Adv}_t - \text{Adv}_{t-1}) \times l$ for each Peasant.

Worked example. The Landlord needs 5 plays and the Peasants need 3 and 7. Then $\text{Adv} = 5 - 3 = 2$, so the Peasants are ahead. The Landlord now plays a strong combination nobody can beat, so its distance drops and Adv falls to 1. $\text{Change} = -1$. The Landlord's reward is $-1.0 \times (-1) \times 50 = +50$, and each Peasant receives $0.5 \times (-1) \times 50 = -25$. The weights -1.0 and 0.5 keep the sums zero across the table, and the two Peasants split the team's share.

Because each reward is a difference between consecutive steps, the rewards telescope. Their sum over a game depends only on the start and end values. In the reinforcement learning literature this is potential-based reward shaping²⁵, a form known to leave the best policy unchanged. The paper does not use that term. To be safe, in the final stage of training they drop the shaped reward entirely and fine-tune on the score margin alone.

4.5 Distributed training

CPU rollout workers (880 cores) play self-play games and stream experience to 8 GPU learners, which average gradients and push fresh weights back every 24 moves (8 per seat).

Three separate networks are kept, one for the Landlord and one for each Peasant seat. Each is trained only on its own seat's data, against the latest versions of the other two. Notable settings are no discounting ($\gamma = 1$), GAE $\lambda = 0.95$, entropy bonus²⁶ weight 0.1, and a batch size of 1024.

5. Related work, briefly

The StarCraft agent AlphaStar also gave its value network hidden state. The Mahjong program Suphx instead let the *policy* see hidden tiles early in training and gradually removed them; PTIE never shows them to the policy, so nothing has to be weaned off.

Poker solvers based on counterfactual regret minimization²⁷ handle tiny action menus well. They lack guarantees beyond two players and do not scale to DouDizhu's move set. Earlier DouDizhu programs combined Monte Carlo tree search²⁸ with Bayesian guesses of hidden cards (DeltaDou), or explicitly modelled opponents' actions. The latter gave only small gains on top of DouZero.

25. Adding $F(s') - F(s)$ to each step's reward for some function F of the state. Over a whole episode these terms cancel except at the ends, so with no discounting the ranking of policies is unchanged. Only the timing of the feedback changes.

26. A term added to the objective that rewards the policy for keeping its action distribution spread out. It discourages collapsing to deterministic play too early and keeps exploration going.

27. CFR: an iterative algorithm that, at each information set, tracks how much better each action would have done than the current mix and shifts probability toward high-regret actions. "Counterfactual" here only means "the value had I played this action instead, weighting by the chance of reaching this spot". It has nothing to do with causal-inference counterfactuals or potential outcomes.

28. A planning method that, at decision time, simulates many possible continuations from the current position, building a tree of the most promising lines and averaging outcomes. In hidden-information games it is usually run over sampled guesses of the hidden cards, which is determinization.

6. Experiments

6.1 Setup

Opponents were DouZero, DeltaDou, CQN (a Q-learning program with action decomposition), the rule-based RHCP-v2, a simple rule bot, and a random player. All used their public code and weights.

Each pairing plays 10,000 random deals. Each deal is played *twice*, swapping which program is Landlord and which plays the Peasants. Statistically this is a matched-pairs design. The luck of the deal, by far the largest source of variance, is shared between the two programs and largely cancels.

6.2 Head-to-head results

PerfectDou vs.	WP	ADP
DouZero (public model)	0.543	+0.14
DeltaDou	0.584	+0.42
RHCP-v2	0.543	+0.51
CQN	0.862	+2.09
Random	0.994	+3.15

PerfectDou beats every opponent on both metrics. One side finding: the public DouZero model failed to beat RHCP-v2 (WP about 0.45), contrary to what DouZero's own paper reported.

Some programs show high WP but negative ADP: they dump strong cards early, win many cheap games and lose expensive ones.

Sample efficiency. After 10^9 training steps, PerfectDou beats a DouZero trained for 10^9 steps (WP 0.732, ADP +1.27). It even edges out a DouZero trained for 10^{10} steps (WP 0.524, ADP +0.01). The final 2.5×10^9 -step model beats the 10^{10} DouZero at WP 0.541, ADP +0.13. That is roughly a tenfold saving in samples. Against skilled humans over 1,260 games, PerfectDou reached WP 0.625 and ADP +0.59.

6.3 Ablations

The ablations²⁹ each train a variant for 10^9 steps and play it against the strong 10^{10} -step DouZero over 100,000 deals.

Variant	What is removed	WP	ADP
PerfectDou	nothing	0.524	+0.014
ImperfectDou	critic sees only imperfect features	0.486	-0.057
RewardlessDou	no shaped reward	0.540	-0.201
Vanilla PPO	both, plus feature design	0.346	-0.709

- **The privileged critic** is worth roughly 4 points of win rate and moves the score margin from negative to slightly positive.
- **The shaped reward** barely moves win rate, but without it the agent gives up many points. It wins slightly more games and loses big ones. The shaping teaches it to care how it wins.

29. Experiments that remove one component at a time to measure its contribution, like dropping a term from a factorial design. These are single training runs, not replicated across random seeds.

- **Removing everything** leaves a much weaker agent.
- **The features alone are not the explanation.** Giving DouZero PerfectDou's richer features ("ImperfectDouZero") made it slightly *worse*.

A statistician should temper this. With 100,000 paired deals, the sampling standard error of a win rate near 0.5 is about 0.002, so 0.524 versus 0.486 is clearly beyond deal-to-deal noise. But each variant is one training run, and run-to-run variation in reinforcement learning is often larger than these gaps. No intervals across seeds are reported.

6.4 Speed

Here "inference"³⁰ means choosing a move. PerfectDou needs about 6 ms per move on one CPU core, against 2 ms for DouZero; the gap comes mostly from feature preprocessing. Search-based DeltaDou and the heavy CQN are orders of magnitude slower.

6.5 Behaviour

As Landlord, PerfectDou bombs less and holds control of rounds longer (about 13 rounds per game versus 9 for DouZero). As Peasants, the two copies bomb more to break the Landlord's control, especially the seat right after the Landlord, as human experts do. In hand-picked examples DouZero strands low pairs or splits its rocket in panic, while PerfectDou blocks early and saves its bomb. The authors credit the critic and the shaped reward; these are anecdotes, not controlled findings.

7. Conclusion

The authors pitch PTIE as a general recipe for hidden-information games, though they test only DouDizhu.

Key takeaways

- Only the policy has to respect hidden information. The critic, the reward and any auxiliary targets used only in training may use the full game state.
- A critic that sees hidden state acts as a strong control variate. It stays unbiased because the policy's action, given what the player sees, is independent of the hidden cards. It cuts variance most when the hidden state strongly predicts the outcome, as a dealt hand does.
- Dense rewards from a hidden-state oracle speed learning a lot. Writing them as step-to-step differences, and switching them off for a final fine-tune, keeps the true objective intact.
- Scoring each legal action by joining its embedding with the state embedding handles large, variable action sets cleanly. It is a natural fit for a transformer over board objects.
- No test-time search was needed to reach state-of-the-art play. For the MTG agent, a privileged critic is the cheap first step, to try before determinized search with costly `fork()` replays.
- Evaluate with paired deals and swapped seats. Report both win rate and score. Treat single-seed ablations as suggestive, not conclusive.

30. False friend. In machine learning, "inference time" means running the trained network forward to choose a move, not statistical inference.

Invalid Action Masking

Shengyi Huang and Santiago Ontañón (2020; revised 2022), *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*, arXiv:2006.14171 (FLAIRS 2022)

In one paragraph

Game agents usually pick from one big, fixed menu of actions, but in any given position most entries on that menu are illegal. The common engineering fix is to "mask" them: set their scores to a huge negative number before turning scores into probabilities, so illegal moves get probability zero. This short paper asks whether that trick is mathematically sound and how much it matters. The answer to the first question is yes: masking is just a deterministic, state-dependent transformation inside the policy, so the usual policy-gradient theory still applies, and the masked gradient leaves the illegal actions' parameters untouched. On the empirical side, in a small real-time strategy game where the share of illegal actions grows with map size, masking keeps learning fast and stable, while the alternative of punishing illegal moves with negative reward collapses on bigger maps. A "naive" variant that samples with the mask but computes gradients without it is shown to make the policy lurch unstably between updates.

Why it matters for the MTG agent

- It is the theoretical licence for the agent's current design. In Pauper, the legal options at a priority window (which land to play, which spell to cast, which creature to block with) are a tiny, constantly changing subset of everything the network can output. Masked PPO is exactly the setup this paper validates.
- It pins down an implementation detail that is easy to get wrong: the mask must be stored with each transition and reapplied when PPO recomputes probabilities at update time. Sampling with the mask but computing log-probabilities without it is the paper's "naive" variant, which trains erratically.
- It warns against reward-shaping substitutes. Giving negative reward for illegal choices fails to scale as the illegal share grows, and MTG's illegal share is enormous.
- It matters for the planned extensions. When distilling search-improved policy targets, or training an auxiliary head that predicts the opponent's next action, the predicted distribution should be the *masked* one, so that probability mass and gradient are only spent on actions that were actually legal.
- It shows the masked policy still behaves sensibly if the mask is removed, but performance degrades as the action space grows. Do not rely on the network having "learned the rules"; always provide the mask, including at inference and inside search rollouts.

Primer: policy gradients and PPO in five minutes

The paper assumes you already know reinforcement learning¹. Here is the minimum needed.

1. A framework in which an agent learns by trial and error: it observes a situation, takes an action, receives a numeric reward, and adjusts its behaviour to collect more total reward over time. There is no labelled "correct action" as in supervised learning.

States, actions, policies

At each step the agent sees a state s (in MTG: board, own hand, life totals, stack) and chooses an action a . Its behaviour is a policy² $\pi_\theta(a|s)$, a probability distribution over actions given the state, with parameters θ (the neural network weights). The environment then returns a reward r and the next state. A whole game is a trajectory³ τ . The formal wrapper is a Markov decision process⁴.

The network outputs one unnormalised score per action, called a logit⁵ l_i , and the softmax converts them to probabilities:

$$\pi_\theta(a_i|s) = \exp(l_i) / \sum_j \exp(l_j).$$

The objective and its gradient

The goal is to maximise expected discounted return $J = E_\tau[\sum_t \gamma^t r_t]$, where the discount factor⁶ γ down-weights the future. The expectation is over trajectories generated by the policy itself, so the parameters affect the distribution we average over. The policy gradient theorem⁷ resolves this:

$$\nabla_\theta J = E_{\tau \sim \pi_\theta}[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t], \text{ where } G_t = \sum_{k \geq 0} \gamma^k r_{t+k}.$$

In words: for every action actually taken, compute the score (gradient of its log-probability) and weight it by the return that followed, G_t . Actions followed by good outcomes become more likely; bad ones less likely. A statistician will recognise $\nabla \log \pi$ as the score function, and the whole thing as a Monte Carlo estimator: unbiased but high-variance. Practical algorithms reduce variance by replacing G_t with an advantage⁸ estimate, using a learned value function⁹.

PPO

Plain policy gradient takes one gradient step per batch of games, which wastes data. Proximal Policy Optimization¹⁰ reuses each batch for several epochs. Once the policy has moved, the data are slightly "off-policy", so PPO reweights by the ratio

$$\rho_t = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t),$$

2. The conditional distribution of actions given the state, $\pi(a|s)$. Think of it as a multinomial regression whose covariates are the state and whose outcome categories are the possible actions.

3. The sequence of states, actions and rewards from start to finish of one episode (one game): $s_0, a_0, r_0, s_1, a_1, r_1, \dots$

4. A model of the environment as a set of states, actions, transition probabilities $P(s'|s,a)$ and rewards, where the next state depends only on the current state and action. MTG with hidden hands is strictly not Markov from one player's view, which is one reason game agents carry history or beliefs.

5. The pre-softmax score for an action. Same meaning as the linear predictor in multinomial logistic regression: probabilities are proportional to $\exp(\text{logit})$.

6. A number γ slightly below 1 (0.99 here) that weights future rewards less than immediate ones, like a discount rate in finance. It also keeps infinite sums finite.

7. The result that the gradient of expected return can be written as an expectation of (score function) $\times$ (return), so it can be estimated from sampled games without differentiating through the environment. It is the same identity as the score-function or likelihood-ratio gradient estimator in statistics.

8. How much better an action turned out than the policy's average in that state: roughly G_t minus a baseline $V(s_t)$. Subtracting a baseline is a control-variate trick; it leaves the expectation unchanged but cuts variance.

9. A second network head that predicts the expected return from a state, $V(s)$. It is a regression of observed returns on states and serves as the baseline above.

10. PPO: the workhorse policy-gradient algorithm. It collects a batch of games with the current policy, then takes several gradient steps on that batch while preventing the policy from moving too far from the one that generated the data.

which is exactly an importance-sampling weight¹¹. It then maximises $E[\min(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t)]$, with $\epsilon = 0.2$. The clip¹² removes any incentive to move the probability of an action by more than about 20% per update. To monitor how far the policy moves, practitioners track the KL divergence between the old and new policy; a sudden spike signals unstable training. Keep that diagnostic in mind; it is the paper's key evidence against the naive variant.

The problem: one menu, many illegal entries

Neural networks want a fixed output size. But games have legal-move sets that change from state to state: five options now, seven next turn. The standard solution is to define one full discrete action space¹³ containing everything that could ever be legal, and have the network score all of it. The catch is that a sample from this full menu is often illegal. In MTG terms: the network might choose "cast Lightning Bolt" when Bolt is not in hand, or "attack with" a creature that is tapped.

When the full menu is huge, almost every random sample is illegal. The usual fix in large game agents is invalid action masking¹⁴, but before this paper it was mostly a one-sentence implementation detail with no theory behind it.

How masking works, and why it is still a valid gradient

A toy example

Setup. One decision, four actions $a_0 \dots a_3$, all logits equal to 1, reward always +1, game ends immediately. Softmax gives probabilities [0.25, 0.25, 0.25, 0.25].

Unmasked update. Suppose a_0 is drawn. The gradient of $\log \pi(a_0)$ with respect to the four logits is "1 minus own probability" for the chosen action and "minus probability" for the others: [0.75, -0.25, -0.25, -0.25]. Times return 1, that is the update direction.

Masked update. Now a_2 is illegal. Replace its logit by $M = -10^8$. The softmax over [1, 1, M, 1] gives [0.33, 0.33, ≈ 0 , 0.33]. If a_0 is drawn, the gradient is [0.67, -0.33, 0, -0.33].

Lesson. Masking does more than renormalise the probabilities over the legal three. It also sets the gradient on the illegal action's logit to zero. The network gets no signal at all about a_2 from this state, which is what we want: there is nothing to learn about a move you could not make.

11. The ratio of target to sampling density used to reweight draws from one distribution so they estimate expectations under another. Here the sampling distribution is the old policy and the target is the current one.

12. Truncating the importance ratio to $[1-\epsilon, 1+\epsilon]$ so that no single sample can push the policy far. Analogous to weight truncation in importance sampling: it trades a little bias for much lower variance and stability.

13. The union of every action that could ever be legal in any state, given a fixed index. The network always outputs one logit per entry, whether or not it is legal now.

14. Replacing the logits of illegal actions with a very large negative constant before the softmax, so their probabilities become effectively zero and they are never sampled. The legal set is supplied by the game engine, not learned.

The proposition

One might worry: the mask comes from the game engine, outside the network. Does the policy gradient theorem still hold when an external process edits the distribution? The authors write the masked policy as

$$\pi'_\theta(\cdot|s) = \text{softmax}(\text{mask}_s(l(s))), \text{ with } \text{mask}_s(l)_i = l_i \text{ if } a_i \text{ is legal in } s, \text{ and } M \text{ otherwise.}$$

For each coordinate, mask_s is either the identity or a constant. Both are differentiable, so π'_θ is a differentiable function of θ , which is all the theorem requires. Hence the masked gradient is the honest policy gradient of the masked policy π' . The authors call this a state-dependent differentiable function: the same logit vector is transformed differently in states with different legal sets.

A clean way to see it as a statistician: the mask is part of the model, not a post-hoc filter. You are simply fitting a multinomial-logit policy whose support depends on a known, observed covariate (the legal set). The gradient is unbiased for that model's objective. Nothing about the mask needs to be learned, because the engine supplies it exactly.

Experimental setup

The game

The testbed is μRTS ¹⁵. The task is simple: one player controls a base and a worker next to resources, and earns +1 whenever the worker harvests and another +1 when it returns the resource to base. Games last at most 200 steps. Maps are 4×4 , 10×10 , 16×16 and 24×24 .

The state is encoded as an image-like grid with 27 binary feature planes¹⁶ per cell. An action is a vector of eight discrete components: which unit acts (any of the $h\times w$ cells), the action type (no-op, move, harvest, return, produce, attack), direction parameters for each type, a unit type to produce, and an attack target cell.

Why the illegal share grows

The "which unit" component has one entry per map cell, but only about two cells (base and worker) ever hold a usable unit. Choosing a legal unit at random has probability $2/16 = 12.5\%$ on the 4×4 map but $2/576 \approx 0.35\%$ on the 24×24 map. So map size is a dial on how much of the menu is illegal, without changing the actual task.

A factorised action head

Scoring every combination of eight components would need roughly $9,216 \times (h\cdot w)^2$ outputs. Instead the network outputs a separate softmax for each component and treats them as independent given the state, so the joint log-probability is the sum of eight component log-probabilities. This multi-discrete¹⁷ trick needs only $2\cdot h\cdot w + 36$ logits. The mask in the experiments is applied only to the "which unit"

15. A deliberately minimal real-time strategy game used in game-agent research: a grid map with bases, workers, resources and combat units, with simultaneous and multi-step actions. Here it is used only for a simple harvesting task.

16. One-hot channels laid over the map: for each cell, indicators for hit points, carried resources, owner, unit type and current action. A convolutional network reads them like colour channels in an image.

17. An action space made of several discrete sub-choices, each with its own softmax, whose probabilities are multiplied. It assumes the sub-choices are conditionally independent given the state, which is a strong simplification when the legal target depends on the chosen action.

and "attack target" components, so some illegal combinations (a wrong direction, say) can still be sampled.

Four ways to handle illegal actions

1. **Penalty.** No mask; an illegal action earns an extra reward $r_{\text{invalid}} \in \{0, -0.01, -0.1, -1\}$. This is reward shaping¹⁸: the agent is supposed to learn the rules from punishment.
2. **Masking.** The proper method: sample from π' and compute gradients with π' .
3. **Naive masking.** Sample from the masked π' , but compute the gradient with the unmasked π . Illegal actions are never executed, but their logits still receive gradient.
4. **Masking removed.** Train exactly as in (2), then evaluate with no mask at all.

Training uses PPO for 500,000 steps with four random seeds per configuration. The appendix lists standard PPO engineering details: advantages computed by generalised advantage estimation¹⁹, advantage normalisation, an entropy bonus²⁰, gradient clipping and learning-rate decay.

Metrics

Average episode return over the last ten games; counts of three types of illegal unit selection (empty cell, busy unit, opponent's unit); t_{solve} , the share of training needed to reach a return of 40; and t_{first} , the share of training before the first positive reward. The last is a pure measure of exploration²¹.

Results

Masking scales; penalties do not

With proper masking, t_{solve} stays roughly flat across map sizes (about 9% to 18% of training) and the first reward arrives almost immediately (well under 0.1% of training) on every map. Every masked run reaches a return of 40.

The penalty approach works on the 4×4 map (returns of 30 to 40 for small penalties) but collapses on anything larger: returns near zero on 10×10 , 16×16 and 24×24 , and not one of these runs ever reaches the threshold. Even finding the first reward takes much longer, around 1.5% to 3.4% of training on 10×10 . The penalty does teach the agent to avoid illegal actions (the illegal-selection counts drop to near zero), but it does so by learning to do nothing useful. The harshest penalty, -1 , was worst everywhere: punishing mistakes so hard that the agent stops exploring. With no penalty at all, the agent wastes almost every step on illegal choices on large maps.

The intuition is a needle-in-a-haystack sampling problem. Without a mask, the probability that random behaviour produces even one legal, rewarding sequence shrinks with the illegal share, so the gradient estimator sees almost no informative samples. The mask cuts the haystack down to the needles.

18. Adding extra, hand-designed rewards to steer learning. It changes the objective the agent optimises and can create unintended incentives.

19. GAE: a way of computing advantages as an exponentially weighted mix of short- and long-horizon return estimates, controlled by a parameter λ (0.97 here). Small λ means lower variance but more bias from the value function.

20. A small reward for keeping the policy's distribution spread out, which discourages premature collapse onto one action and preserves exploration.

21. Trying actions whose value is still unknown in order to discover rewards. If random behaviour almost never stumbles on a reward, the gradient estimator has nothing to work with.

Naive masking: good returns, unstable updates

Surprisingly, naive masking reached the highest returns on the 4×4 map (about 60, as the agent learned to walk across the map to a second resource patch) and matched masking elsewhere. But two warning signs appeared. First, the KL divergence between consecutive policies was far higher than for any other method: the policy changes drastically at each update, which tends to break on harder tasks. Second, its t_{solve} grew sharply with map size, reaching about 49% of training on 24×24 versus about 18% for proper masking.

Why? The naive estimator is internally inconsistent. Data are sampled from π' , but the score is computed for π . For the statistician, this is like drawing from one distribution and plugging the draws into the score of another without any importance correction: the result is not the gradient of any coherent objective. In practice each update keeps pushing down the logits of illegal actions, which the agent could never take anyway. Those logits drift, the unmasked distribution changes a lot, and PPO's ratio and clipping, which assume a consistent policy, behave erratically.

Removing the mask after training

An agent trained with masking and then run without it still scores reasonably: about 34 on 4×4 , falling to about 17 on the two largest maps, and it starts choosing empty cells as the acting unit more often as maps grow. That is clearly worse than with the mask, but much better than any penalty-trained agent on larger maps. The network partially absorbs which units are usable, even though masked logits never received gradient, presumably because the legal logits are pushed up and their competitors are left behind.

Related approaches

The authors briefly place masking among alternatives: embedding discrete actions in a continuous space and snapping to the nearest legal one; training a separate network to eliminate actions (used in text-based games); and "action space shaping", where designers remove or merge actions by hand. They note that earlier ablations had shown masking helps, but had not studied how the benefit grows with the share of illegal actions.

Limitations worth knowing

- The task is tiny and single-agent (harvesting only, no opponent), with four seeds per setting. The scaling story is convincing; the fine-grained differences between methods are not statistically strong.
- The mask covers only two of eight action components, and the components are modelled as independent. In MTG, legality is sequential and conditional: whether a target is legal depends on which spell you chose. That calls for an autoregressive²² head with a fresh mask at each sub-step, which this paper does not test.
- The proposition is short and somewhat obvious once stated; its value is clarifying that masking is a model choice, not a hack, and what the gradient does.

22. Sampling a compound action one piece at a time, each piece conditioned on the earlier ones (choose spell, then choose target given the spell). Each step can then get its own exact legal mask.

Key takeaways

- Masking illegal actions is mathematically clean: it is a deterministic, state-dependent part of the policy, and its gradient is a valid policy gradient that leaves illegal actions' logits untouched.
- As the share of illegal actions grows, masking keeps learning fast while negative-reward penalties fail, mainly because the unmasked agent almost never finds a reward to learn from.
- Be consistent: use the same mask when sampling, when recording old log-probabilities, and when recomputing probabilities during PPO updates. Storing the mask in the rollout buffer is not optional.
- Mixing a masked sampler with an unmasked gradient (naive masking) can look fine on returns but produces large policy jumps (high KL) and gets worse with scale.
- For the MTG agent: always supply the engine's legal-action mask, including in search rollouts and when training on search-derived targets or opponent-action predictions; prefer an autoregressive, per-step mask for spell-then-target choices.
- A masked policy partly internalises the rules, but not reliably; never deploy it without the mask.

Hearthstone from scratch: self-play plus a bag of tricks

Changnan Xiao, Yongxin Zhang, Xuefeng Huang, Qinhan Huang, Jie Chen, Peng Sun (2023), "Mastering Strategy Card Game (Hearthstone) with Improved Techniques", IEEE Conference on Games (CoG 2023); arXiv:2303.05197

In one paragraph

A ByteDance team takes a recipe that had already won a research card-game competition and applies it to a 2015 version of Hearthstone. The recipe: one network that both builds the deck and plays, trained by self-play against a pool of its own past versions. On top of it they stack six practical changes and measure each in 45,000-game head-to-heads: drop hero choice from learning, stop discounting, randomise early deck picks, slow data production so training data is less stale, re-tune the off-policy correction, and give each hero its own network. The final honest model wins 74% against the starting baseline. A variant that peeks at part of the opponent's deck reaches 80%. The best models won all four best-of-five tournaments against a streamer who had once been top 10 on the Chinese ladder, and no look-ahead search is used anywhere.

Why it matters for the MTG agent

- **Ready-made opponent-pool rules.** 60% of games against the current self, 40% against past snapshots chosen by win record. A new snapshot joins the pool only after it beats every member more than 55% of the time.
- **Stale data cost 10 points of win rate.** Halving the number of game-playing workers, so the learner stopped training on data from old weights, was one of their biggest gains.
- **Use $\gamma = 1$** with a terminal ± 1 reward and about 100 decisions per game.
- **Put hidden information into training targets, not into the policy.** Their deck-peeking model won more against other bots but blundered against the human. PerfectDou puts hidden information only into the critic. For your opponent-action head, the same rule applies: hidden cards may supervise it during training, but the policy should act only on public information.
- **Card, then target, with masks,** as in the masking chapter.
- **Meta-choices can collapse.** When hero choice was learned by policy gradient, it collapsed onto one hero. Deck or sideboard choice learned the same way may fail similarly.
- **No search, but no stack either.** Hearthstone lacks instant-speed interaction, so "search was unnecessary" does not carry over to Pauper.

The game and why it is hard

Hearthstone has MTG's skeleton: build a 30-card deck, then play it against an opponent whose deck and hand are hidden. A game has three stages: **pick a hero** (Mage, Warrior or Hunter, each with a reusable "hero power" and exclusive cards), **build a deck** from about 270 cards, and **battle** to bring the enemy hero from 30 health to zero.

Three differences from MTG matter most:

- **No stack, no instant-speed interaction.** You act only on your own turn, and the opponent cannot respond to a spell. Each turn is therefore a one-player planning problem.
- **Mana grows by itself**, by one crystal per turn. There are no lands, so there is no mana screw or flood, and that whole layer of deck and mulligan decisions is gone.
- **More randomness**, because many cards have random effects on top of the shuffle.

The game is finite, two-player and zero-sum¹, so a Nash equilibrium² exists if players may randomise. Hidden information rules out the planning methods used for chess, though, so actually finding it is the hard part. Deck building also produces rock-paper-scissors cycles (aggro beats control beats midrange...), so beating the previous version does not mean beating everything.

Background: the inherited recipe

The learning problem

Reinforcement learning³ seeks a policy⁴ π maximising $E_{\pi}[\sum_t \gamma^t r_t]$, the expected sum of rewards r_t weighted by a discount rate⁵ γ . Each player sees only part of the state, so the problem is a POMDP⁶. With two adversaries, an outer loop must also decide whom the learner trains against.

One network for deck building and battle

Earlier work trained the deck-builder and the battler alternately, freezing one while improving the other. The predecessor paper uses a single end-to-end policy with a stage switch:

$$\pi_{\theta}(\cdot \mid o) = \delta \cdot \pi_{\theta_{CB}}(\cdot \mid o) + (1 - \delta) \cdot \pi_{\theta_{BT}}(\cdot \mid o)$$

Here $\delta = 1$ during card building (CB) and 0 during battle (BT). The two heads share card and hero embeddings⁷, and the value function⁸ reads features from both. As a result, deck choices get credit from the final result through one gradient, with no hand-written card scores.

Optimistic smooth fictitious play (OSFP)

This is the outer loop. **Fictitious play**⁹ repeatedly best-responds to the average of past strategies. Only the *average* converges, the way an ergodic average settles while individual Markov-chain draws keep wandering.

1. One player's gain is exactly the other's loss. One-versus-one MTG is also zero-sum.

2. A strategy pair where neither player gains by changing only their own. In two-player zero-sum games it is the unexploitable strategy, the one with the best guaranteed worst-case expected result.

3. Learning a decision rule from reward signals by trial and error. The data are generated by the rule being learned, so the sampling distribution shifts as estimation proceeds.

4. A conditional distribution $\pi(a \mid o)$ of actions given the current observation. Learning a policy means fitting its parameters.

5. A factor $\gamma \leq 1$ that shrinks later rewards. Values below 1 reduce variance but change the target: with $\gamma = 0.99$, a win 100 steps away counts only about 0.37.

6. Partially observable Markov decision process: the true state evolves as a Markov chain driven by actions, but the agent sees only part of it. The ideal agent acts on its posterior over the hidden state.

7. Learned vectors representing each card or hero, so that cards that behave alike get similar vectors.

8. The network's estimate of expected future return. With a ± 1 terminal reward and $\gamma = 1$, it is a rescaled win probability.

9. A classical game-theory scheme: each round, best-respond to the average of the opponent's past strategies. In two-player zero-sum games, the running average converges to a Nash equilibrium.

Smooth replaces the hard best response with a smooth best response¹⁰, which RL computes here by training against the sampled opponents. **Optimistic** puts extra weight on the newest model, which the predecessor paper argues gives last-iterate convergence¹¹.

The practical loop (appendix) is a template for your pool:

1. Training proceeds in learning periods of 320 million samples.
2. For each game, with probability $p = 0.6$ the opponent is the current learner (the "optimistic" weight). Otherwise a pool snapshot is drawn according to its running record against the learner. The sampling function is not given in the text; its job is to favour opponents the learner still struggles against.
3. After each period, the learner joins the pool if it beats *every* member at a win rate above $\xi = 0.55$, or if 6 periods pass without admission.

The guarantees cover an idealised version; this loop is a well-motivated heuristic.

Applying it to Hearthstone

Inputs and actions

The observation includes everything a human can see: hand, both boards, both graveyards, mana and the current decision type. An LSTM¹² with 256 units carries history. Playing a card is two categorical steps: first a "type" (hand card, board minion, hero power or "end turn"), then a target. This autoregressive decomposition¹³ keeps each step small, and a 0/1 mask removes illegal options before the softmax.

The policy gradient: actors, learner and V-trace

Training uses an actor-learner architecture¹⁴, so the data are off-policy¹⁵. Two quantities drive the update:

$$\delta_t = r_t + \gamma V_{t+1} - V_t; \quad \rho_t = \pi(a_t | o_t) / \mu(a_t | o_t)$$

δ_t is the temporal-difference error¹⁶. ρ_t is the familiar importance-sampling ratio, the same one PPO uses. V-trace¹⁷ sets the value target to

$$v_t = V_t + \sum_{i=0..k} \gamma^i [\prod_{j<i} \min(\rho_{t+j}, \bar{c})] \cdot \min(\rho_{t+i}, \bar{\rho}) \cdot \delta_{t+i}$$

10. Choose actions with softmax probabilities, proportional to $\exp(\text{value} / \text{temperature})$, instead of always taking the argmax. This is the logit or quantal-choice model. In RL, an entropy bonus plays this role.

11. The newest checkpoint itself, not just the average of all checkpoints, approaches equilibrium. You can stop at any time and deploy the latest model.

12. A recurrent network layer that carries a hidden state across steps, so the policy can condition on history. Think of it as a learned non-linear summary statistic of the past.

13. Factorising $P(\text{card}, \text{target}) = P(\text{card}) \cdot P(\text{target} | \text{card})$ and sampling step by step, so each step has a few dozen options instead of the full product.

14. Many CPU "actors" play games with a periodically refreshed copy of the weights and stream trajectories to a GPU "learner" that updates them. The actors' data therefore always come from a slightly outdated policy.

15. Generated by a stale behaviour policy μ rather than the policy π being updated. This is estimation under one distribution using samples drawn from another.

16. A one-step residual: reward plus predicted value of the next state, minus predicted value of this state.

17. An off-policy correction from the IMPALA system that builds multi-step value targets from stale data using truncated importance ratios. Truncation bounds the variance at the cost of some bias.

In words: start from the current estimate and add the next k residuals. Each residual is weighted by its own capped ratio and by the product of capped ratios before it (the "trace"). The caps give the usual truncated-weights trade: lower caps mean less variance but more bias. A single tiny ratio cuts off everything after it. The value net is regressed onto v_t . The policy gradient is the capped ratio times the advantage¹⁸, times $\nabla \log \pi$. An extra term, UPGO¹⁹, is added with equal weight.

Improvements and experiments

Each A-versus-B comparison uses $2 \times 3 \times 3 \times 2,500 = 45,000$ games, covering both seats and all nine hero pairings. The standard error is about 0.24 points, so the differences are not noise. However, each comparison is a *single run* with no repeat seeds, and changes are stacked, so the gains are sequential increments, not independent effects.

Model	Change	Win-rate gain vs previous
b0	Inherited recipe, uniform hero	baseline
b1	$\gamma = 1$ instead of 0.99	about +7
b1.5	Random first deck picks	about even
b2	Half the actors	+10.0
b3	Re-tuned V-trace + PPO clipping	+15.1
b4	One network per hero	+6.5
c5	Peeks at opponent's decklist	+5.5

" +a" means the new model wins $(50 + a)\%$ against the previous one. Against b0, b4 wins 73.6% and c5 wins 80.2%.

Do not learn the hero pick

Adding hero choice as a third head (a one-shot, bandit-like²⁰ decision) made the pick collapse onto one hero. Entropy tuning only switched between "one hero" and "uniform". The model was good in Hunter mirrors and poor elsewhere. They sampled heroes uniformly instead.

Stop discounting

Games are short and rewarded only at the end, so the undiscounted return already has bounded variance. Discounting with 0.99 tilts the objective toward fast wins over wins. $\gamma = 1$ gained about 7 points.

Random-CB

Before each game, each player draws $n \in \{0, 1, 2, 4\}$ with probabilities (0.5, 0.25, 0.125, 0.125), and the first n deck picks are made uniformly at random. This is an exploration²¹ device that exposes the

18. How much better an action turned out than expected, here $r_t + \gamma v_{t+1} - V_t$. Subtracting V_t is a control variate: it lowers variance without changing the expected gradient.

19. Upgoing policy update, from AlphaStar: a policy-gradient term that bootstraps only along trajectories that went better than expected.

20. A multi-armed bandit is a single repeated choice among options with unknown payoffs and no state.

21. Deliberately trying non-greedy actions so the data cover options the policy currently undervalues. Without it, the policy learns only about what it already does, a support problem.

battler to varied decks. Win rate barely moved, but the decks became recognisably human (one resembled a famous 2015 "Flamewaker Mage" list). It is turned off at evaluation.

Balance data production and consumption

A ring buffer²² gives uncontrolled reuse, so they use a first-in-first-out queue where actors wait. Surplus actors, however, just fill the queue with data from older weights. b1.5 produced data more than twice as fast as the learner consumed it. Halving the actors brought the ratio near 1 and gained 10 points, with *less* hardware.

Re-tuning the off-policy correction

The theory only needs $\bar{c} \leq \bar{\rho}$, not both ≤ 1 , so they raised the caps slightly to 1.007. They also found many ratios near 10^{-4} , which killed the trace, so they added a floor of 0.001: each weight becomes $\min(\max(\rho, 0.001), 1.007)$. A floor on an importance weight is a deliberate bias. Actions the current policy considers nearly impossible get more weight than they should, in exchange for keeping long-range credit flowing. They also swapped the plain capped-ratio gradient for PPO's clipped surrogate²³ ($\epsilon = 0.2$), with one change from standard PPO: the advantage uses the V-trace target rather than GAE²⁴. Together: +15.1, the largest gain.

One network per hero

Doubling or tripling GPUs and batch size at fixed network size did not help. Separate networks per hero (three times the compute) gained 6.5 points: heroes play very differently, and a shared network of limited size could not specialise. The best honest model used 24 V100 GPUs and 5,856 CPU cores.

Cheating, asymmetrically

The policy sees the first n cards of the opponent's decklist (never the hand). If both training players cheated equally, the learner would never meet an honest opponent. So they draw n_1, n_2 uniformly from 0–30, give the learner n_1 and its opponent $\min(n_1, n_2)$, and the learner always knows at least as much. The result was +5.5 over b4 and deliberately counter-built decks. Unlike PerfectDou, the hidden information here enters the *policy*, not only the critic.

Machine versus human

The opponent had a best rank of top 10 in China (60–70 at the time). He then played four best-of-five tournaments under Conquest rules²⁵: two against b4 (23 days of training) and two against c5 (16 days). He did not know which model was which.

Model	Tournament 1	Tournament 2
b4, 23 days (honest)	3:0	3:0
c5, 16 days (cheat)	3:1	3:2

22. A fixed-size circular buffer: new data overwrite old slots whether used or not, so some samples are used twice and others never.

23. Take the smaller of $\text{ratio} \times \text{advantage}$ and $\text{clip}(\text{ratio}, 1 - \epsilon, 1 + \epsilon) \times \text{advantage}$, removing any incentive to move far from the data-generating policy in one step. See the masking chapter.

24. Generalised advantage estimation, PPO's usual advantage: an exponentially weighted sum of TD residuals with no correction for staleness.

25. Each player brings one deck per hero, and a deck that wins a game is retired for the rest of the match, so you must win with every deck.

The honest model cleared the board sensibly and found exact lethal turns, including a one-turn kill from 21 health with its self-taught deck. The cheat model sometimes built pointed counters (two healing minions against a burn deck) but also played a healer at full health, an outright blunder. Stronger in the bot table, it looked weaker against the human.

Statistically this is thin: one human, 19 games. It is a demonstration, not an estimate. Note also a mild intransitivity in the bot table. b4 beats b3 56.5% head-to-head, yet b3 does better against b0 (75.7% versus 73.6%). That is the rock-paper-scissors structure that justifies keeping an opponent pool.

Conclusion

The claim is modest: the end-to-end-policy-plus-OSFP framework transfers from a research card game to a commercial one, and a few cheap engineering adjustments add up to a large gain. Its value is the ordered, measured list of what helped.

Key takeaways

- Self-play against a gated pool of past selves (60% current self, admission above 55% against everyone) reached master level in Hearthstone without search.
- Most of the gain came from pipeline and estimator details: matching actors to the learner (+10), importance-weight clipping plus PPO clipping (+15), and $\gamma = 1$ (+7).
- Log the distribution of π/μ . Many near-zero ratios mean your multi-step corrections are silently truncated.
- Pre-game meta-choices (hero, and by analogy deck or sideboard plan) collapsed under plain policy gradient. Sample them or treat them separately.
- Specialised networks per hero beat extra compute. For MTG, try per-archetype conditioning before scaling up.
- Hidden information in the deployed policy raised bot win rate but caused human-visible mistakes. Use it in the critic and in auxiliary targets such as your opponent-action head.
- Hearthstone's lack of a stack makes "no search needed" a weaker signal for MTG, where instant-speed interaction strengthens the case for search.

Part II

Original papers, reference lists removed

POLICY IMPROVEMENT BY PLANNING WITH GUMBEL

Ivo Danihelka^{1, 2}, Arthur Guez¹, Julian Schrittwieser¹, David Silver^{1, 2}

¹DeepMind, London, UK

²University College London
danihelka@google.com

ABSTRACT

AlphaZero is a powerful reinforcement learning algorithm based on approximate policy iteration and tree search. However, AlphaZero can fail to improve its policy network, if not visiting all actions at the root of a search tree. To address this issue, we propose a policy improvement algorithm based on sampling actions without replacement. Furthermore, we use the idea of policy improvement to replace the more heuristic mechanisms by which AlphaZero selects and uses actions, both at root nodes and at non-root nodes. Our new algorithms, Gumbel AlphaZero and Gumbel MuZero, respectively without and with model-learning, match the state of the art on Go, chess, and Atari, and significantly improve prior performance when planning with few simulations.

1 INTRODUCTION

In 2018, AlphaZero (Silver et al., 2018) demonstrated a single algorithm achieving state-of-the-art results on Go, chess, and Shogi. The community reacted quickly. Leela Chess Zero (Linscott et al., 2018) was created to reproduce AlphaZero results on chess, winning Top Chess Engine Championship in 2019. Soon, all top-rated classical chess engines replaced traditional evaluations functions with Efficiently Updatable Neural Network (Nasu, 2018).

AlphaZero was itself generalized by MuZero (Schrittwieser et al., 2020). While AlphaZero requires a black-box model of the environment, MuZero learns an abstract model of the environment. Essentially, MuZero learns the rules of Go, chess, and Shogi from interactions with the environment. This allows MuZero to excel also at Atari and continuous control from pixels (Hubert et al., 2021).

In this work, we redesign and improve AlphaZero. In particular, we consider the mechanisms by which AlphaZero selects and uses actions, which are based upon a variety of heuristic ideas that have proven especially effective in Go, chess, and Atari (Silver et al., 2018; Schrittwieser et al., 2020). However when using a small number of simulations, some of AlphaZero’s mechanisms perform poorly. We use the principle of policy improvement to suggest new mechanisms with a better theoretical foundation. More specifically, we consider each mechanism in turn, alongside our proposed modifications:

- **Selecting actions to search at the root node.** To explore different actions during training, AlphaZero selects actions by adding Dirichlet noise to its policy network, and then performs a search using the perturbed policy. However, this does not ensure a policy improvement. We instead propose to sample actions without replacement by using the Gumbel-Top-k trick (Section 2), and perform a search using the same Gumbel values to influence the selection of the best action (Section 3.3), and show that this guarantees a policy improvement when action-values are correctly evaluated.
- **Selecting actions at the root node.** AlphaZero uses a variant of the PUCB algorithm (Rosin, 2011) to select actions at the root node. This algorithm was designed to optimize cumulative regret in a *bandit-with-predictor* setting (i.e. given prior recommendations from the policy network). However, no ancestors are dependent upon the evaluation of the root node, and the performance of the Monte-Carlo tree search therefore only depends upon the final recommended action at the root node, and not upon the intermediate actions selected during search (Bubeck et al., 2011). Consequently, we propose to use the Sequential Halving algorithm (Karnin et al., 2013) at the root node to optimize simple regret in a stochastic bandit with a predictor (Section 3.4).

- **Selecting actions in the environment.** Once search is complete, AlphaZero selects an action by sampling from an (annealed) categorical distribution based upon the visit counts of root actions resulting from the search procedure. We instead propose to select the singular action resulting from the Sequential Halving search procedure.
- **Policy network update.** AlphaZero updates its policy network towards a categorical distribution based upon the visit counts of root actions. However, even if the considered actions are correctly evaluated, this does not guarantee a policy improvement, especially when using small numbers of simulations (Grill et al., 2020). We instead propose a policy improvement based upon the root action values computed during search, and update the policy network towards that policy improvement (Section 4).
- **Selecting actions at non-root nodes.** AlphaZero uses the PUCT algorithm to select actions at non-root nodes. We instead propose to select actions according to a policy improvement (similar to the proposal of Grill et al. (2020)) based upon a completion of the action values. Furthermore, rather than sampling directly from this policy improvement, we propose a deterministic action selection procedure that matches the empirical visit counts to the desired policy improvement (Section 5).

The proposed modifications are applicable also to MuZero or any agent with a policy network and an expensive Q-network. The modifications are most helpful when using a small number of simulations, relative to the number of actions. When using a large number of simulations, AlphaZero works well. We tried to ensure that the new search is principled, better with a smaller number of simulations, and never worse. We succeeded on all tested domains: Go, chess, and Atari.

2 BACKGROUND

Before explaining the improved search, we will explain the Gumbel-Max trick and the Gumbel-Top-k trick. The Gumbel-Max trick was popularized by Gumbel-Softmax for a gradient approximation. In this paper, we are not interested in approximate gradients. Instead, we use the Gumbel-Top-k trick to sample without replacement.

Gumbel-Max trick. (Gumbel, 1954; Luce, 1959; Maddison et al., 2017; Jang et al., 2017)

Let π be a categorical distribution with logits $\in \mathbb{R}^k$, such that $\text{logits}(a)$ is the logit of the action a . We can obtain a sample A from the distribution π by first generating a vector of k Gumbel variables and then taking argmax :

$$(g \in \mathbb{R}^k) \sim \text{Gumbel}(0) \quad (1)$$

$$A = \arg \max_a (g(a) + \text{logits}(a)). \quad (2)$$

Gumbel-Top-k trick. (Yellott, 1977; Vieira, 2014; Kool et al., 2019) The Gumbel-Max trick can be generalized to sampling n actions *without replacement*, by taking n top actions:

$$(g \in \mathbb{R}^k) \sim \text{Gumbel}(0) \quad (3)$$

$$A_1 = \arg \max_a (g(a) + \text{logits}(a)) \quad (4)$$

$$\vdots$$

$$A_n = \arg \max_{a \notin \{A_1, \dots, A_{n-1}\}} (g(a) + \text{logits}(a)). \quad (5)$$

We will denote the set of n top actions by $\text{argtop}(g + \text{logits}, n) = \{A_1, A_2, \dots, A_n\}$.

3 PLANNING AT THE ROOT

We are interested in improving AlphaZero Monte-Carlo Tree Search (MCTS). In this section we will focus on the action selection at the root of the search tree.

3.1 PROBLEM SETTING

Both AlphaZero and MuZero have access to a policy network. At the root of the search tree, they can explore n simulations, before selecting an action for the real environment. We will formalize the

problem as a deterministic bandit with a predictor and we will later extend it to a stochastic bandit and MCTS.

Bandit. A k -armed deterministic bandit is a vector of Q-values $q \in \mathbb{R}^k$, such that $q(a)$ is the Q-value of the action a . The agent interacts with the bandit in n simulations (aka rounds). In each simulation $t \in \{1, \dots, n\}$, the agent selects an action $A_t \in \{0, \dots, k-1\}$ and visits the action to observe the Q-value $q(A_t)$.

The objective is to maximize the Q-value from a special last action A_{n+1} . That means we want to maximize $\mathbb{E}[q(A_{n+1})]$. This objective is equivalent to minimization of *simple regret*. The simple regret differs from the *cumulative regret* from all n simulations. Bubeck et al. (2011), Hay & Russell (2011), and Tolpin & Shimony (2012) already argued that at the root of the search tree we care about the simple regret.

The problem becomes interesting when the number of possible actions is larger than the number of simulations, i.e., when $k > n$. For example, 19x19 Go has 362 possible actions and we will do experiments with as few as $n = 2$ simulations. Fortunately, the policy network can help.

Predictor. In the *bandit-with-predictor* setting (Rosin, 2011), the agent is equipped with a predictor: the policy network. Before any interaction with the bandit, the policy network predicts the best action by producing a probability distribution π . The agent can use the policy network predictions to make more informed decisions.

Policy improvement. Naturally, we would like to have an agent that acts better than, or as well as, the policy network. We would like to obtain a policy improvement. If the agent’s action selection produces a *policy improvement*, then

$$\mathbb{E}[q(A_{n+1})] \geq \sum_a \pi(a)q(a), \quad (6)$$

where the probability $\pi(a)$ is the policy network prediction for the action a .¹ The policy network can then keep improving by modeling an improved policy.

3.2 MOTIVATING COUNTEREXAMPLE

We will show that the commonly used heuristics fail to produce a policy improvement.

Example 1. Acting with the best action from the top- n most probable actions fails to produce a policy improvement. Let’s demonstrate that. Let $q = (0, 0, 1)$ be the Q-values and let $\pi = (0.5, 0.3, 0.2)$ be the probabilities produced by the policy network. The value of the policy network is $\sum_a \pi(a)q(a) = 0.2$. For $n = 2$ simulations, the set of the most probable actions is $\{0, 1\}$. With that, the heuristic would select $A_{n+1} = \arg \max_{a \in \{0, 1\}} q(a)$. The expected value of such action is $\mathbb{E}[q(A_{n+1})] = 0$, which is worse than the value of the policy network.

You can find other counterexamples by generating random q and π vectors and testing the policy improvement (Inequality 6). The AlphaZero action selection is explained in Appendix A.

3.3 PLANNING WITH GUMBEL

We will design a policy improvement algorithm for the deterministic bandit with a predictor π . After n simulations, the algorithm should propose an action A_{n+1} with $\mathbb{E}[q(A_{n+1})] \geq \sum_a \pi(a)q(a)$.

One possibility is to sample n actions from π , and then to select from the sampled actions the action with the highest $q(a)$. Instead of sampling with replacement, we can reduce the variance by sampling without replacement.

Still, the sampled actions contain a limited amount of information about π . We should exploit the knowledge of π and its logits when selecting A_{n+1} . The main idea is to sample n actions without replacement by using the Gumbel-Top- k trick, and then to use the *same* Gumbel g to select the action with the highest $g(a) + \text{logits}(a) + \sigma(q(a))$. The σ can be any monotonically increasing transformation. The pseudocode for the algorithm is in Algorithm 1.

¹Inequality 6 can be strict, if we assume that an action has a positive advantage and its $\pi(a) > 0$.

Algorithm 1 Policy Improvement by Planning with Gumbel

Require: k : number of actions.
Require: $n \leq k$: number of simulations.
Require: logits $\in \mathbb{R}^k$: predictor logits from a policy network π .
 Sample k Gumbel variables:
 $(g \in \mathbb{R}^k) \sim \text{Gumbel}(0)$
 Find n actions with the highest $g(a) + \text{logits}(a)$:
 $\mathcal{A}_{\text{topn}} = \text{argtop}(g + \text{logits}, n)$
 Get $q(a)$ for each $a \in \mathcal{A}_{\text{topn}}$ by visiting the actions.
 From the $\mathcal{A}_{\text{topn}}$ actions, find the action with the highest $g(a) + \text{logits}(a) + \sigma(q(a))$:
 $A_{n+1} = \arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a) + \sigma(q(a)))$
return A_{n+1}

The algorithm produces a policy improvement, because

$$q(\arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a) + \sigma(q(a)))) \geq q(\arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a))). \quad (7)$$

This holds for any Gumbel g , so it holds also for expectations: $\mathbb{E}[q(A_{n+1})] \geq \mathbb{E}_{A \sim \pi}[q(A)]$. The $\arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a))$ is equivalent to sampling from the policy network π (see the Gumbel-Max trick or Appendix B). By using the same Gumbel vector g in the argtop and $\arg \max$, we avoid a double-counting bias.

The prior knowledge contained in the logits can help on partially observable environments, or when working with approximate or stochastic Q-values.

3.4 PLANNING ON A STOCHASTIC BANDIT

We can now extend the algorithm to a stochastic bandit. A stochastic bandit provides only a stochastic estimate of the expected Q-value $q(a)$. In that setting, we will use the empirical mean $\hat{q}(a)$ instead of $q(a)$. Obviously, the empirical mean would be better estimated, if visiting an action multiple times. We have to choose which actions to visit and how many times. We can control this in two places. First, we can control the number of actions sampled without replacement. Second, we can use a bandit algorithm to efficiently explore the set of sampled actions.

There are multiple bandit algorithms for simple regret minimization. In our preliminary experiments, Sequential Halving (Karnin et al., 2013) was easier to tune than UCB-E (Audibert et al., 2010) and UCB $\sqrt{\cdot}$ (Tolpin & Shimony, 2012). Conveniently, Sequential Halving does not have problem-dependent hyperparameters.

We present Sequential Halving with Gumbel in Algorithm 2, with an illustration in Figure 1. Sequential Halving is used to identify the action with the highest $g(a) + \text{logits}(a) + \sigma(\hat{q}(a))$.

Algorithm 2 Sequential Halving with Gumbel

Require: k : number of actions.
Require: $m \leq k$: number of actions sampled without replacement.
Require: n : number of simulations.
Require: logits $\in \mathbb{R}^k$: predictor logits from a policy network π .
 Sample k Gumbel variables:
 $(g \in \mathbb{R}^k) \sim \text{Gumbel}(0)$
 Find m actions with the highest $g(a) + \text{logits}(a)$:
 $\mathcal{A}_{\text{topm}} = \text{argtop}(g + \text{logits}, m)$
 Use Sequential Halving with n simulations to identify the best action from the $\mathcal{A}_{\text{topm}}$ actions, by comparing $g(a) + \text{logits}(a) + \sigma(\hat{q}(a))$.
 $A_{n+1} = \arg \max_{a \in \text{Remaining}} (g(a) + \text{logits}(a) + \sigma(\hat{q}(a)))$
return A_{n+1}

For a concrete instantiation of σ , we use

$$\sigma(\hat{q}(a)) = (c_{\text{visit}} + \max_b N(b)) c_{\text{scale}} \hat{q}(a), \quad (8)$$

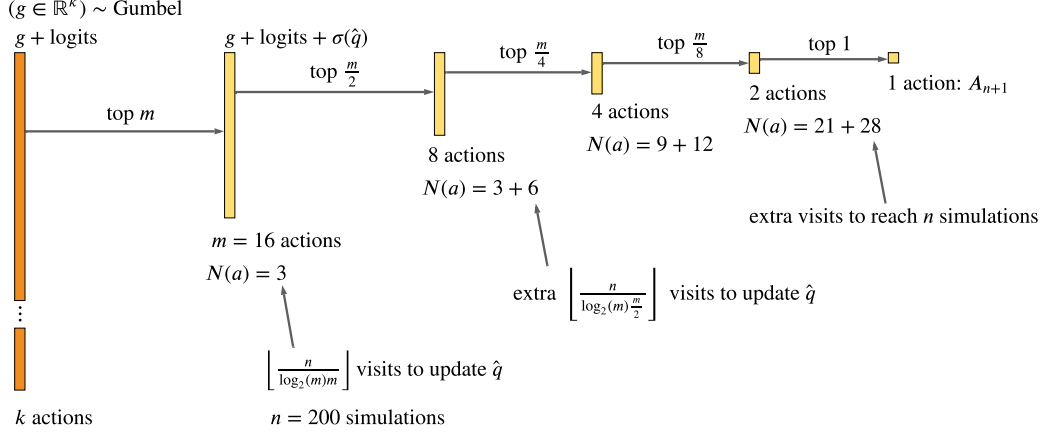


Figure 1: The number of considered actions and their visit counts $N(a)$, when using Sequential Halving with Gumbel on a k -armed stochastic bandit. The search uses $n = 200$ simulations and starts by sampling $m = 16$ actions without replacement. Sequential Halving divides the budget of n simulations equally to $\log_2(m)$ phases. In each phase, all considered actions are visited equally often. After each phase, one half of the actions is rejected. From the original k actions, only the best actions will remain.

where $\max_b N(b)$ is the visit count of the most visited action. The transformation progressively increases the scale for $\hat{q}(a)$ and reduces the effect of the prior policy. This scale is inspired by the policy updates in MPO (Abdolmaleki et al., 2018; Vieillard et al., 2020). The finite scale for the Q-values provides regularized policy optimization and puts into effect the prior knowledge contained in the logits. Experimentally, $c_{\text{visit}} = 50$, $c_{\text{scale}} = 1.0$ allowed us to use the same hyperparameters even if changing the number of simulations.

4 LEARNING AN IMPROVED POLICY

After the search, we have A_{n+1} from a (potentially) improved policy. Like AlphaZero, we would like to distill the improved policy to the policy network. One possibility is to train the policy network π to predict the A_{n+1} . That defines a simple policy loss:

$$L_{\text{simple}}(\pi) = -\log \pi(A_{n+1}). \quad (9)$$

Using completed Q-values. We will explain a different way to train the policy network, by extracting more knowledge from the search. Beside A_{n+1} , the search also gives us $q(a)$ (or its approximation) for the visited actions. We can construct an improved policy by first completing the vector of Q-values:

$$\text{completedQ}(a) = \begin{cases} q(a) & \text{if } N(a) > 0 \\ v_\pi, & \text{otherwise,} \end{cases} \quad (10)$$

where the unknown Q-values of the unvisited actions are replaced by $v_\pi = \sum_a \pi(a)q(a)$. While in practice we do not have the exact v_π , we have instead its approximation $\hat{v}_\pi$ from a value network. Even when training on off-policy data we devised a helpful v_π approximation (Appendix D).

With the completed Q-values, a new improved policy is constructed by

$$\pi' = \text{softmax}(\text{logits} + \sigma(\text{completedQ})), \quad (11)$$

where σ is a monotonically increasing transformation. We provide a proof of a policy improvement in Appendix C. Intuitively, the completed Q-values give zero advantage to the unvisited actions.

After constructing the new improved policy π' , we can distill it to the policy network π :

$$L_{\text{completed}}(\pi) = \text{KL}(\pi', \pi). \quad (12)$$

This loss trains all actions, not only the action A_{n+1} . Later, we will investigate the effect of the loss in Figure 3a.

5 PLANNING AT NON-ROOT NODES

To design an action selection for the non-root nodes of a search tree, we take inspiration from Grill et al. (2020). That allows us to interpret MCTS as regularized policy optimization. At a non-root node, we construct an improved policy π' by using the completed Q-values (Equation 11).

To select an action at the non-root node, one possibility is to sample the action from π' . However, sampling at non-root nodes adds unwanted variance to the estimated Q-values. Instead, we can design a deterministic action selection with the smallest mean-squared-error between the π' probabilities and the produced normalized visit counts. Such action selection would select

$$\arg \min_a \sum_b \left(\pi'(b) - \underbrace{\frac{N(b) + \mathbb{I}\{a=b\}}{1 + \sum_c N(c)}}_{\text{Normalized visit counts, if taking } a} \right)^2, \quad (13)$$

where the indicator $\mathbb{I}\{a=b\}$ is 1 if $a=b$, and zero otherwise. After a bit of algebra (Appendix E), we obtain a simpler, more efficient expression:

$$\arg \max_a \left(\pi'(a) - \frac{N(a)}{1 + \sum_b N(b)} \right). \quad (14)$$

This deterministic action selection selects the actions proportionally to π' and avoids an extra variance. We recommend a deterministic action selection only for non-root nodes. At the root node, the Gumbel noise is helpful for trying different actions in different episodes, while ensuring an improved expected value.

6 RELATED WORK

Rosin (2011) introduced the bandit with a predictor and designed PUCB (“Predictor + UCB”) for cumulative regret minimization. AlphaGo (Silver et al., 2016), AlphaGo Zero (Silver et al., 2017), AlphaZero (Silver et al., 2018), and MuZero (Schrittwieser et al., 2020) used a deep policy network as the predictor in a variant of the PUCB algorithm. Bertsekas (2019; 2021; 2022) provides an in-depth discussion of policy iteration, policy improvement, and their connection to rollout.

If not using a predictor, UCT (Kocsis & Szepesvári, 2006) would need to visit each action before being able to compare them. Rapid Action Value Estimation (Gelly & Silver, 2011) then helps to form rough estimates of the action values by aggregating statistics from all future states. Gelly & Silver (2011) also initialized the action value estimates with a heuristic evaluation function. The best heuristic used a learned linear network. Hamrick et al. (2020) later extended it to a deep Q-network.

Cazenave (2014) and Pepels et al. (2014) applied Sequential Halving to MCTS. Fabiano & Cazenave (2021) introduced Sequential Halving Using Scores. The ‘scores’ can be any prior offset to the Q-values. The way to obtain scores from a policy network was left as an open problem. We can now view the $g + \text{logits}$ as special scores.

MCTS is related to regularized policy optimization. Grill et al. (2020) analyzed AlphaZero tree search and discovered that AlphaZero approximates a regularized policy optimization. The approximation error is large if using a small number of simulations. To avoid the approximation error, Grill et al. (2020) used a regularized policy optimization directly inside the tree search. In the setting without a predictor, Xiao et al. (2019) compared UCT to a new MCTS with an entropy regularizer. Dam et al. (2021) generalized it to relative entropy and Tsallis entropy. Regularized policy optimization is helpful when working with approximate Q-values (Vieillard et al., 2020) or when doing an approximate policy iteration (Kakade & Langford, 2002; Schulman et al., 2015).

TreeQN (Farquhar et al., 2018) uses a breadth-first search inside a network architecture. The network can do a lot of computation before producing a Q-value. To reduce the computation demands, Dynamic Planning Networks (Tasfi & Capretz, 2018) extended TreeQN to sample only some actions. To approximate the gradient, Dynamic Planning Networks use Gumbel-Softmax (Maddison et al., 2017; Jang et al., 2017). Although we use Gumbel variables, we do not employ approximate gradients from Gumbel-Softmax. We use the Gumbel-Top-k trick to construct efficient planning with a provable policy improvement.

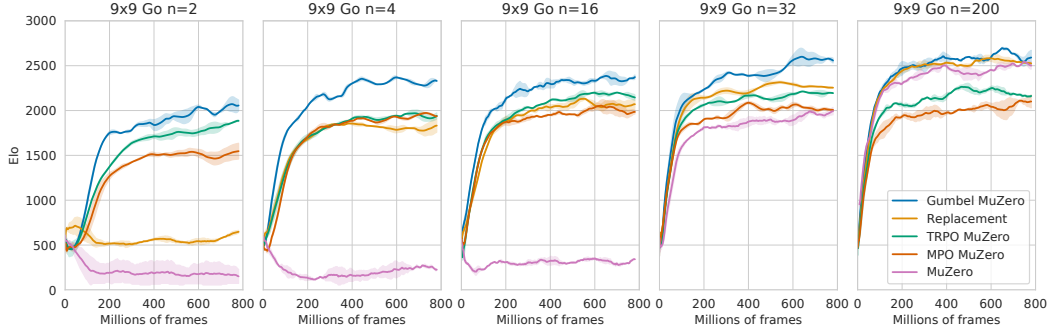


Figure 2: Elo on 9x9 Go, when training with $n \in \{2, 4, 16, 32, 200\}$ simulations. Evaluation uses 800 simulations. Shades denote standard errors from 2 seeds.

For sampling without replacement, the unordered set estimator by Kool et al. (2020) provides an elegant, unbiased estimate of a gradient. However, gradient descent needs multiple steps to reach the solution of a regularized policy optimization problem (Tomar et al., 2020). Furthermore, the exact computation of the unordered set estimator requires $O(2^m)$ operations, which would be prohibitively expensive for $m = 16$. Practical time complexity can be achieved by using an importance-weighted estimator (Vieira, 2017; Nauman & Den Hengst, 2020).

Continuous actions can be supported by sampling k actions with replacement and then using the sampled actions as discrete actions with uniform logits for the rest of the search. This was done in Sampled MuZero (Hubert et al., 2021). Similarly, Critic Weighted Policy (Wang et al., 2020) uses sampling with replacement.

7 EXPERIMENTS

In the experiments, we compare AlphaZero or MuZero to the proposed planning with Gumbel and other alternatives:

MuZero: The newest version of MuZero (Schrittwieser et al., 2021), with ResNet v2 style pre-activation residual blocks (He et al., 2016) and the Adam optimizer (Kingma & Ba, 2014). **Gumbel MuZero:** MuZero with the modified root of the search tree to use Sequential Halving with Gumbel. The policy loss uses the completed Q-values (Equation 12). **Gumbel MuZero sampled with replacement (Replacement):** An ablation to Gumbel MuZero by sampling m actions with replacement, as in Sampled MuZero (Hubert et al., 2021). **TRPO MuZero:** MuZero with modified learning, acting, and the root of the search tree to use the regularized policy optimization with the TRPO regularizer $KL(\pi, \pi_{\text{new}})$ (Schulman et al., 2015; Grill et al., 2020). **MPO MuZero:** TRPO MuZero but with the MPO regularizer $KL(\pi_{\text{new}}, \pi)$ (Abdolmaleki et al., 2018; Grill et al., 2020). **Full Gumbel MuZero:** Gumbel MuZero with a principled action selection also for the non-root search nodes (Section 5). In the plots, we will show Full Gumbel MuZero only if it produces results significantly different from Gumbel MuZero.

We conducted the experiments on Go, chess, and Atari. We present the main results here and we report additional ablations and experimental details in Appendix F.

7.1 9x9 Go

On Go, we use Elo to compare MuZero and other agents. While an agent trains by self-play, its Elo is computed by evaluation versus reference opponents. One of the opponents is Pachi (Baudiš & Gailly, 2011) with 10k simulations per move. We anchored the Elo to have this Pachi at 1000 Elo. For example, a difference of 500 Elo corresponds to a 95% win probability for the player with the higher Elo.²

²The corresponding win probability is $\frac{1}{1 + 10^{-\frac{\text{EloDifference}}{400}}}$ (Elo, 1978).

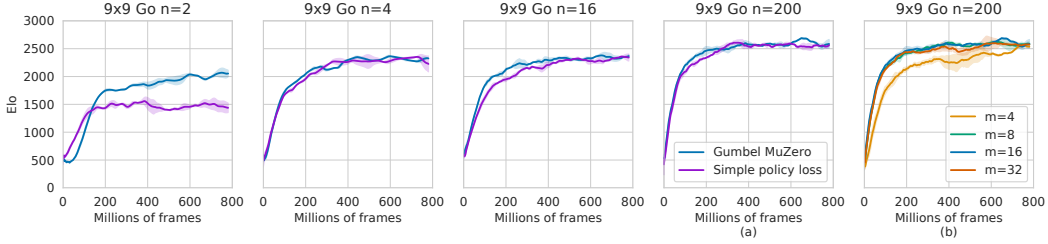


Figure 3: Gumbel MuZero ablations on 9x9 Go. **(a)** Policy loss ablations, when training with $n \in \{2, 4, 16, 200\}$ simulations. Gumbel MuZero uses the policy loss with completed Q-values. **(b)** Sensitivity to the number of sampled actions. Gumbel MuZero samples m actions without replacement.

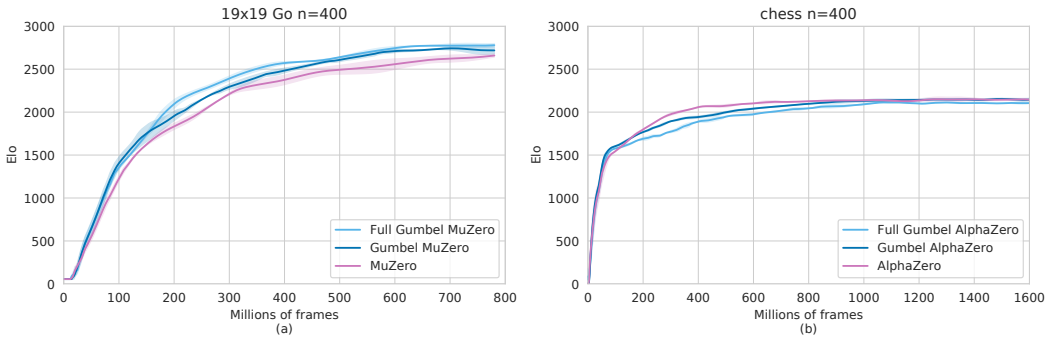


Figure 4: Large-scale experiments with $n = 400$ simulations per move. **(a)** Elo on 19x19 Go, when training MuZero. **(b)** Elo on chess, when training AlphaZero.

In Figure 2 we investigate the impact of the number of simulations on the obtained Elo. When training an agent by self-play, the agent uses n simulations per move. In the five plots, the n varies from 2 to 200. In evaluation, we allow all agents to use 800 simulations. The speed of the training does not affect the speed of training. In the 9x9 Go results, MuZero fails to learn from 16 or less simulations. Strikingly, Gumbel MuZero learns reliably even with 2 simulations.

In Figure 3a we compare the simple policy loss (Equation 9) and the policy loss with the completed Q-values (Equation 12). The simple policy loss would be enough for many applications. We used the completed Q-values also in TRPO MuZero and MPO MuZero. Without the completed Q-values, TRPO MuZero and MPO MuZero would fail to produce a policy improvement.

In Figure 3b we study Gumbel MuZero’s sensitivity to the number of sampled actions. When sampling $m = 4$ actions without replacement, the simulation budget is spent on the small number of actions. The learning was then slower. In all other Go experiments, we sample $m = \min(n, 16)$ actions without replacement.

7.2 LARGE-SCALE 19X19 GO AND CHESS

In Figure 4a we demonstrate that Gumbel MuZero is not worse than MuZero on 19x19 Go. MuZero is excellent on 19x19 Go and Gumbel MuZero reaches or exceeds its performance. The Elo is still anchored to have Pachi at 1000 Elo.

Similarly, in Figure 4b we show Gumbel AlphaZero performance on chess. We train AlphaZero on chess, because AlphaZero learns faster than MuZero on chess. On Go, MuZero learns faster than AlphaZero.

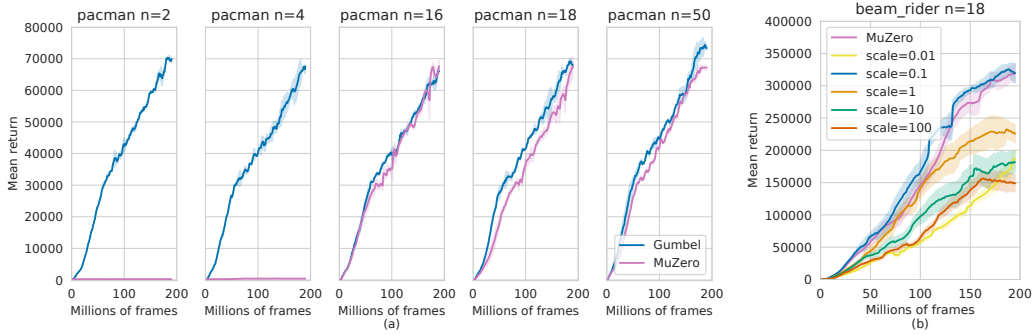


Figure 5: Atari results. **(a)** Mean return on `ms_pacman`, when training Gumbel MuZero and MuZero with $n \in \{2, 4, 16, 18, 50\}$ simulations. MuZero fails to learn from 4 or less simulations. **(b)** Mean return on `beam_rider` for Gumbel MuZero with $c_{\text{scale}} \in \{0.01, 0.1, 1, 10, 100\}$, compared to MuZero with $n = 50$ simulations. Shades denote standard errors from 10 seeds.

7.3 ATARI

Our last set of experiments is on Atari. We use the Arcade Learning Environment (Bellemare et al., 2013) with sticky actions (Machado et al., 2018). The network sizes and hyperparameters match the MuZero setup by Schrittwieser et al. (2021). On Atari, MuZero does not use more simulations at evaluation. The reported score is the mean return from the last 200 training episodes. MuZero with $n = 50$ simulations works well on Atari and establishes the state of the art (Schrittwieser et al., 2021).

In Figure 5a we show the obtained mean return on `ms_pacman`. Gumbel MuZero again learns reliably even with $n = 2$ simulations. Atari has only 18 actions, so we sample $m = \min(n, 18)$ actions without replacement. In the experiments with $n \leq 18$, Gumbel MuZero selects A_{n+1} from the n visited actions, without any Sequential Halving. This confirms that planning with Gumbel is the key ingredient responsible for the policy improvement from a small number of simulations.

Atari is challenging, because different games can have very different reward scales. MuZero normalizes the Q-values by dividing them by $\max(\hat{v}_\pi, \max_a \hat{q}(a)) - \min(\hat{v}_\pi, \min_a \hat{q}(a))$ found inside the tree search (Schrittwieser et al., 2020). A normalized advantage is then in $[-1, 1]$. For Gumbel MuZero, we use the same normalization and we scale the normalized Q-values by $c_{\text{visit}} = 50$ and $c_{\text{scale}} = 0.1$. A scaled normalized advantage is then approximately in $[-5, 5]$. Thanks to the bounded advantage, Gumbel MuZero has a bounded total variation distance between π and π' (Hessel et al., 2021).

In Figure 5b we use `beam_rider` as an example of a partially observable game and we study the importance of the prior knowledge contained in the logits. Gumbel MuZero selects an action based on $g(a) + \text{logits}(a) + (c_{\text{visit}} + \max_b N(b))c_{\text{scale}}\hat{q}(a)$ (Equation 8). If c_{scale} is large, Gumbel MuZero focuses on $\hat{q}(a)$ and neglects the logits. Indeed, Gumbel MuZero performance is worse on `beam_rider` if using large c_{scale} . In the future, we can try normalizing the Q-values by the standard deviation of an advantage estimator and we can try clipping the normalized advantages, as in Muesli (Hessel et al., 2021).

8 CONCLUSION

We redesigned AlphaZero tree search. With the principle of policy improvement, we replaced five heuristic mechanisms in AlphaZero. On Go, chess, and Atari, we validated that Gumbel MuZero and Gumbel AlphaZero keep improving, even when learning from two simulations. On top of that, Gumbel MuZero provides a principled way to achieve state-of-the-art results. We hope that future research will benefit from the clean theoretical foundation, the faster experimentation with a small number of simulations, and the released open-source code.³

³<https://github.com/deepmind/mctx>

Published as a conference paper at ICLR 2022

ACKNOWLEDGMENTS

We would like to thank Fabio Viola, Matteo Hessel, and Laurent Sifre for building the core of the open-sourced Monte Carlo tree search. Loic Matthey and Benjamin Van Roy provided helpful comments on the paper draft. Thomas Hubert, Iurii Kemaev, Tor Lattimore, and Jean-bastien Grill shared their wisdom.

APPENDIX

Content

- A - AlphaZero action selection
- B - Policy improvement proof for planning with Gumbel
- C - Policy improvement proof for completed Q-values
- D - Mixed value approximation
- E - Derivation of the deterministic action selection
- F - Experimental details

A ALPHAZERO ACTION SELECTION

We will explain AlphaZero’s (Silver et al., 2018) action selection here. On the deterministic bandit, AlphaZero would select actions $A_1, \dots, A_n$ by

$$\arg \max_a \left[q^?(a) + c_1 \pi(a) \frac{\sqrt{1 + \sum_b N(b)}}{1 + N(a)} \right], \quad (15)$$

where $q^?(a) \in [0, 1]$ is $q(a)$ for the already visited actions and zero otherwise. The $c_1 > 0$ is a factor independent of a . After the search, AlphaZero acts by sampling from an (annealed) categorical distribution based upon the visit counts of the root actions.

At the root of the search tree, AlphaZero perturbs π by Dirichlet noise to avoid visiting always the most probable actions. That does not ensure a policy improvement, because AlphaZero with Dirichlet noise can produce a worse policy by adding noise to a potentially optimal policy network.

B POLICY IMPROVEMENT PROOF FOR PLANNING WITH GUMBEL

We will prove that Algorithm 1 generates A_{n+1} such that $\mathbb{E}[q(A_{n+1})] \geq \mathbb{E}_{A \sim \pi}[q(A)]$. For the right-hand side, the Gumbel-Max trick tells us that $\mathbb{E}_{A \sim \pi}[q(A)]$ is equal to $\mathbb{E}_{(g \in \mathbb{R}^k) \sim \text{Gumbel}(0)}[q(\arg \max_a (g(a) + \text{logits}(a)))]$. First, we will show that we can replace the $\arg \max_a$ with $\arg \max_{a \in \mathcal{A}_{\text{topn}}}$. Remember that $\mathcal{A}_{\text{topn}}$ is defined as $\mathcal{A}_{\text{topn}} = \text{argtop}(g + \text{logits}, n)$ and that we use the same Gumbel vector g in the argtop and $\arg \max$. The set $\mathcal{A}_{\text{topn}}$ then includes the action with the highest $g(a) + \text{logits}(a)$ and we can replace the $\arg \max_a$ with $\arg \max_{a \in \mathcal{A}_{\text{topn}}}$.

After these rewrites, we have to prove that

$$\mathbb{E}[q(A_{n+1})] \geq \mathbb{E}_{(g \in \mathbb{R}^k) \sim \text{Gumbel}(0)}[q(\arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a)))]. \quad (16)$$

On the left-hand side, $\mathbb{E}[q(A_{n+1})]$ is equal to

$$\mathbb{E}_{(g \in \mathbb{R}^k) \sim \text{Gumbel}(0)}[q(\arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a) + \sigma(q(a))))]. \quad (17)$$

We can finish the proof by proving that for any vector $g \in \mathbb{R}^k$ we have

$$q(\arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a) + \sigma(q(a)))) \geq q(\arg \max_{a \in \mathcal{A}_{\text{topn}}} (g(a) + \text{logits}(a))). \quad (18)$$

This is true, because σ is a monotonically increasing transformation.

C POLICY IMPROVEMENT PROOF FOR COMPLETED Q-VALUES

We will prove that $\pi'_{\text{completed}} = \text{softmax}(\text{logits} + \sigma(\text{completedQ}))$ produces a policy improvement. We will start by showing that $\pi'_{\text{completed}}$ is produced by a specific instance of Algorithm 3. We will then prove that any instance of Algorithm 3 produces a policy improvement.

Algorithm 3 Policy improvement when having v_π **Require:** π, v_π .**Require:** $q(a)$ for each visited action. The visited actions can be from any distribution.Initialize π' with π .

For the visited actions:

If $q(a) > v_\pi$, increase the $\pi'(a)$ logit.If $q(a) < v_\pi$, decrease the $\pi'(a)$ logit.**return** π'

C.1 SPECIFIC INSTANCE

Algorithm 3 is more general than the usage of the completed Q-values. Specifically, Algorithm 3 would produce $\pi'_{\text{completed}}$, if updating the logits by $\sigma(\text{completedQ}(a)) - \sigma(v_\pi)$. This update increases the logit, if $q(a) > v_\pi$. This update decreases the logit, if $q(a) < v_\pi$. And the update does not modify the logits of the unvisited actions. The resulting $\text{softmax}(\text{logits} + \sigma(\text{completedQ}) - \sigma(v_\pi))$ is equal to $\text{softmax}(\text{logits} + \sigma(\text{completedQ}))$, because the constant offset $\sigma(v_\pi)$ does not change the softmax output.

C.2 POLICY IMPROVEMENT PROOF FOR ANY INSTANCE

We will now prove that π' from Algorithm 3 satisfies

$$\sum_a \pi'(a)q(a) \geq \sum_a \pi(a)q(a). \quad (19)$$

Notice that $\pi'(a)$ for any unvisited action a will be $c_z \pi(a)$, with a normalization constant $c_z > 0$.

For one visited action: Let's denote the visited (aka expanded) action by a_{ex} . First, if $\pi(a_{\text{ex}}) = 1$ then $v_\pi = q(a_{\text{ex}})$ and the policy will remain unchanged.

Let's now consider the case with $\pi(a_{\text{ex}}) < 1$. The v_π can be rewritten as

$$v_\pi = \pi(a_{\text{ex}})q(a_{\text{ex}}) + (1 - \pi(a_{\text{ex}})) \sum_{a \neq a_{\text{ex}}} \frac{\pi(a)q(a)}{\sum_{b \neq a_{\text{ex}}} \pi(b)}. \quad (20)$$

Let's denote the weighted sum by q_{miss} :

$$q_{\text{miss}} = \sum_{a \neq a_{\text{ex}}} \frac{\pi(a)q(a)}{\sum_{b \neq a_{\text{ex}}} \pi(b)}. \quad (21)$$

We notice that the q_{miss} does not change if scaling π by a constant $c_z > 0$.We will now rewrite the left-hand side of Inequality 19 to use q_{miss} :

$$\sum_a \pi'(a)q(a) = \quad (22)$$

$$= \pi'(a_{\text{ex}})q(a_{\text{ex}}) + (1 - \pi'(a_{\text{ex}})) \sum_{a \neq a_{\text{ex}}} \frac{c_z \pi(a)q(a)}{\sum_{b \neq a_{\text{ex}}} c_z \pi(b)} \quad (23)$$

$$= \pi'(a_{\text{ex}})q(a_{\text{ex}}) + (1 - \pi'(a_{\text{ex}}))q_{\text{miss}} \quad (24)$$

$$= \pi'(a_{\text{ex}})(q(a_{\text{ex}}) - q_{\text{miss}}) + q_{\text{miss}}. \quad (25)$$

The right-hand side of Inequality 19 can be also rewritten:

$$v_\pi = \pi(a_{\text{ex}})q(a_{\text{ex}}) + (1 - \pi(a_{\text{ex}}))q_{\text{miss}} \quad (26)$$

$$= \pi(a_{\text{ex}})(q(a_{\text{ex}}) - q_{\text{miss}}) + q_{\text{miss}}. \quad (27)$$

With these rewrites, Inequality 19 becomes

$$\pi'(a_{\text{ex}})(q(a_{\text{ex}}) - q_{\text{miss}}) \geq \pi(a_{\text{ex}})(q(a_{\text{ex}}) - q_{\text{miss}}). \quad (28)$$

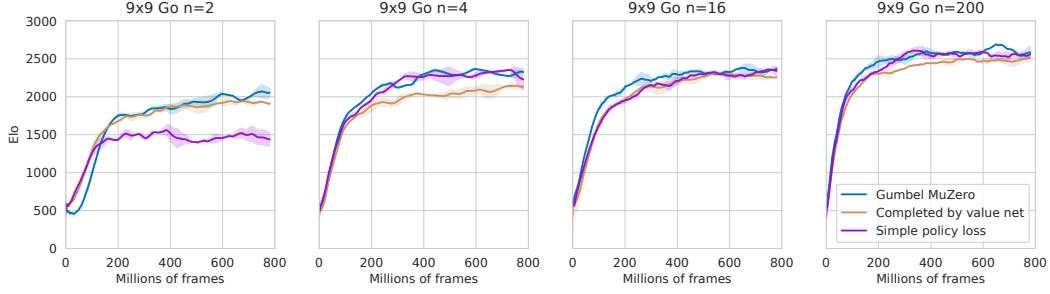


Figure 6: Detailed policy loss ablations. Gumbel MuZero uses the policy loss with Q-values completed by the v_{mix} value estimator from Appendix D. That works better than Q-values completed by the raw value network $\hat{v}_\pi$.

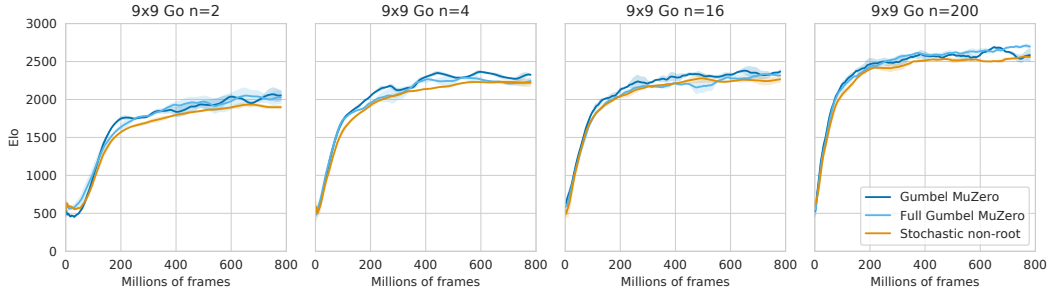


Figure 7: A comparison of different action selections at the non-root nodes. Gumbel MuZero uses the unmodified (deterministic) MuZero action selection at non-root nodes. Full Gumbel MuZero uses the deterministic action selection from Equation 14, which we compare to stochastic sampling from π' at non-root nodes.

The $q(a_{\text{ex}}) - q_{\text{miss}}$ can be negative, zero or positive.

If $q(a_{\text{ex}}) = q_{\text{miss}}$, the inequality is satisfied.

If $q(a_{\text{ex}}) > q_{\text{miss}}$, we want $\pi'(a_{\text{ex}}) \geq \pi(a_{\text{ex}})$.

If $q(a_{\text{ex}}) < q_{\text{miss}}$, we want $\pi'(a_{\text{ex}}) \leq \pi(a_{\text{ex}})$.

We do not know q_{miss} so we cannot use it in an algorithm. We will instead show that $q(a_{\text{ex}}) > q_{\text{miss}}$ is equivalent to $q(a_{\text{ex}}) > v_\pi$, when $\pi(a_{\text{ex}}) < 1$:

$$q(a_{\text{ex}}) > v_\pi \quad (29)$$

$$q(a_{\text{ex}}) > \pi(a_{\text{ex}})q(a_{\text{ex}}) + (1 - \pi(a_{\text{ex}}))q_{\text{miss}} \quad (30)$$

$$(1 - \pi(a_{\text{ex}}))q(a_{\text{ex}}) > (1 - \pi(a_{\text{ex}}))q_{\text{miss}} \quad (31)$$

$$q(a_{\text{ex}}) > q_{\text{miss}}. \quad (32)$$

So we directly arrived at Algorithm 3.

For multiple visited actions: We will focus on one of the visited actions. If the logits of the other visited actions are unmodified, the algorithm is equivalent to using only one visited action. If the logits of the other visited actions are modified by Algorithm 3, they can further help to improve the policy.

D MIXED VALUE APPROXIMATION

We will construct an approximation of v_π . The exact v_π is defined by $v_\pi = \sum_a \pi(a)q(a)$. We have an approximate $\hat{v}_\pi$ from a value network, we know π , and we have $q(a)$ for the visited actions. With

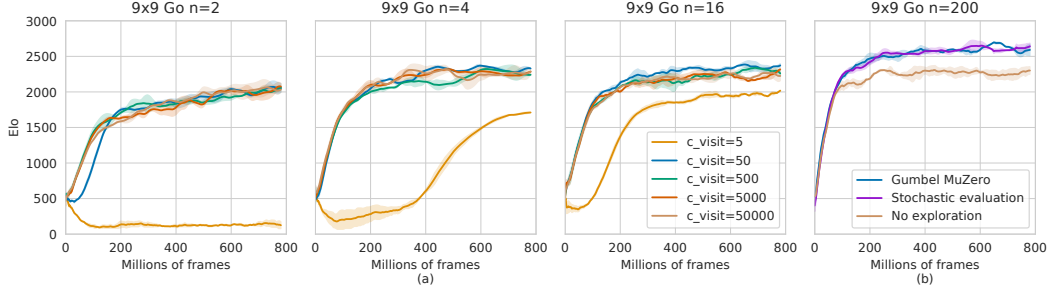


Figure 8: Additional Gumbel MuZero ablations on 9x9 Go. **(a)** Sensitivity to Q-value scaling by c_{visit} . **(b)** On the perfect-information game, Gumbel MuZero used zero Gumbel noise at evaluation. Although, evaluation with stochastic Gumbel noise is not worse. During training, MuZero and Gumbel MuZero benefit from explorative acting proportional to the visit counts.

these inputs, we approximate v_π by a consistent estimator:

$$v_{\text{mix}} = \frac{1}{1 + \sum_b N(b)} \left(\hat{v}_\pi + \frac{\sum_b N(b)}{\sum_{b \in \{b: N(b) > 0\}} \pi(b)} \sum_{a \in \{a: N(a) > 0\}} \pi(a) q(a) \right). \quad (33)$$

The estimator interpolates $\hat{v}_\pi$ and the weighted average of the available Q-values. This is an unsophisticated estimator, with results in Figure 6. You are welcome to explore other possibilities.

E DERIVATION OF THE DETERMINISTIC ACTION SELECTION

We will derive Equation 14 from Equation 13:

$$\arg \min_a \sum_b \left(\pi'(b) - \frac{N(b) + \mathbb{I}\{a = b\}}{1 + \sum_c N(c)} \right)^2 \quad (34)$$

$$= \arg \min_a \sum_b \left(\left(\pi'(b) - \frac{N(b)}{1 + \sum_c N(c)} \right) - \frac{\mathbb{I}\{a = b\}}{1 + \sum_c N(c)} \right)^2 \quad (35)$$

$$= \arg \min_a \sum_b -2 \left(\pi'(b) - \frac{N(b)}{1 + \sum_c N(c)} \right) \frac{\mathbb{I}\{a = b\}}{1 + \sum_c N(c)} \quad (36)$$

$$= \arg \min_a - \sum_b \left(\pi'(b) - \frac{N(b)}{1 + \sum_c N(c)} \right) \mathbb{I}\{a = b\} \quad (37)$$

$$= \arg \max_a \left(\pi'(a) - \frac{N(a)}{1 + \sum_c N(c)} \right). \quad (38)$$

The simplification was possible, because additive terms independent of a do not affect $\arg \min_a$. While widely applicable, the deterministic action selection provides only a small benefit on 9x9 Go (Figure 7).

F EXPERIMENTAL DETAILS

In general, we use hyperparameters consistent with the newest MuZero experiments (Schrittwieser et al., 2021). MuZero’s pseudocode is available thanks to Schrittwieser et al. (2020). Gumbel MuZero does not need to set the Dirichlet noise hyperparameters, because Gumbel MuZero does not use Dirichlet noise.

In a tree search, the Q-values $\hat{q}(a)$ are provided by the visited child nodes. We do not modify the structure of AlphaZero’s search tree. Inside the tree search, the Q-values are normalized to be from

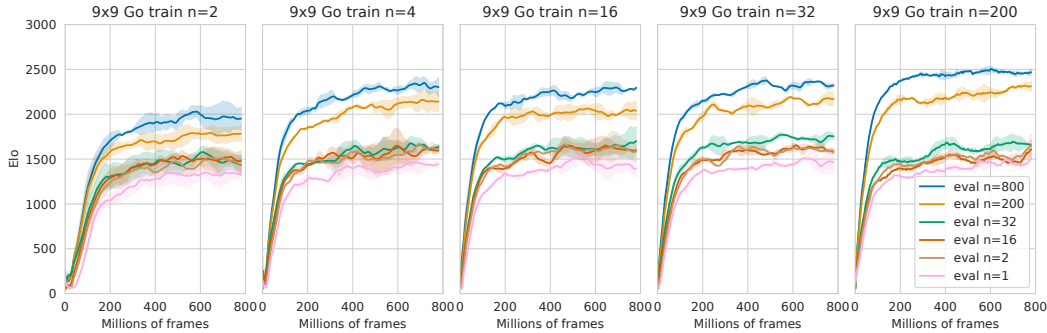


Figure 9: Gumbel MuZero Elo on 9x9 Go, evaluated with different numbers of simulations. The evaluation with $n = 1$ simulation acts with the most probable action from the policy network.

the $[0, 1]$ interval. We use the normalized Q-values also in Full Gumbel MuZero, but the algorithm does not require Q-values from a specific interval. In all Go and chess experiments, Gumbel MuZero scales the Q-values by $c_{\text{visit}} = 50$ and $c_{\text{scale}} = 1.0$. On the perfect-information game of Go, Gumbel MuZero is not very sensitive to the scale of the Q-values. Any $c_{\text{visit}} \geq 50$ produced similar results (Figure 8a).

In each phase of Sequential Halving, we use at least one new visit. For example, in the first phase, we update $\hat{q}(a)$ by $\max\left(1, \left\lfloor \frac{n}{\lceil \log_2(m) \rceil m} \right\rfloor\right)$ visits. This allows us to experiment with an incomplete or no Sequential Halving. When Sequential Halving runs out of the budget of n simulations, we stop the search. The agent then selects as A_{n+1} the action with the highest $g(a) + \text{logits}(a) + \sigma(\hat{q}(a))$ from the set of the most visited actions. Fabiano & Cazenave (2021) provide a different way to deal with the rounding in Sequential Halving.

During training, MuZero acts with explorative actions in the first 30 moves of each self-play game. MuZero samples the explorative actions proportionally to the visit counts, like AlphaGo Zero (Silver et al., 2017). Gumbel MuZero benefits from the same exploration (Figure 8b).

Figure 9 shows the importance of the number of simulations at evaluation time. For example, in the first subplot a network is trained with $n = 2$ simulations and the same network is evaluated with 800, 200, 32, 16, 2, and 1 simulations.

To run the experiments, we used Google Cloud Tensor Processing Units v3 (TPUs). On 9x9 Go, MuZero is not limited by lack of data if using 3-times more TPUs for self-play than for training. By using a smaller number of simulations, we can substantially reduce the number of TPUs needed for self-play. Table 1 lists the obtained speedups if not being limited by the TPUs for training.

Table 1: The speedup from a smaller number of simulations on 9x9 Go.

	Training step speedup
MuZero $n = 200$	1.0
Full Gumbel MuZero $n = 200$	1.0
Gumbel MuZero $n = 200$	1.0
Gumbel MuZero $n = 32$	5.9
Gumbel MuZero $n = 16$	11.3
Gumbel MuZero $n = 8$	16.2
Gumbel MuZero $n = 4$	24.3

F.1 NETWORK ARCHITECTURE

In all Go and chess experiments, we used a modified version of the AlphaZero network architecture that is roughly 2-times faster at the same accuracy. Concretely, we replace the dense residual blocks

with bottleneck blocks (He et al., 2016), and replace every 8th block with a global broadcasting residual block (Figures 11, 13). The broadcasting block is similar to global pooling or squeeze-and-excitation (Hu et al., 2018), but we found it to be more stable when training from self-play.

Figure 10 provides data to support the architecture change. For each network architecture the figure shows multiple data points, differing by the number of layers. For dense networks, the number of layers is from $\{12, 16, 20, 24, 28, 32, 36, 40, 48\}$. For bottleneck networks, data points with 56 and 64 layers are shown as well.

Both value MSE and policy accuracy improve consistently with larger but slower networks. For any constant number of inferences per second, the bottleneck blocks and the broadcast/pooling blocks lead to significant improvements.

For the architecture search, the Elo was computed at a fixed computational budget of 300ms per move. For example, the budget allows us to do 3200 simulations when using a network with 256 planes and 32 blocks with bottlenecks and broadcasting (achieving 10^4 inferences/second). Here the Elo is anchored to have the base AlphaZero architecture at 0 Elo; 95% confidence intervals are about 30 Elo wide. We observe that diminishing returns in the prediction accuracy for very large networks lead to decreasing Elo, as the improved predictions fail to compensate for the much lower number of simulations in MCTS. Gumbel AlphaZero was not yet used for the architecture search.

Given the efficient network architecture, we used a small 6 layer network on 9x9 Go, and a bigger 32 layer network in the large-scale 19x19 Go experiments. The networks used 256 hidden planes, 128 bottleneck planes and a broadcasting block in every 8th layer.

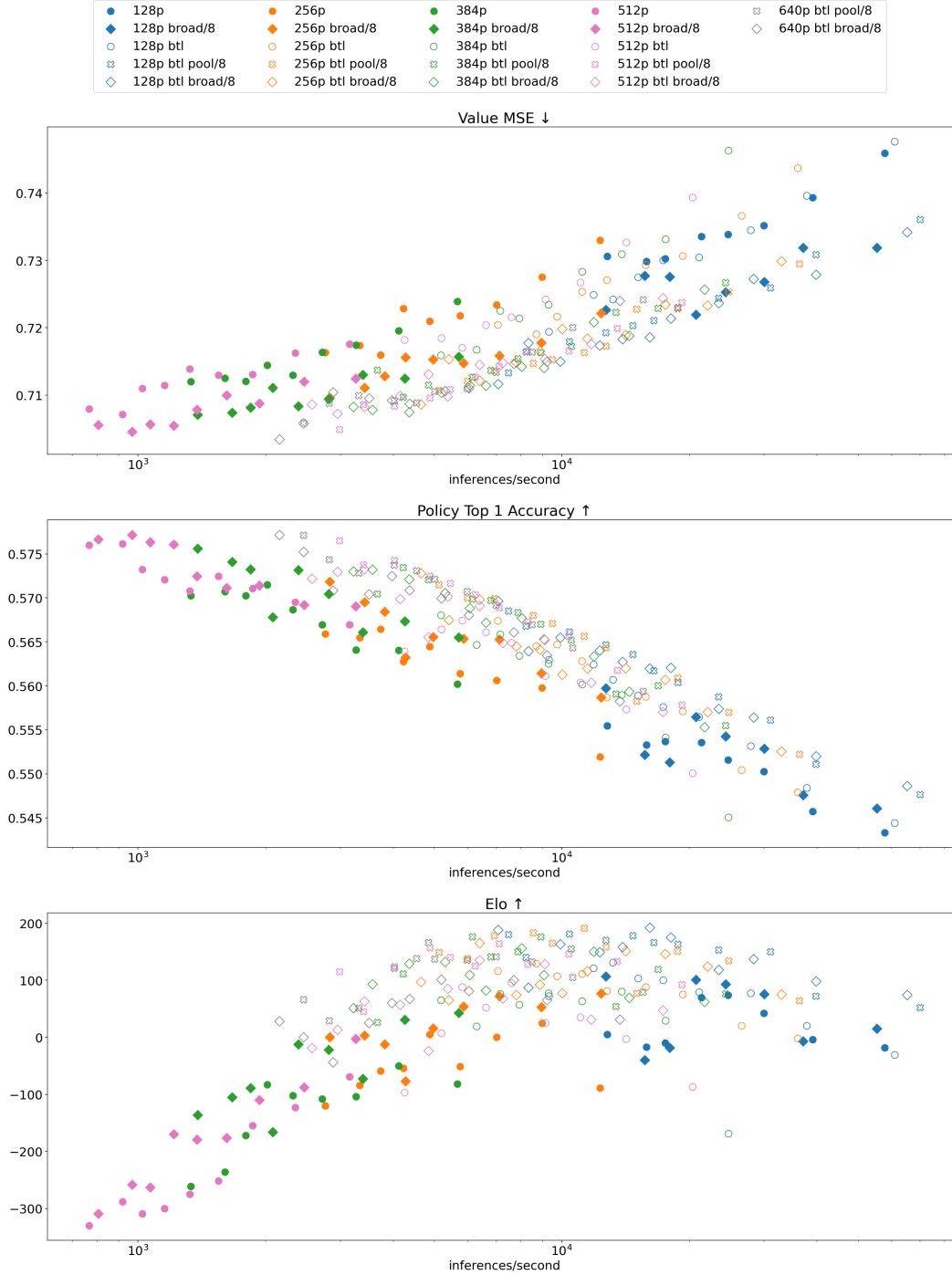


Figure 10: Scaling of value Mean Squared Error (MSE), policy accuracy and playing strength in Elo vs inference speed for different network architectures. In the legend, *btl* indicates bottleneck residual blocks; *broad/8* and *pool/8* indicate broadcast (Figure 11) and pooling blocks (Figure 12) in every 8th layer. *128p*, *256p*, etc. indicate the number of hidden planes.

```

class BasicBlock(hk.Module):
    """Basic block composed of an inner op, a norm op and a non linearity."""

    def __init__(self, make_inner_op, non_linearity=jax.nn.relu, name='basic'):
        super().__init__(name=name)
        self._op = make_inner_op()
        self._norm = hk.BatchNorm(create_scale=False, create_offset=True, decay_rate=0.999,
                                   eps=1e-3)
        self._non_linearity = non_linearity

    def __call__(self, x: Tensor, call_args: CallArgs):
        x = self._op(x)
        x = self._norm(x, is_training=call_args.is_training,
                        test_local_stats=call_args.test_local_stats)
        return self._non_linearity(x)

class ResBlock(hk.Module):
    r"""Creates a residual block with an optional bottleneck."""

    def __init__(self, stack_size: int, make_first_op, make_inner_op, make_last_op, name):
        super().__init__(name=name)
        assert stack_size >= 2

        self._blocks = []
        for i, make_op in enumerate([make_first_op] + [make_inner_op] *
                                     (stack_size - 2) + [make_last_op]):
            self._blocks.append(
                BasicBlock(
                    make_inner_op=make_op,
                    non_linearity=lambda x: x if i == stack_size - 1 else jax.nn.relu,
                    name=f'basic_{i}'))

    def __call__(self, x: Tensor, call_args: CallArgs):
        res = x
        for b in self._blocks:
            res = b(res, call_args)
        return jax.nn.relu(x + res)

class BroadcastResBlock(ResBlock):
    """A residual block that broadcasts information across spatial dimensions.

    The block consists of a sequence of three layers:
    - a layer that mixes information across channels, e.g. a 1x1 convolution.
    - a layer that mixes information within each channel, a dense layer.
    - another layer to mix across channels.

    The same set of weights is used for mixing information within each channel.
    """

    def __init__(self, make_mix_channel_op, name):

        def broadcast(x: jnp.ndarray):
            n, h, w, c = x.shape

            # Process all planes at once, applynig the same linear layer to each.
            x = x.transpose((0, 3, 1, 2)) # NHWC -> NCHW
            x = x.reshape((n, c, h * w))
            x = hk.Linear(h * w, name='broadcast')(x)
            x = jax.nn.relu(x)
            x = x.reshape((n, c, h, w))
            x = x.transpose((0, 2, 3, 1)) # NCHW -> NHWC
            return x

        super().__init__(
            stack_size=3,
            make_first_op=make_mix_channel_op,
            make_inner_op=lambda: broadcast,
            make_last_op=make_mix_channel_op,
            name=name)

```

Figure 11: Bottleneck and broadcast blocks used by the board game network, implemented in JAX (Bradbury et al., 2018) with the neural network library Haiku.

```

class PoolResBlock(hk.Module):
    """A residual block that broadcasts information across spatial dimensions.

    The block consists of a sequence of three layers:
    - a layer that mixes information across channels, e.g. a 1x1 convolution.
    - a layer that mixes information within each channel, a dense layer.
    - another layer to mix across channels.

    The same set of weights is used for mixing information within each channel.
    """

    def __init__(self,
                  make_mix_channel_op: MakeForwardModule,
                  name='pool'):
        super().__init__(name=name)
        self._block = functools.partial(
            BasicBlock, make_inner_op=make_mix_channel_op)

    def __call__(self, x: Tensor, call_args: CallArgs):
        a = self._block(non_linearity=jax.nn.relu, name='input_a')(x, call_args)
        b = self._block(non_linearity=jax.nn.relu, name='input_b')(x, call_args)

        b_planes = jnp.concatenate([jnp.mean(b, (1, 2)), jnp.max(b, (1, 2))], -1)
        b_planes = hk.Linear(a.shape[-1], name='mix_channels')(b_planes)
        c = a + b_planes[:, None, None, :]

        x = x + self._block(non_linearity=lambda x: x, name='output')(c, call_args)
        return jax.nn.relu(x)

```

Figure 12: Pooling block, based on Wu (2019).

```

def make_conv(output_channels: int, kernel_shape: int):
    return functools.partial(hk.Conv2D, output_channels, kernel_shape, with_bias=False,
                             w_init=hk.initializers.TruncatedNormal(0.01))

def make_network(num_layers: int, output_channels: int, bottleneck_channels: int,
                 broadcast_every_n: int):
    blocks = [
        BasicBlock(make_inner_op=make_conv(output_channels), non_linearity=jax.nn.relu,
                    name='init_conv')
    ]

    for i in range(num_layers):
        if broadcast_every_n > 0 and i % broadcast_every_n == broadcast_every_n - 1:
            blocks.append(BroadcastResBlock(
                make_mix_channel_op=make_conv(output_channels, kernel_shape=1),
                name=f'broadcast_{i}'))
        elif bottleneck_channels > 0:
            blocks.append(ResBlock(
                stack_size=4,
                make_first_op=make_conv(bottleneck_channels, kernel_shape=1),
                make_inner_op=make_conv(bottleneck_channels, kernel_shape=3),
                make_last_op=make_conv(output_channels, kernel_shape=1),
                name=f'bottleneck_res_{i}'))
        else:
            blocks.append(ResBlock(
                stack_size=2,
                make_first_op=make_conv(output_channels, kernel_shape=3),
                make_inner_op=make_conv(output_channels, kernel_shape=3),
                make_last_op=make_conv(output_channels, kernel_shape=3),
                name=f'res_{i}'))

    return blocks

```

Figure 13: Usage of modules defined in Figure 11 to create the network used for board game experiments. Alternatively, PoolResBlock can be used instead of BroadcastResBlock.

Combining Deep Reinforcement Learning and Search for Imperfect-Information Games

Noam Brown* Anton Bakhtin* Adam Lerer Qucheng Gong
Facebook AI Research
{noambrown,yolo,alerer,qucheng}@fb.com

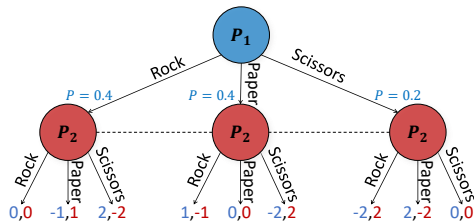
Abstract

The combination of deep reinforcement learning and search at both training and test time is a powerful paradigm that has led to a number of successes in single-agent settings and perfect-information games, best exemplified by AlphaZero. However, prior algorithms of this form cannot cope with imperfect-information games. This paper presents ReBeL, a general framework for self-play reinforcement learning and search that provably converges to a Nash equilibrium in any two-player zero-sum game. In the simpler setting of perfect-information games, ReBeL reduces to an algorithm similar to AlphaZero. Results in two different imperfect-information games show ReBeL converges to an approximate Nash equilibrium. We also show ReBeL achieves superhuman performance in heads-up no-limit Texas hold'em poker, while using far less domain knowledge than any prior poker AI.

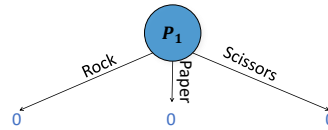
1 Introduction

Combining reinforcement learning with search at both training and test time (**RL+Search**) has led to a number of major successes in AI in recent years. For example, the AlphaZero algorithm achieves state-of-the-art performance in the perfect-information games of Go, chess, and shogi [55].

However, prior RL+Search algorithms do not work in imperfect-information games because they make a number of assumptions that no longer hold in these settings. An example of this is illustrated in Figure 1a, which shows a modified form of Rock-Paper-Scissors in which the winner receives two points (and the loser loses two points) when either player chooses Scissors [15]. The figure shows the game in a sequential form in which player 2 acts after player 1 but does not observe player 1's action.



(1a) Variant of Rock-Paper-Scissors in which the optimal player 1 policy is (R=0.4, P=0.4, S=0.2). Terminal values are color-coded. The dotted lines mean player 2 does not know which node they are in.



(1b) The player 1 subgame when using perfect-information one-ply search. Leaf values are determined by the full-game equilibrium. There is insufficient information for finding (R=0.4, P=0.4, S=0.2).

The optimal policy for both players in this modified version of the game is to choose Rock and Paper with 40% probability, and Scissors with 20%. In that case, each action results in an expected value of zero. However, as shown in Figure 1b, if player 1 were to conduct one-ply lookahead search as is

*Equal contribution

done in perfect-information games (in which the equilibrium value of a state is substituted at a leaf node), then there would not be enough information for player 1 to arrive at this optimal policy.

This illustrates a critical challenge of imperfect-information games: unlike perfect-information games and single-agent settings, the value of an action may depend on the probability it is chosen. Thus, a state defined only by the sequence of actions and observations does not have a unique value and therefore existing RL+Search algorithms such as AlphaZero are not sound in imperfect-information games. Recent AI breakthroughs in imperfect-information games have highlighted the importance of search at test time [40, 12, 14, 37], but combining RL and search during training in imperfect-information games has been an open problem.

This paper introduces ReBeL (Recursive Belief-based Learning), a general RL+Search framework that converges to a Nash equilibrium in two-player zero-sum games. ReBeL builds on prior work in which the notion of “state” is expanded to include the probabilistic belief distribution of all agents about what state they may be in, based on common knowledge observations and policies for all agents. Our algorithm trains a value network and a policy network for these expanded states through self-play reinforcement learning. Additionally, the algorithm uses the value and policy network for search during self play.

ReBeL provably converges to a Nash equilibrium in all two-player zero-sum games. In perfect-information games, ReBeL simplifies to an algorithm similar to AlphaZero, with the major difference being in the type of search algorithm used. Experimental results show that ReBeL is effective in large-scale games and defeats a top human professional with statistical significance in the benchmark game of heads-up no-limit Texas hold’em poker while using far less expert domain knowledge than any previous poker AI. We also show that ReBeL approximates a Nash equilibrium in Liar’s Dice, another benchmark imperfect-information game, and open source our implementation of it.²

2 Related Work

At a high level, ReBeL resembles past RL+Search algorithms used in perfect-information games [59, 56, 1, 55, 50]. These algorithms train a value network through self play. During training, a search algorithm is used in which the values of leaf nodes are determined via the value function. Additionally, a policy network may be used to guide search. These forms of RL+Search have been critical to achieving superhuman performance in benchmark perfect-information games. For example, so far no AI agent has achieved superhuman performance in Go without using search at both training and test time. However, these RL+Search algorithms are not theoretically sound in imperfect-information games and have not been shown to be successful in such settings.

A critical element of our imperfect-information RL+Search framework is to use an expanded notion of “state”, which we refer to as a **public belief state (PBS)**. PBSs are defined by a common-knowledge belief distribution over states, determined by the public observations shared by all agents and the policies of all agents. PBSs can be viewed as a multi-agent generalization of belief states used in partially observable Markov decision processes (POMDPs) [31]. The concept of PBSs originated in work on decentralized multi-agent POMDPs [43, 45, 19] and has been widely used since then in imperfect-information games more broadly [40, 20, 53, 29].

ReBeL builds upon the idea of using a PBS value function during search, which was previously used in the poker AI DeepStack [40]. However, DeepStack’s value function was trained not through self-play RL, but rather by generating random PBSs, including random probability distributions, and estimating their values using search. This would be like learning a value function for Go by randomly placing stones on the board. This is not an efficient way of learning a value function because the vast majority of randomly generated situations would not be relevant in actual play. DeepStack coped with this by using handcrafted features to reduce the dimensionality of the public belief state space, by sampling PBSs from a distribution based on expert domain knowledge, and by using domain-specific abstractions to circumvent the need for a value network when close to the end of the game.

An alternative approach for depth-limited search in imperfect-information games that does not use a value function for PBSs was used in the Pluribus poker AI to defeat elite humans in multiplayer poker [15, 14]. This approach trains a population of “blueprint” policies without using search. At test time, the approach conducts depth-limited search by allowing each agent to choose a blueprint policy from the population at leaf nodes. The value of the leaf node is the expected value of each

²<https://github.com/facebookresearch/rebel>

agent playing their chosen blueprint policy against all the other agents' choice for the rest of the game. While this approach has been successful in poker, it does not use search during training and therefore requires strong blueprint policies to be computed without search. Also, the computational cost of the search algorithm grows linearly with the number of blueprint policies.

3 Notation and Background

We assume that the rules of the game and the agents' policies (including search algorithms) are **common knowledge** [2].³ That is, they are known by all agents, all agents know they are known by all agents, etc. However, the outcome of stochastic algorithms (i.e., the random seeds) are not known. In Section 6 we show how to remove the assumption that we know another player's policy.

Our notation is based on that of factored observation games [34] which is a modification of partially observable stochastic games [26] that distinguishes between private and public observations. We consider a game with $\mathcal{N} = \{1, 2, \dots, N\}$ agents.

A **world state** $w \in \mathcal{W}$ is a state in the game. $\mathcal{A} = \mathcal{A}_1 \times \mathcal{A}_2 \times \dots \times \mathcal{A}_N$ is the space of joint actions. $\mathcal{A}_i(w)$ denotes the legal actions for agent i at w and $a = (a_1, a_2, \dots, a_N) \in \mathcal{A}$ denotes a joint action. After a joint action a is chosen, a transition function $\mathcal{T}$ determines the next world state w' drawn from the probability distribution $\mathcal{T}(w, a) \in \Delta\mathcal{W}$. After joint action a , agent i receives a reward $\mathcal{R}_i(w, a)$.

Upon transition from world state w to w' via joint action a , agent i receives a **private observation** from a function $\mathcal{O}_{\text{priv}(i)}(w, a, w')$. Additionally, all agents receive a **public observation** from a function $\mathcal{O}_{\text{pub}}(w, a, w')$. Public observations may include observations of publicly taken actions by agents. For example, in many recreational games, including poker, all betting actions are public.

A **history** (also called a trajectory) is a finite sequence of legal actions and world states, denoted $h = (w^0, a^0, w^1, a^1, \dots, w^t)$. An **infostate** (also called an **action-observation history (AOH)**) for agent i is a sequence of an agent's observations and actions $s_i = (O_i^0, a_i^0, O_i^1, a_i^1, \dots, O_i^t)$ where $O_i^k = (\mathcal{O}_{\text{priv}(i)}(w^{k-1}, a^{k-1}, w^k), \mathcal{O}_{\text{pub}}(w^{k-1}, a^{k-1}, w^k))$. The unique infostate corresponding to a history h for agent i is denoted $s_i(h)$. The set of histories that correspond to s_i is denoted $\mathcal{H}(s_i)$.

A **public state** is a sequence $s_{\text{pub}} = (O_{\text{pub}}^0, O_{\text{pub}}^1, \dots, O_{\text{pub}}^t)$ of public observations. The unique public state corresponding to a history h and an infostate s_i is denoted $s_{\text{pub}}(h)$ and $s_{\text{pub}}(s_i)$, respectively. The set of histories that match the sequence of public observation of s_{pub} is denoted $\mathcal{H}(s_{\text{pub}})$.

For example, consider a game where two players roll two six-sided dice each. One die of each player is publicly visible; the other die is only observed by the player who rolled it. Suppose player 1 rolls a 3 and a 4 (with 3 being the hidden die), and player 2 rolls a 5 and a 6 (with 5 being the hidden die). The history (and world state) is $h = ((3, 4), (5, 6))$. The set of histories corresponding to player 2's infostate is $\mathcal{H}(s_2) = \{((x, 4), (5, 6)) \mid x \in \{1, 2, 3, 4, 5, 6\}\}$, so $|\mathcal{H}(s_2)| = 6$. The set of histories corresponding to s_{pub} is $\mathcal{H}(s_{\text{pub}}) = \{((x, 4), (y, 6)) \mid x, y \in \{1, 2, 3, 4, 5, 6\}\}$, so $|\mathcal{H}(s_{\text{pub}})| = 36$.

Public states provide an easy way to reason about common knowledge in a game. All agents observe the same public sequence s_{pub} , and therefore it is common knowledge among all agents that the true history is some $h \in \mathcal{H}(s_{\text{pub}})$.⁴

An agent's **policy** π_i is a function mapping from an infostate to a probability distribution over actions. A **policy profile** π is a tuple of policies $(\pi_1, \pi_2, \dots, \pi_N)$. The expected sum of future rewards (also called the **expected value (EV)**) for agent i in history h when all agents play policy profile π is denoted $v_i^\pi(h)$. The EV for the entire game is denoted $v_i(\pi)$. A **Nash equilibrium** is a policy profile such that no agent can achieve a higher EV by switching to a different policy [42]. Formally, π^* is a Nash equilibrium if for every agent i , $v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi_{-i}^*)$ where π_{-i} denotes the policy of all agents other than i . A **Nash equilibrium policy** is a policy π_i^* that is part of some Nash equilibrium π^* .

A **subgame** is defined by a root history h in a perfect-information game and all histories that can be reached going forward. In other words, it is identical to the original game except it starts at h .

³This is a common assumption in game theory. One argument for it is that in repeated play an adversary would eventually determine an agent's policy.

⁴As explained in [34], it may be possible for agents to infer common knowledge beyond just public observations. However, doing this additional reasoning is inefficient both theoretically and practically.

A **depth-limited subgame** is a subgame that extends only for a limited number of actions into the future. Histories at the bottom of a depth-limited subgame (i.e., histories that have no legal actions in the depth-limited subgame) but that have at least one legal action in the full game are called **leaf nodes**. In this paper, we assume for simplicity that search is performed over fixed-size depth-limited subgame (as opposed to Monte Carlo Tree Search, which grows the subgame over time [23]).

A game is **two-player zero-sum (2p0s)** if there are exactly two players and $\mathcal{R}_1(w, a) = -\mathcal{R}_2(w, a)$ for every world state w and action a . In 2p0s perfect-information games, there always exists a Nash equilibrium that depends only on the current world state w rather than the entire history h . Thus, in 2p0s perfect-information games a policy can be defined for world states and a subgame can be defined as rooted at a world state. Additionally, in 2p0s perfect-information games every world state w has a unique value $v_i(w)$ for each agent i , where $v_1(w) = -v_2(w)$, defined by both agents playing a Nash equilibrium in any subgame rooted at that world state. Our theoretical and empirical results are limited to 2p0s games, though related techniques have been empirically successful in some settings with more players [14]. A typical goal for RL in 2p0s perfect-information games is to learn v_i . With that value function, an agent can compute its optimal next move by solving a depth-limited subgame that is rooted at its current world state and where the value of every leaf node z is set to $v_i(z)$ [54, 49].

4 From World States to Public Belief States

In this section we describe a mechanism for converting any imperfect-information game into a continuous state (and action) space perfect-information game where the state description contains the probabilistic belief distribution of all agents. In this way, techniques that have been applied to perfect-information games can also be applied to imperfect-information games (with some modifications).

For intuition, consider a game in which one of 52 cards is privately dealt to each player. On each turn, a player chooses between three actions: fold, call, and raise. Eventually the game ends and players receive a reward. Now consider a modification of this game in which the players cannot see their private cards; instead, their cards are seen by a “referee”. On a player’s turn, they announce the probability they would take each action with each possible private card. The referee then samples an action on the player’s behalf from the announced probability distribution for the player’s true private card. When this game starts, each player’s belief distribution about their private card is uniform random. However, after each action by the referee, players can update their belief distribution about which card they are holding via Bayes’ Rule. Likewise, players can update their belief distribution about the *opponent’s* private card through the same operation. Thus, the probability that each player is holding each private card is common knowledge among all players at all times in this game.

A critical insight is that *these two games are strategically identical*, but the latter contains no private information and is instead a continuous state (and action) space perfect-information game. While players do not announce their action probabilities for each possible card in the first game, we assume (as stated in Section 3) that all players’ policies are common knowledge, and therefore the probability that a player would choose each action for each possible card is indeed known by all players. Of course, at test time (e.g., when our agent actually plays against a human opponent) the opponent does not actually announce their entire policy and therefore our agent does not know the true probability distribution over opponent cards. We later address this problem in Section 6.

We refer to the first game as the **discrete representation** and the second game as the **belief representation**. In the example above, a history in the belief representation, which we refer to as a **public belief state (PBS)**, is described by the sequence of public observations and 104 probabilities (the probability that each player holds each of the 52 possible private card); an “action” is described by 156 probabilities (one per discrete action per private card). In general terms, a PBS is described by a joint probability distribution over the agents’ possible infostates [43, 45, 19].⁵ Formally, let $S_i(s_{\text{pub}})$ be the set of infostates that player i may be in given a public state s_{pub} and let $\Delta S_1(s_{\text{pub}})$ denote a probability distribution over the elements of $S_1(s_{\text{pub}})$. Then $\text{PBS } \beta = (\Delta S_1(s_{\text{pub}}), \dots, \Delta S_N(s_{\text{pub}}))$.⁶ In perfect-information games, the discrete representation and belief representation are identical.

⁵One could alternatively define a PBS as a probability distribution over histories in $\mathcal{H}(s_{\text{pub}})$ for public state s_{pub} . However, it is proven that any PBS that can arise in play can always be described by a joint probability distribution over the agents’ possible infostates [45, 52], so we use this latter definition for simplicity.

⁶Frequently, a PBS can be compactly summarized by discarding parts of the history that are no longer relevant. For example, in poker we do not need to track the entire history of actions, but just the amount of money each player has in the pot, the public board cards, and whether there were any bets in the current round.

Since a PBS is a history of the perfect-information belief-representation game, a subgame can be rooted at a PBS.⁷ The discrete-representation interpretation of such a subgame is that at the start of the subgame a history is sampled from the joint probability distribution of the PBS, and then the game proceeds as it would in the original game. The value for agent i of PBS β when all players play policy profile π is $V_i^\pi(\beta) = \sum_{h \in \mathcal{H}(s_{\text{pub}}(\beta))} p(h|\beta) v_i^\pi(h)$. Just as world states have unique values in 2p0s perfect-information games, in 2p0s games (both perfect-information and imperfect-information) every PBS β has a unique value $V_i(\beta)$ for each agent i , where $V_1(\beta) = -V_2(\beta)$, defined by both players playing a Nash equilibrium in the subgame rooted at the PBS.

Since any imperfect-information game can be viewed as a perfect-information game consisting of PBSs (i.e., the belief representation), in theory we could approximate a solution of any 2p0s imperfect-information game by running a perfect-information RL+Search algorithm on a discretization of the belief representation. However, as shown in the example above, belief representations can be very high-dimensional continuous spaces, so conducting search (i.e., approximating the optimal policy in a depth-limited subgame) as is done in perfect-information games would be intractable. Fortunately, in 2p0s games, *these high-dimensional belief representations are convex optimization problems*. ReBeL leverages this fact by conducting search via an iterative gradient-ascent-like algorithm.

ReBeL’s search algorithm operates on supergradients (the equivalent of subgradients but for concave functions) of the PBS value function at leaf nodes, rather than on PBS values directly. Specifically, the search algorithms require the values of *infostates* for PBSs [16, 40]. In a 2p0sum game, the value of infostate s_i in β assuming all other players play Nash equilibrium π^* is the maximum value that player i could obtain for s_i through any policy in the subgame rooted at β . Formally,

$$v_i^{\pi^*}(s_i|\beta) = \max_{\pi_i} \sum_{h \in \mathcal{H}(s_i)} p(h|s_i, \beta_{-i}) v_i^{\langle \pi_i, \pi_{-i}^* \rangle}(h) \quad (1)$$

where $p(h|s_i, \beta_{-i})$ is the probability of being in history h assuming s_i is reached and the joint probability distribution over infostates for players other than i is β_{-i} . Theorem 1 proves that infostate values can be interpreted as a supergradient of the PBS value function in 2p0s games.

Theorem 1. *For any PBS $\beta = (\beta_1, \beta_2)$ (for the beliefs over player 1 and 2 infostates respectively) and any policy π^* that is a Nash equilibrium of the subgame rooted at β ,*

$$v_1^{\pi^*}(s_1|\beta) = V_1(\beta) + \bar{g} \cdot \hat{s}_1 \quad (2)$$

where $\bar{g}$ is a supergradient of an extension of $V_1(\beta)$ to unnormalized belief distributions and $\hat{s}_1$ is the unit vector in direction s_1 .

All proofs are presented in the appendix.

Since ReBeL’s search algorithm uses infostate values, so rather than learn a PBS value function ReBeL instead learns an infostate-value function $\hat{v} : \mathcal{B} \rightarrow \mathbb{R}^{|S_1|+|S_2|}$ that directly approximates for each s_i the average of the sampled $v_i^{\pi^*}(s_i|\beta)$ values produced by ReBeL at β .⁸

5 Self Play Reinforcement Learning and Search for Public Belief States

In this section we describe ReBeL and prove that it approximates a Nash equilibrium in 2p0s games. At the start of the game, a depth-limited subgame rooted at the initial PBS β_r is generated. This subgame is solved (i.e., a Nash equilibrium is approximated) by running T iterations of an iterative equilibrium-finding algorithm in the discrete representation of the game, but using the learned value network $\hat{v}$ to approximate leaf values on every iteration. During training, the infostate values at β_r computed during search are added as training examples for $\hat{v}$ and (optionally) the subgame policies are added as training examples for the policy network. Next, a leaf node z is sampled and the process repeats with the PBS at z being the new subgame root. Appendix B shows detailed pseudocode.

⁷Past work defines a subgame to be rooted at a public state [16, 8, 41, 40, 11, 33, 57, 52]. However, imperfect-information subgames rooted at a public state do not have well-defined values.

⁸Unlike the PBS value $V_i(\beta)$, the infostate values may not be unique and may depend on which Nash equilibrium is played in the subgame. Nevertheless, any linear combination of supergradients is itself a supergradient since the set of all supergradients is a convex set [48].

Algorithm 1 ReBeL: RL and Search for Imperfect-Information Games

```
function SELFPLAY( $\beta_r, \theta^v, \theta^\pi, D^v, D^\pi$ ) ▷  $\beta_r$  is the current PBS
  while !IS TERMINAL( $\beta_r$ ) do
     $G \leftarrow \text{CONSTRUCTSUBGAME}(\beta_r)$ 
     $\bar{\pi}, \pi^{t_{\text{warm}}} \leftarrow \text{INITIALIZEPOLICY}(G, \theta^\pi)$  ▷  $t_{\text{warm}} = 0$  and  $\pi^0$  is uniform if no warm start
     $G \leftarrow \text{SETLEAFVALUES}(G, \bar{\pi}, \pi^{t_{\text{warm}}}, \theta^v)$ 
     $v(\beta_r) \leftarrow \text{COMPUTE EV}(G, \pi^{t_{\text{warm}}})$ 
     $t_{\text{sample}} \sim \text{unif}\{t_{\text{warm}} + 1, T\}$  ▷ Sample an iteration
    for  $t = (t_{\text{warm}} + 1) \dots T$  do
      if  $t = t_{\text{sample}}$  then
         $\beta_r' \leftarrow \text{SAMPLELEAF}(G, \pi^{t-1})$  ▷ Sample one or multiple leaf PBSs
         $\pi^t \leftarrow \text{UPDATEPOLICY}(G, \pi^{t-1})$ 
         $\bar{\pi} \leftarrow \frac{t}{t+1} \bar{\pi} + \frac{1}{t+1} \pi^t$ 
         $G \leftarrow \text{SETLEAFVALUES}(G, \bar{\pi}, \pi^t, \theta^v)$ 
         $v(\beta_r) \leftarrow \frac{t}{t+1} v(\beta_r) + \frac{1}{t+1} \text{COMPUTE EV}(G, \pi^t)$ 
      Add  $\{\beta_r, v(\beta_r)\}$  to  $D^v$  ▷ Add to value net training data
      for  $\beta \in G$  do ▷ Loop over the PBS at every public state in  $G$ 
        Add  $\{\beta, \bar{\pi}(\beta)\}$  to  $D^\pi$  ▷ Add to policy net training data (optional)
       $\beta_r \leftarrow \beta_r'$ 
```

5.1 Search in a depth-limited imperfect-information subgame

In this section we describe the search algorithm ReBeL uses to solve depth-limited subgames. We assume for simplicity that the depth of the subgame is pre-determined and fixed. The subgame is solved in the discrete representation and the solution is then converted to the belief representation. There exist a number of iterative algorithms for solving imperfect-information games [5, 61, 28, 36, 35]. We describe ReBeL assuming the **counterfactual regret minimization - decomposition (CFR-D)** algorithm is used [61, 16, 40]. CFR is the most popular equilibrium-finding algorithm for imperfect-information games, and CFR-D is an algorithm that solves depth-limited subgames via CFR. However, ReBeL is flexible with respect to the choice of search algorithm and in Section 8 we also show experimental results for **fictitious play (FP)** [5].

On each iteration t , CFR-D determines a policy profile π^t in the subgame. Next, the value of every discrete representation leaf node z is set to $\hat{v}(s_i(z) | \beta_z^{\pi^t})$, where $\beta_z^{\pi^t}$ denotes the PBS at z when agents play according to π^t . This means that the value of a leaf node during search is conditional on π^t . Thus, the leaf node values change every iteration. Given π^t and the leaf node values, each infostate in β_r has a well-defined value. This vector of values, denoted $v^{\pi^t}(\beta_r)$, is stored. Next, CFR-D chooses a new policy profile π^{t+1} , and the process repeats for T iterations.

When using CFR-D, the *average* policy profile $\bar{\pi}^T$ converges to a Nash equilibrium as $T \rightarrow \infty$, rather than the policy on the final iteration. Therefore, after running CFR-D for T iterations in the subgame rooted at PBS β_r , the value vector $(\sum_{t=1}^T v^{\pi^t}(\beta_r))/T$ is added to the training data for $\hat{v}(\beta_r)$.

Appendix I introduces **CFR-AVG**, a modification of CFR-D that sets the value of leaf node z to $\hat{v}(s_i(z) | \beta_z^{\bar{\pi}^t})$ rather than $\hat{v}(s_i(z) | \beta_z^{\pi^t})$, where $\bar{\pi}^t$ denotes the average policy profile up to iteration t . CFR-AVG addresses some weaknesses of CFR-D.

5.2 Self-play reinforcement learning

We now explain how ReBeL trains a PBS value network through self play. After solving a subgame rooted at PBS β_r via search (as described in Section 5.1), the value vector for the root infostates is added to the training dataset for $\hat{v}$. Next, a leaf PBS β_r' is sampled and a new subgame rooted at β_r' is solved. This process repeats until the game ends.

Since the subgames are solved using an iterative algorithm, we want $\hat{v}$ to be accurate for leaf PBSs on every iteration. Therefore, a leaf node z is sampled according to π^t on a *random* iteration $t \sim \text{unif}\{0, T-1\}$, where T is the number of iterations of the search algorithm.⁹ To ensure sufficient

⁹For FP, we pick a random agent i and sample according to $(\pi_i^t, \bar{\pi}_{-i}^t)$ to reflect the search operation.

exploration, one agent samples random actions with probability $\epsilon > 0$.¹⁰ In CFR-D $\beta'_r = \beta_z^{\pi^t}$, while in CFR-AVG and FP $\beta'_r = \beta_z^{\pi^t}$.

Theorem 2 states that, with perfect function approximation, running Algorithm 1 will produce a value network whose error is bounded by $\mathcal{O}(\frac{1}{\sqrt{T}})$ for any PBS that could be encountered during play, where T is the number of CFR iterations being run in subgames.

Theorem 2. *Consider an idealized value approximator that returns the most recent sample of the value for sampled PBSs, and 0 otherwise. Running Algorithm 1 with T iterations of CFR in each subgame will produce a value approximator that has error of at most $\frac{C}{\sqrt{T}}$ for any PBS that could be encountered during play, where C is a game-dependent constant.*

ReBeL as described so far trains the value network through bootstrapping. One could alternatively train the value network using rewards actually received over the course of the game when the agents do not go off-policy. There is a trade-off between bias and variance between these two approaches [51].

5.3 Adding a policy network

Algorithm 1 will result in $\hat{v}$ converging correctly even if a policy network is not used. However, initializing the subgame policy via a policy network may reduce the number of iterations needed to closely approximate a Nash equilibrium. Additionally, it may improve the accuracy of the value network by allowing the value network to focus on predicting PBS values over a more narrow domain.

Algorithm 1 can train a policy network $\hat{\Pi} : \beta \rightarrow (\Delta\mathcal{A})^{|S_1|+|S_2|}$ by adding $\bar{\pi}^T(\beta)$ for each PBS β in the subgame to a training dataset each time a subgame is solved (i.e., T iterations of CFR have been run in the subgame). Appendix J describes a technique, based on [10], for warm starting equilibrium finding given the initial policy from the policy network.

6 Playing According to an Equilibrium at Test Time

This section proves that running Algorithm 1 at test time with an accurately trained PBS value network will result in playing a Nash equilibrium policy in expectation even if we do not know the opponent’s policy. During self play training we assumed, as stated in Section 3, that both players’ policies are common knowledge. This allows us to exactly compute the PBS we are in. However, at test time we do not know our opponent’s entire policy, and therefore we do not know the PBS. This is a problem for conducting search, because search is always rooted at a PBS. For example, consider again the game of modified Rock-Paper-Scissors illustrated in Figure 1a. For simplicity, assume that $\hat{v}$ is perfect. Suppose that we are player 2 and player 1 has just acted. In order to now conduct search as player 2, our algorithm requires a root PBS. What should this PBS be?

An intuitive choice, referred to as **unsafe** search [25, 22], is to first run CFR for T iterations for player 1’s first move (for some large T), which results in a player 1 policy such as $(R = 0.4001, P = 0.3999, S = 0.2)$. Unsafe search passes down the beliefs resulting from that policy, and then computes our optimal policy as player 2. This would result in a policy of $(R = 0, P = 1, S = 0)$ for player 2. Clearly, this is not a Nash equilibrium. Moreover, if our opponent knew we would end up playing this policy (which we assume they would know since we assume they know the algorithm we run to generate the policy), then they could exploit us by playing $(R = 0, P = 0, S = 1)$.

This problem demonstrates the need for **safe** search, which is a search algorithm that ensures we play a Nash equilibrium policy in expectation. Importantly, it is *not* necessary for the policy that the algorithm outputs to always be a Nash equilibrium. It is only necessary that the algorithm outputs a Nash equilibrium policy *in expectation*. For example, in modified Rock-Paper-Scissors it is fine for an algorithm to output a policy of 100% Rock, so long as the probability it outputs that policy is 40%.

All past safe search approaches introduce additional constraints to the search algorithm [16, 41, 11, 57]. Those additional constraints hurt performance in practice compared to unsafe search [16, 11] and greatly complicate search, so they were never fully used in any competitive agent. Instead, all previous search-based imperfect-information game agents used unsafe search either partially or entirely [40, 12, 15, 14, 53]. Moreover, using prior safe search techniques at test time may result in the agent encountering PBSs that were not encountered during self-play training and therefore may result in poor approximations from the value and policy network.

¹⁰The algorithm is still correct if all agents sample random actions with probability ϵ , but that is less efficient because the value of a leaf node that can only be reached if both agents go off policy is irrelevant.

We now prove that safe search can be achieved without any additional constraints by simply *running the same algorithm at test time that we described for training*. This result applies regardless of how the value network was trained and so can be applied to prior algorithms that use PBS value functions [40, 53]. Specifically, when conducting search at test time we pick a *random* iteration and assume all players’ policies match the policies on that iteration. Theorem 3, the proof of which is in Section G, states that once a value network is trained according to Theorem 2, using Algorithm 1 at test time (without off-policy exploration) will approximate a Nash equilibrium.

Theorem 3. *If Algorithm 1 is run at test time with no off-policy exploration, a value network with error at most δ for any leaf PBS that was trained to convergence as described in Theorem 2, and with T iterations of CFR being used to solve subgames, then the algorithm plays a $(\delta C_1 + \frac{\delta C_2}{\sqrt{T}})$ -Nash equilibrium, where C_1, C_2 are game-specific constants.*

Since a random iteration is selected, we may select a very early iteration, or even the first iteration, in which the policy is extremely poor. This can be mitigated by using modern equilibrium-finding algorithms, such as Linear CFR [13], that assign little or no weight to the early iterations.

7 Experimental Setup

We measure **exploitability** of a policy π^* , which is $\sum_{i \in \mathcal{N}} \max_{\pi} v_i(\pi, \pi^*_{-i}) / |\mathcal{N}|$. All CFR experiments use alternating-updates Linear CFR [13]. All FP experiments use alternating-updates Linear Optimistic FP, which is a novel variant we present in Appendix H.

We evaluate on the benchmark imperfect-information games of heads-up no-limit Texas hold’em poker (HUNL) and Liar’s Dice. The rules for both games are provided in Appendix C. We also evaluate our techniques on turn endgame hold’em (TEH), a variant of no-limit Texas hold’em in which both players automatically check/call for the first two of the four betting rounds in the game.

In HUNL and TEH, we reduce the action space to at most nine actions using domain knowledge of typical bet sizes. However, our agent responds to any “off-tree” action at test time by adding the action to the subgame [15, 14]. The bet sizes and stack sizes are randomized during training. For TEH we train on the full game and measure exploitability on the case of both players having \$20,000, unperturbed bet sizes, and the first four board cards being $3\spadesuit 7\heartsuit T\diamond K\spadesuit$. For HUNL, our agent uses far less domain knowledge than any prior competitive AI agent. Appendix D discusses the poker domain knowledge we leveraged in ReBeL.

We approximate the value and policy functions using artificial neural networks. Both networks are MLPs with GeLU [27] activation functions and LayerNorm [3]. Both networks are trained with Adam [32]. We use pointwise Huber loss as the criterion for the value function and mean squared error (MSE) over probabilities for the policy. In preliminary experiments we found MSE for the value network and cross entropy for the policy network did worse. See Appendix E for the hyperparameters.

We use PyTorch [46] to train the networks. We found data generation to be the bottleneck due to the sequential nature of the FP and CFR algorithms and the evaluation of all leaf nodes on each iteration. For this reason we use a single machine for training and up to 128 machines with 8 GPUs each for data generation.

8 Experimental Results

Figure 2 shows ReBeL reaches a level of exploitability in TEH equivalent to running about 125 iterations of full-game tabular CFR. For context, top poker agents typically use between 100 and 1,000 tabular CFR iterations [4, 40, 12, 15, 14]. Our self-play algorithm is key to this success; Figure 2 shows a value network trained on random PBSs fails to learn anything valuable.

Table 1 shows results for ReBeL in HUNL. We compare ReBeL to BabyTartanian8 [9] and Slumbot, prior champions of the Computer Poker Competition, and to the local best response (LBR) [39] algorithm. We also present results against Dong Kim, a top human HUNL expert that did best among the four top humans that played against Libratus. Kim played 7,500 hands. Variance was reduced by using AIVAT [17]. ReBeL played faster than 2 seconds per hand and never needed more than 5 seconds for a decision.

Beyond just poker, Table 2 shows ReBeL also converges to an approximate Nash in several versions of Liar’s Dice. Of course, tabular CFR does better than ReBeL when using the same number of CFR iterations, but tabular CFR quickly becomes intractable to run as the game grows in size.

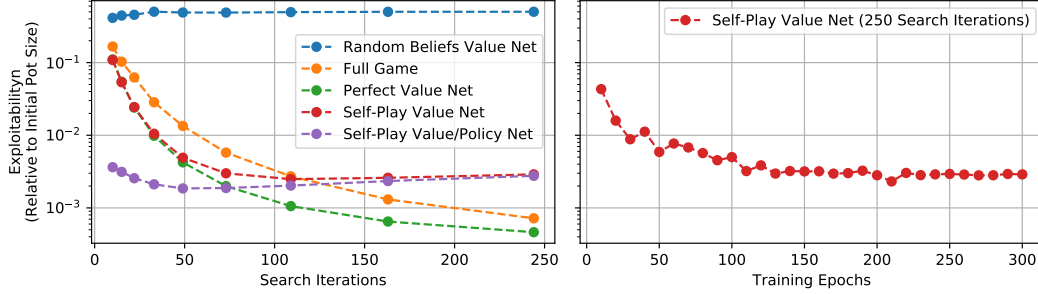


Figure 2: Convergence of different techniques in TEH. All subgames are solved using CFR-AVG. Perfect Value Net uses an oracle function to return the exact value of leaf nodes on each iteration. Self-Play Value Net uses a value function trained through self play. Self-Play Value/Policy Net additionally uses a policy network to warm start CFR. Random Beliefs trains the value net by sampling PBSs at random.

Bot Name	Slumbot	BabyTartanian8 [9]	LBR [39]	Top Humans
DeepStack [40]	-	-	383 $\pm$ 112	-
Libratus [12]	-	63 $\pm$ 14	-	147 $\pm$ 39
Modicum [15]	11 $\pm$ 5	6 $\pm$ 3	-	-
ReBeL (<i>Ours</i>)	45 $\pm$ 5	9 $\pm$ 4	881 $\pm$ 94	165 $\pm$ 69

Table 1: Head-to-head results of our agent against benchmark bots BabyTartanian8 and Slumbot, as well as top human expert Dong Kim, measured in thousandths of a big blind per game. We also show performance against LBR [39] where the LBR agent must call for the first two betting rounds, and can either fold, call, bet $1 \times$ pot, or bet all-in on the last two rounds. The $\pm$ shows one standard deviation. For Libratus, we list the score against all top humans in aggregate; Libratus beat Dong Kim by 29 with an estimated $\pm$ of 78.

Algorithm	1x4f	1x5f	1x6f	2x3f
Full-game FP	0.012	0.024	0.039	0.057
Full-game CFR	0.001	0.001	0.002	0.002
ReBeL FP	0.041	0.020	0.040	0.020
ReBeL CFR-D	0.017	0.015	0.024	0.017

Table 2: Exploitability of different algorithms of 4 variants of Liar’s Dice: 1 die with 4, 5, or 6 faces and 2 dice with 3 faces. The top two rows represent baseline numbers when a tabular version of the algorithms is run on the entire game for 1,024 iterations. The bottom 2 lines show the performance of ReBeL operating on subgames of depth 2 with 1,024 search iterations. For exploitability computation of the bottom two rows, we averaged the policies of 1,024 playthroughs and thus the numbers are upper bounds on exploitability.

9 Conclusions

We present ReBeL, an algorithm that generalizes the paradigm of self-play reinforcement learning and search to imperfect-information games. We prove that ReBeL computes an approximate Nash equilibrium in two-player zero-sum games, demonstrate convergence in Liar’s Dice, and demonstrate that it produces superhuman performance in the benchmark game of heads-up no-limit Texas hold’em.

ReBeL has some limitations that present avenues for future research. Most prominently, the input to its value and policy functions currently grows linearly with the number of infostates in a public state. This is intractable in games such as Recon Chess [44] that have strategic depth but very little common knowledge. ReBeL’s theoretical guarantees are also limited only to two-player zero-sum games.

Nevertheless, ReBeL achieves low exploitability in benchmark games and superhuman performance in heads-up no-limit Texas hold’em while leveraging far less expert knowledge than any prior bot. We view this as a major step toward developing universal techniques for multi-agent interactions.

Broader Impact

We believe ReBeL is a major step toward general equilibrium-finding algorithms that can be deployed in large-scale multi-agent settings while requiring relatively little domain knowledge. There are numerous potential future applications of this work, including auctions, negotiations, cybersecurity, and autonomous vehicle navigation, all of which are imperfect-information multi-agent interactions.

The most immediate risk posed by this work is its potential for cheating in recreational games such as poker. While AI algorithms already exist that can achieve superhuman performance in poker, these algorithms generally assume that participants have a certain number of chips or use certain bet sizes. Retraining the algorithms to account for arbitrary chip stacks or unanticipated bet sizes requires more computation than is feasible in real time. However, ReBeL can compute a policy for arbitrary stack sizes and arbitrary bet sizes in seconds.

Partly for this reason, we have decided not to release the code for poker. We instead open source our implementation for Liar's Dice, a recreational game that is not played as competitively by humans. The implementation in Liar's Dice is also easier to understand and the size of Liar's Dice can be more easily adjusted, which we believe makes the game more suitable as a domain for research.

A List of contributions

This paper makes several contributions, which we summarize here.

- **RL+Search in two-player zero-sum imperfect-information games.** Prior work has developed RL+Search for two-player zero-sum perfect-information games. There has also been prior work on learning value functions in fully cooperative imperfect-information games [19] and limited subsets of zero-sum imperfect-information games [29]. However, we are not aware of any prior RL+Search algorithms for two-player zero-sum games in general. We view this as the central contribution of this paper.
- **Alternative to safe search techniques.** Theorem 3 proves that, when doing search at test time with an accurate PBS value function, one can empirically play according to a Nash equilibrium by sampling a random iteration and passing down the beliefs produced by that iteration’s policy. This result applies regardless of how the value function was trained and therefore applies to earlier techniques that use a PBS value function, such as DeepStack [40].
- **Subgame decomposition via CFR-AVG.** We describe the CFR-AVG algorithm in Appendix I. CFR-D [16] is a way to conduct depth-limited solving of a subgame with CFR when given a value function for PBSs. CFR-D is theoretically sound but has certain properties that may reduce performance in a self-play setting. CFR-AVG is a theoretically sound alternative to CFR-D that does not have these weaknesses. However, in order to implement CFR-AVG efficiently, in our experiments we modify the algorithm in a way that is not theoretically sound but empirically performs well in poker. Whether or not this modified form of CFR-AVG is theoretically sound remains an open question.
- **Connection between PBS gradients and infostate values.** Theorem 1 proves that all of the algorithms described in this paper can in theory be conducted using only V_1 , not $\hat{v}$ (that is, a value function that outputs a single value for a PBS, rather than a vector of values for the infostates in the PBS). While this connection does not have immediate practical consequences, it does point toward a way of deploying the ideas in this paper to settings with billions or more infostates per PBS.
- **Fictitious Linear Optimistic Play (FLOP).** Section H introduces FLOP, a novel variant of Fictitious Play that is inspired by recent work on regret minimization algorithms [13]. We show that FLOP empirically achieves near- $O(\frac{1}{T})$ convergence in the limit in both poker and Liar’s Dice, does far better than any previous variant of FP, and in some domains is a reasonable alternative to CFR.

B Pseudocode for ReBeL

Algorithm 2 presents ReBeL in more detail.

We define the average of two policies to be the policy that is, in expectation, identical to picking one of the two policies and playing that policy for the entire game. Formally, if $\pi = \alpha\pi_1 + (1 - \alpha)\pi_2$, then $\pi(s_i) = \frac{(x_i^{\pi_1}(s_i)\alpha)\pi_1(s_i) + (x_i^{\pi_2}(s_i)(1-\alpha))\pi_2(s_i)}{x_i^{\pi_1}(s_i)\alpha + x_i^{\pi_2}(s_i)(1-\alpha)}$ where $x_i^{\pi_1}(s_i)$ is the product of the probabilities for all agent i actions leading to s_i . Formally, $x_i^\pi(s_i)$ of infostate $s_i = (O_i^0, a_i^0, O_i^1, a_i^1, \dots, O_i^t)$ is $x_i^\pi(s_i) = \prod_t(a_i^t)$.

C Description of Games used for Evaluation

C.1 Heads-up no-limit Texas hold’em poker (HUNL)

HUNL is the two-player version of no-limit Texas hold’em poker, which is the most popular variant of poker in the world. For each “hand” (game) of poker, each player has some number of chips (the *stack*) in front of them. In our experiments, stack size varies during training between \$5,000 and \$25,000 but during testing is always \$20,000, as is standard in the AI research community. Before play begins, Player 1 commits a *small blind* of \$50 to the pot and Player 2 commits a *big blind* of \$100 to the pot.

Algorithm 2 ReBeL

```

function REBEL-LINEAR-CFR-D( $\beta_r, \theta^v, \theta^\pi, D^v, D^\pi$ ) ▷  $\beta_r$  is the current PBS
  while !IS TERMINAL( $\beta_r$ ) do
     $G \leftarrow \text{CONSTRUCTSUBGAME}(\beta_r)$ 
     $\bar{\pi}, \pi^{t_{\text{warm}}} \leftarrow \text{INITIALIZEPOLICY}(G, \theta^\pi)$  ▷  $t_{\text{warm}} = 0$  and  $\pi^0$  is uniform if no warm start
     $G \leftarrow \text{SETLEAFVALUES}(\beta_r, \pi^{t_{\text{warm}}}, \theta^v)$ 
     $v(\beta_r) \leftarrow \text{COMPUTE EV}(G, \pi^{t_{\text{warm}}})$ 
     $t_{\text{sample}} \sim \text{linear}\{t_{\text{warm}} + 1, T\}$  ▷ Probability of sampling iteration  $t$  is proportional to  $t$ 
    for  $t = (t_{\text{warm}} + 1) .. T$  do
      if  $t = t_{\text{sample}}$  then
         $\beta'_r \leftarrow \text{SAMPLELEAF}(G, \pi^{t-1})$  ▷ Sample one or multiple leaf PBSs
         $\pi^t \leftarrow \text{UPDATEPOLICY}(G, \pi^{t-1})$ 
         $\bar{\pi} \leftarrow \frac{t}{t+2} \bar{\pi} + \frac{2}{t+2} \pi^t$ 
         $G \leftarrow \text{SETLEAFVALUES}(\beta_r, \pi^t, \theta^v)$ 
         $v(\beta_r) \leftarrow \frac{t}{t+2} v(\beta_r) + \frac{2}{t+2} \text{COMPUTE EV}(G, \pi^t)$ 
      Add  $\{\beta_r, v(\beta_r)\}$  to  $D^v$  ▷ Add to value net training data
      for  $\beta \in G$  do ▷ Loop over the PBS at every public state in  $G$ 
        Add  $\{\beta, \bar{\pi}(\beta)\}$  to  $D^\pi$  ▷ Add to policy net training data (optional)
       $\beta_r \leftarrow \beta'_r$ 

function SETLEAFVALUES( $\beta, \pi, \theta^v$ )
  if IS LEAF( $\beta$ ) then
    for  $s_i \in \beta$  do ▷ For each infostate  $s_i$  corresponding to  $\beta$ 
       $v(s_i) = \hat{v}(s_i | \beta, \theta^v)$ 
  else
    for  $a \in \mathcal{A}(\beta)$  do
      SETLEAFVALUES( $\mathcal{T}(\beta, \pi, a), \pi, \theta^v$ )

function SAMPLELEAF( $G, \pi$ )
   $i^* \sim \text{unif}\{1, N\}, h \sim \beta_r$  ▷ Sample a history randomly from the root PBS and a random player
  while !IS LEAF( $h$ ) do
     $c \sim \text{unif}[0, 1]$ 
    for  $i = 1 .. N$  do
      if  $i == i^*$  and  $c < \epsilon$  then ▷ we set  $\epsilon = 0.25$  during training,  $\epsilon = 0$  at test time
        sample an action  $a_i$  uniform random
      else
        sample an action  $a_i$  according to  $\pi_i(s_i(h))$ 
     $h \sim \tau(h, a)$ 
  return  $\beta_h$  ▷ Return the PBS corresponding to leaf node  $h$ 

```

Once players commit their blinds, they receive two private cards from a standard 52-card deck. The first of four rounds of betting then occurs. On each round of betting, players take turns deciding whether to fold, call, or raise. If a player folds, the other player receives the money in the pot and the hand immediately ends. If a player calls, that player matches the opponent’s number of chips in the pot. If a player raises, that player adds more chips to the pot than the opponent. The initial raise of the round must be at least \$100, and every subsequent raise on the round must be at least as large as the previous raise. A player cannot raise more than either player’s stack size. A round ends when both players have acted in the round and the most recent player to act has called. Player 1 acts first on the first round. On every subsequent round, player 2 acts first.

Upon completion of the first round of betting, three *community* cards are publicly revealed. Upon completion of the second betting round, another community card is revealed, and upon completion of the third betting round a final fifth community card is revealed. After the fourth betting round, if no player has folded, then the player with the best five-card poker hand, formed from the player’s two private cards and the five community cards, is the winner and takes the money in the pot. In case of a tie, the money is split.

C.2 Turn endgame hold’em (TEH)

TEH is identical to HUNL except both players automatically call for the first two betting rounds, and there is an initial \$1,000 per player in the pot at the start of the third betting round. We randomize the stack sizes during training to be between \$5,000 and \$50,000 per player. The action space of TEH is reduced to at most three raise sizes ($0.5 \times \text{pot}$, $1 \times \text{pot}$, or all-in for the first raise in a round, and $0.75 \times \text{pot}$ or all-in for subsequent raises), but the raise sizes for non-all-in raises are randomly perturbed by up to $\pm 0.1 \times \text{pot}$ each game during training. Although we train on randomized stack sizes, bet sizes, and board cards, we measure exploitability on the case of both players having \$20,000, unperturbed bet sizes, and the first four board cards being $3\heartsuit 7\heartsuit T\spadesuit K\clubsuit$. In this way we can train on a massive game while still measuring NashConv tractably. Even without the randomized stack and bet sizes, TEH has roughly $2 \cdot 10^{11}$ infostates.

C.3 Liar’s Dice

Liar’s Dice is a two-player zero-sum game in our experiments, though in general it can be played with more than two players. At the beginning of a game each player privately rolls d dice with f faces each. After that a betting stage starts where players take turns trying to predict how many dice of a specific kind there are among all the players, e.g., 4 dice with face 5. A player’s bid must either be for more dice than the previous player’s bid, or the same number of dice but a higher face. The round ends when a player challenges the previous bid (a call of *liar*). If all players together have at least as many dice of the specified face as was predicted by the last bid, then the player who made the bid wins. Otherwise the player who challenged the bid wins. We use the highest face as a *wild* face, i.e., dice with this face count towards a bid for any face.

D Domain Knowledge Leveraged in our Poker AI Agent

The most prominent form of domain knowledge in our ReBeL poker agent is the simplification of the action space during self play so that there are at most 8 actions at each decision point. The bet sizes are hand-chosen based on conventional poker wisdom and are fixed fractions of the pot, though each bet size is perturbed by $\pm 0.1 \times \text{pot}$ during training to ensure diversity in the training data.

We specifically chose not to leverage domain knowledge that has been widely used in previous poker AI agents:

- All prior top poker agents, including DeepStack [40], Libratus [12], and Pluribus [14], have used *information abstraction* to bucket similar infostates together based on domain-specific features [30, 21, 6]. Even when computing an exact policy, such as during search or when solving a poker game in its entirety [24, 4], past agents have used *lossless abstraction* in which strategically identical infostates are bucketed together. For example, a flush of spades may be strategically identical to a flush of hearts.

Our agent does not use any information abstraction, whether lossy or lossless. The agent computes a unique policy for each infostate. The agent’s input to its value and policy

network is a probability distribution over pairs of cards for each player, as well as all public board cards, the amount of money in the pot relative to the stacks of the players, and a flag for whether a bet has occurred on this betting round yet.

- DeepStack trained its value network on random PBSs. In addition to reducing the dimensionality of its value network input by using information abstraction, DeepStack also sampled PBSs according to a handcrafted algorithm that would sample more realistic PBSs compared to sampling uniform random. We show in Section 8 that training on PBSs sampled uniformly randomly without information abstraction results in extremely poor performance in a value network.

Our agent collects training data purely from self play without any additional heuristics guiding which PBSs are sampled, other than an exploration hyperparameter that was set to $\epsilon = 0.25$ in all experiments.

- In cases where both players bet all their chips before all board cards are revealed, past poker AIs compute the exact expected value of all possible remaining board card outcomes. This is expensive to do in real time on earlier rounds, so past agents pre-compute this expected value and look it up during training and testing. Using the exact expected value reduces variance and makes learning easier.

Our agent does not use this shortcut. Instead, the agent learns these “all-in” expected values on its own. When both agents have bet all their chips, the game proceeds as normal except neither player is allowed to bet.

- The search space in DeepStack [40] extends to the start of the next betting round, except for the third betting round (out of four) where it instead extends to the end of the game. Searching to the end of the game on the third betting round was made tractable by using information abstraction on the fourth betting round (see above). Similarly, Libratus [11], Modicum [15], and Pluribus [14] all search to the end of the game when on the third betting round. Searching to the end of the game has the major benefit of not requiring the value network to learn values for the end of the third betting round. Thus, instead of the game being three “levels” deep, it is only two levels deep. This reduces the potential for propagation of errors.

Our agent always solves to the end of the current betting round, regardless of which round it is on.

- The depth-limited subgames in DeepStack extended to the start of the next betting round on the second betting round. On the first betting round, it extended to the end of the first betting round for most of training and to the start of the next betting round for the last several CFR iterations. Searching to the start of the next betting round was only tractable due to the abstractions mentioned previously and due to careful optimizations, such as implementing CFR on a GPU.

Our agent always solves to the end of the current betting round regardless of which round it is on. We implement CFR only on a single-thread CPU and avoid any abstractions. Since a subgame starts at the beginning of a betting round and ends at the start of the next betting round, our agent must learn six “layers” of values (end of first round, start of second round, end of second round, start of third round, end of third round, start of fourth round) compared to three for DeepStack (end of first round, start of second round, start of third round).

- DeepStack used a separate value network for each of the three “layers” of values (end of first round, start of second round, start of third round). Our agent uses a single value network for all situations.

E Hyper parameters

In this section we provide details of the value and policy networks and the training procedures.

We approximate the value and policy functions using artificial neural networks. The input to the value network consists of three components for both games: agent index, representation of the public state, and a probability distribution over infostates for both agents. For poker, the public state representation consists of the board cards and the common pot size divided by stack size; for Liar’s Dice it is the last bid and the acting agent. The output of the network is a vector of values for each possible infostate of the indexed agent, e.g., each possible poker hand she can hold.

We trained a policy network only for poker. The policy network state representation additionally contains pot size fractions for both agents separately as well as a flag for whether there have been any bets so far in the round. The output is a probability distribution over the legal actions for each infostate.

As explained in section 7 we use Multilayer perceptron with GeLU [27] activation functions and LayerNorm [3] for both value and policy networks.

For poker we represent the public state as a concatenation of a vector of indices of the board cards, current pot size relative to the stack sizes, and binary flag for the acting player. The size of the full input is

$$1(\text{agent index}) + 1(\text{acting agent}) + 1(\text{pot}) + 5(\text{board}) + 2 \times 1326(\text{infostate beliefs})$$

We use card embedding for the board cards similar to [7] and then apply MLP. Both the value and the policy networks contain 6 hidden layers with 1536 layers each. For all experiments we set the probability to explore a random action to $\epsilon = 25\%$ (see Section 5.2). To store the training data we use a simple circular buffer of size 12M and sample uniformly. Since our action abstraction contains at most 9 legal actions, the size of the target vector for the policy network is 9 times bigger than one used for the value network. In order to make it manageable, we apply linear quantization to the policy values. As initial data is produced with a random value network, we remove half of the data from the replay buffer after 20 epochs.

For the full game we train the network with Adam optimizer with learning rate 3×10^{-4} and halved the learning rate every 800 epochs. One epoch is 2,560,000 examples and the batch size 1024. We used 90 DGX-1 machines, each with $8 \times 32\text{GB}$ Nvidia V100 GPUs for data generation. We report results after 1,750 epochs. For TEH experiments we use higher initial learning rate 4×10^{-4} , but halve it every 100 epochs. We report results after 300 epochs.

For Liar’s Dice we represent the state as a concatenation of a one hot vector for the last bid and binary flag for the acting player. The size of the full input is

$$1(\text{agent index}) + 1(\text{acting agent}) + n_{\text{dice}}n_{\text{faces}}(\text{last bid}) + 2n_{\text{faces}}^{n_{\text{dice}}}(\text{infostate beliefs}).$$

The value network contains 2 hidden layers with 256 layers each. We train the network with Adam optimizer with learning rate 3×10^{-4} and halved the learning rate every 400 epochs. One epoch is 25,600 examples and the batch size 512. During both training and evaluation we run the search algorithm for 1024 iterations. We use single GPU for training and 60 CPU threads for data generation. We trained the network for 1000 epochs. To reduce the variance in RL+Search results, we evaluated the three last checkpoints and reported averages in table 2.

E.1 Human Experiments for HUNL

We evaluated our HUNL agent against Dong Kim, a top human professional specializing in HUNL. Kim was one of four humans that played against Libratus [12] in the man-machine competition which Libratus won. Kim lost the least to Libratus. However, due to high variance, it is impossible to statistically compare the performance of the individual humans that participated in the competition.

A total of 7,500 hands were played between Kim and the bot. Kim was able to play from home at his own pace on any schedule he wanted. He was also able to play up to four games simultaneously against the bot. To incentivize strong play, Kim was offered a base compensation of $\$1 \pm \$0.05x$ for each hand played, where x signifies his average win/loss rate in terms of big blinds per hundred hands played. Kim was guaranteed a minimum of \$0.75 per hand and could earn no more than \$2 per hand. Since final compensation was based on the variance-reduced score rather than the raw score, Kim was not aware of his precise performance during the experiment.

The bot played at an extremely fast pace. No decision required more than 5 seconds, and the bot on average plays faster than 2 seconds per hand in self play. To speed up play even further, the bot cached subgames it encountered on the preflop. When the same subgame was encountered again, it would simply reuse the solution it had already computed previously.

Kim’s variance-reduced score, which we report in Section 8, was a loss of 165 ± 69 where the $\pm$ indicates one standard error. His raw score was a loss of 358 ± 188 .

F Proof Related to Value Functions (Theorem 1)

We start by proving some preliminary Lemmas. For simplicity, we will sometimes prove results for only one player, but the results hold WLOG for both players.

For some policy profile $\pi = (\pi_1, \pi_2)$, let $v_i^\pi(\beta) : \mathcal{B} \rightarrow \mathbb{R}^{|\mathcal{S}_i|}$ be a function that takes as input a PBS and outputs infostate values for player i .

Lemma 1. *Let $V_1^{\pi_2}(\beta)$ be player 1's value at β assuming that player 2 plays π_2 in a 2p0s game. $V_1^{\pi_2}(\beta)$ is linear in β_1 .*

Proof. This follows directly from the definition of $v_i^\pi(s_i|\beta)$ along with the definition of V_1 ,

$$V_1^{\pi_2}(\beta) = \sum_{s_1 \in \mathcal{S}_1(s_{\text{pub}})} \beta_1(s_1) v_1(s_1|\beta, (BR(\pi_2), \pi_2))$$

□

Lemma 2. *$V_1(\beta) = \min_{\pi_2} V_1^{\pi_2}(\beta)$, and the set of π_2 that attain $V_1(\beta)$ at β_0 are precisely the Nash equilibrium policies at β_0 . This also implies that $V_1(\beta)$ is concave.*

Proof. By definition, the Nash equilibrium at β is the minimum among all choices of π_2 of the value to player 1 of her best response to π_2 . Any π_2 that achieves this Nash equilibrium value when playing a best response is a Nash equilibrium policy.

From Lemma 1, we know that each $V_1^{\pi_2}(\beta)$ is linear, which implies that $V_1(\beta)$ is concave since any function that is the minimum of linear functions is concave. □

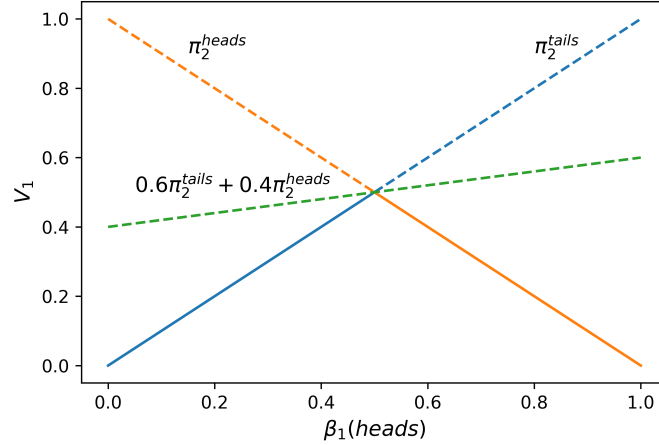


Figure 3: Illustration of Lemma 2. In this simple example, the subgame begins with some probability $\beta(\text{heads})$ of a coin being heads-up, which player 1 observes. Player 2 then guesses if the coin is heads or tails, and wins if he guesses correctly. The payoffs for Player 2's pure strategies are shown as the lines marked π_2^{heads} and π_2^{tails} . The payoffs for a mixed strategy is a linear combination of the pure strategies. The value for player 1 is the minimum among all the lines corresponding to player 2 strategies, denoted by the solid lines.

Now we can turn to proving the Theorem.

Consider a function $\tilde{V}_1$ that is an extension of V_1 to unnormalized probability distributions over $\mathcal{S}_1$ and $\mathcal{S}_2$; i.e. $\tilde{V}_i((s_{\text{pub}}, b_1, b_2)) = V_i((s_{\text{pub}}, b_1/|b_1|_1, b_2/|b_2|_1))$. $\tilde{V}_i = V_i$ on the simplex of valid beliefs, but we extend it in this way to $\mathbb{R}_{\geq 0}^{|\mathcal{S}_i|} \setminus \vec{0}$ so that we can consider gradients w.r.t. $p(s_1)$.

We will use the term ‘supergradient’ to be the equivalent of the subgradient for concave functions. Formally, g is a supergradient of concave function F at x_0 iff for any x in the domain of F ,

$$F(x) - F(x_0) \leq g \cdot (x - x_0).$$

Also, $\text{superg}(F) = -\text{subg}(-F)$.

Theorem (Restatement of Theorem 1). *For any belief $\beta = (\beta_1, \beta_2)$ (for the beliefs over player 1 and 2 infostates respectively) and any policy π^* that is a Nash equilibrium of the subgame rooted at β ,*

$$v_1^{\pi^*}(s_1|\beta) = V_1(\beta) + \bar{g} \cdot \hat{s}_1 \quad (3)$$

for some supergradient $\bar{g}$ of $\tilde{V}_1(\beta)$ with respect to β_1 , where $\hat{s}_1$ is the unit vector in direction s_1 .

Proof. Lemma 2 shows that $V_1(\beta)$ is a concave function in β_1 , and its extension $\tilde{V}$ off the simplex is constant perpendicular to the simplex, so $\tilde{V}$ is concave as well. Therefore the notion of a supergradient is well-defined.

Now, consider some policy profile $\pi^* = (\pi_1^*, \pi_2^*)$ that is a Nash equilibrium of $G(\beta_0)$. $V_1^{\pi^*}$ is a linear function and tangent to V_1 at β_0 ; therefore, its gradient is a supergradient of V_1 at β_0 . Its gradient is

$$\nabla_{\beta_1} V_1^{\pi^*}(\beta) = \nabla_{\beta_1} \sum_{s_1 \in S_1(s_{\text{pub}})} \beta_1(s_1) v_1^{\pi^*}(s_1|\beta) \quad (4)$$

$$= \sum_{s_1 \in S_1(s_{\text{pub}})} \hat{s}_1 v_1^{\pi^*}(s_1|\beta) + \beta(s_1) \nabla_{\beta_1} v_1^{\pi^*}(s_1|\beta) \quad (5)$$

$$= \sum_{s_1 \in S_1(s_{\text{pub}})} \hat{s}_1 v_1^{\pi^*}(s_1|\beta) \quad (6)$$

Equation 6 follows from the fact that for a fixed opponent policy π_2^* , each player 1 infostate s_1 is an independent MDP whose value $v_1^{\pi^*}(s_1|\beta)$ doesn't depend on the probabilities β_1 of being in the different infostates (although it might depend on β_2 since player 1 doesn't observe these).

Note also that the gradient of $V_1^{\pi^*}$ is correct even when some infostates s_1 have probability 0, due to the fact that we defined $v_1^{\pi^*}(s_1|\beta)$ as the value of player 1 playing a *best response* to π_2 at each infostate s_1 (rather than just playing the equilibrium policy π_1^* , which may play arbitrarily at unvisited infostates).

Finally, let's compute $g \cdot \hat{s}_1$ at some β_1 on the simplex (i.e. $|\beta_1|_1 = 1$).

$$g = \nabla_{\beta_1/|\beta_1|_1} V_1^{\pi^*}(s_{\text{pub}}, \beta_1/|\beta_1|_1, \beta_2) \cdot \frac{d}{d\beta_1} \left(\frac{\beta_1}{|\beta_1|_1} \right) \quad (\text{chain rule}) \quad (7)$$

$$= \left(\sum_{s'_1 \in S_1(s_{\text{pub}})} \hat{s}'_1 v_1^{\pi^*}(s'_1|\beta) \right) \cdot (|\beta_1|_1 - \beta_1)/(|\beta_1|_1)^2 \quad (\text{Eq. 4}) \quad (8)$$

$$= \left(\sum_{s'_1 \in S_1(s_{\text{pub}})} \hat{s}'_1 v_1^{\pi^*}(s'_1|\beta) \right) \cdot (1 - \beta_1) \quad (\text{since } |\beta_1|_1 = 1) \quad (9)$$

$$= \sum_{s'_1 \in S_1(s_{\text{pub}})} \hat{s}'_1 v_1^{\pi^*}(s'_1|\beta) - \sum_{s'_1 \in S_1(s_{\text{pub}})} \beta_1(s'_1) v_1^{\pi^*}(s'_1|\beta) \quad (10)$$

$$= \sum_{s'_1 \in S_1(s_{\text{pub}})} \hat{s}'_1 v_1^{\pi^*}(s'_1|\beta) - V_1(\beta) \quad (11)$$

$$g \cdot \hat{s}_1 = v_1^{\pi^*}(s_1|\beta) - V_1(\beta) \quad (12)$$

And we're done. □

G Proofs Related to Subgame Solving (Theorems 2 and 3)

Lemma 3. *Running Algorithm 1 for $N \rightarrow \infty$ times in a depth-limited subgame rooted at PBS β_r will compute an infostate value vector $v_i^{\pi^*}(\beta_r)$ corresponding to the values of the infostates when π^**

is played in the (not depth-limited) subgame rooted at β_r , where π^* is a $\frac{C}{\sqrt{T}}$ -Nash equilibrium for some constant C .

Proof. A key part of our proof is the insight that Algorithm 1 resembles the CFR-D algorithm from [16] if Algorithm 1 were modified such that there was no random sampling and every call to the value network was replaced with a recursive call to the CFR-D algorithm.

Suppose the subgame rooted at β_r extends to the end of the game and therefore there are no calls to the neural network. Then CFR is proven to compute a $\frac{C}{\sqrt{T}}$ -Nash equilibrium [61], which we will call π^* , and Algorithm 1 will indeed learn a value vector $v_i^{\pi^*}(\beta_r)$ for β_r corresponding to the infostate values of β_r when π^* is played in the subgame. Thus, the base case for the inductive proof holds.

Now suppose we have a depth-limited subgame rooted at β_r . Assume that for every leaf PBS $\beta_z^{\pi^t}$ for iterations $t \leq T$, where π^t is the policy in the subgame on iteration t , that we have already computed an infostate value vector $v_i^{\pi^*}(\beta_z^{\pi^t})$ corresponding to the values of the infostates in $\beta_z^{\pi^t}$ when π^* is played in the subgame rooted at $\beta_z^{\pi^t}$, where π^* is a $\frac{C}{\sqrt{T}}$ -Nash equilibrium for some constant C . Assume also that Lemma 3 holds for all leaf PBSs $\beta_z^{\pi^{T+1}}$.

First, for leaf PBSs that neither player reaches with positive probability, the values for the leaf PBS have no affect on CFR or the values computed for the root infostates because CFR weights the values by the probability the player reaches the PBS [61].

Now consider a leaf PBS $\beta_z^{\pi^{T+1}}$ that some player reaches with positive probability. Since $v_i^{\pi^*}(\beta_z^{\pi^t})$ has already been computed for all $\beta_z^{\pi^t}$ and since we are running a deterministic algorithm, so $v_i^{\pi^*}(\beta_z^{\pi^t})$ will not change with subsequent calls of Algorithm 1 on $\beta_z^{\pi^t}$ for all $t \leq T$. Thus, π^t will be the same for all $t \leq T$. Since Algorithm 1 samples a random CFR iteration, and since the leaf PBS $\beta_z^{\pi^{T+1}}$ is sampled with positive probability for some player when iteration $T+1$ is sampled, so the algorithm will sample $\beta_z^{\pi^{T+1}}$ N' times, where $N' \rightarrow \infty$ as $N \rightarrow \infty$. Since Lemma 3 holds for $\beta_z^{\pi^{T+1}}$, so eventually $v_i^{\pi^*}(\beta_z^{\pi^{T+1}})$ will be computed for $\beta_z^{\pi^{T+1}}$. Therefore, due to CFR-D [16], Lemma 3 will hold for β_r and the inductive step is proven. $\square$

Theorem (Restatement of Theorem 2). *Consider an idealized value approximator that returns the most recent sample of the value for sampled PBSs, and 0 otherwise. Running Algorithm 1 with T iterations of CFR in each subgame will produce a value approximator that produces values that correspond to a $\frac{C}{\sqrt{T}}$ -equilibrium policy for any PBS that could be encountered during play, where C is a game-dependent constant.*

Proof. Since we run Algorithm 1 for $N \rightarrow \infty$ times at the root of the game, so by Lemma 3, Theorem 2 is true. $\square$

Theorem (Restatement of Theorem 3). *If Algorithm 1 is run at test time with no off-policy exploration, a value network that has error at most δ for any leaf PBS, and with T iterations of CFR being used to solve subgames, then the algorithm plays a $(\delta C_1 + \frac{\delta C_2}{\sqrt{T}})$ -Nash equilibrium, where C_1, C_2 are game-specific constants.*

Proof. We prove the theorem inductively. Consider first a subgame near the end of the game that is not depth-limited. I.e., it has no leaf nodes. Clearly, the policy π^* that Algorithm 1 using CFR plays in expectation is a $\frac{k_1}{\sqrt{T}}$ -Nash equilibrium for game-specific constant k_1 in this subgame.

Rather than play the average policy over all T iterations $\bar{\pi}^T$, one can equivalently pick a random iteration $t \sim \text{uniform}\{1, T\}$ and play according to π^t , the policy on iteration t . This algorithm is also a $\frac{k_1}{\sqrt{T}}$ -Nash equilibrium in expectation.

Next, consider a depth-limited subgame G such that for any leaf PBS β^t on any CFR iteration t , the policy that Algorithm 1 plays in the subgame rooted at β^t is in expectation a δ -Nash equilibrium in the subgame. If one computes a policy for G using tabular CFR-D [16] (or, as discussed in Section I, using CFR-AVG), then by Theorem 2 in [16], the average policy over all iterations is $k_2\delta + \frac{k_3}{\sqrt{T}}$ -Nash equilibrium.

Just as before, rather than play according to this average policy $\bar{\pi}^T$, one can equivalently pick a random iteration $t \sim \text{uniform}\{1, T\}$ and play according to π^t . Doing so would also result in a $k_2\delta + \frac{k_3}{\sqrt{T}}$ -Nash equilibrium in expectation. This is exactly what Algorithm 1 does.

Since there are a finite number of “levels” in a game, which is a game-specific constant, Algorithm 1 plays according to a $\delta C_1 + \frac{\delta C_2}{\sqrt{T}}$ -Nash equilibrium. □

H Fictitious Linear Optimistic Play

Fictitious Play (FP) [5] is an extremely simple iterative algorithm that is proven to converge to a Nash equilibrium in two-player zero-sum games. However, in practice it does so at an extremely slow rate. On the first iteration, all agents choose a uniform policy π_i^0 and the average policy $\bar{\pi}_i^0$ is set identically. On each subsequent iteration t , agents compute a best response to the other agents’ average policy $\pi_i^t = \arg\max_{\pi_i} v_i(\pi_i, \bar{\pi}_{-i}^{t-1})$ and update their average policies to be $\bar{\pi}_i^t = \frac{t-1}{t} \bar{\pi}_i^{t-1} + \frac{1}{t} \pi_i^t$. As $t \rightarrow \infty$, $\bar{\pi}^t$ converges to a Nash equilibrium in two-player zero-sum games.

It has also been proven that a family of algorithms similar to FP known as **generalized weakened fictitious play (GWFP)** also converge to a Nash equilibrium so long as they satisfy certain properties [60, 38], mostly notably that in the limit the policies on each iteration converge to best responses.

In this section we introduce a novel variant of FP we call **Fictitious Linear Optimistic Play (FLOP)** which is a form of GWFP. FLOP is inspired by related variants in CFR, in particular Linear CFR [13]. FLOP converges to a Nash equilibrium much faster than FP while still being an extremely simple algorithm. However, variants of CFR such as Linear CFR and Discounted CFR [13] still converge much faster in most large-scale games.

In FLOP, the initial policy π_i^0 is uniform. On each subsequent iteration t , agents compute a best response to an *optimistic* [18, 47, 58] form of the opponent’s average policy in which π_{-i}^{t-1} is given extra weight: $\pi_i^t = \arg\max_{\pi_i} v_i(\pi_i, \frac{t}{t+2} \bar{\pi}_{-i}^{t-1} + \frac{2}{t+2} \pi_{-i}^{t-1})$. The average policy is updated to be $\bar{\pi}_i^t = \frac{t-1}{t+1} \bar{\pi}_i^{t-1} + \frac{2}{t+1} \pi_i^t$. Theorem 4 proves that FLOP is a form of GWFP and therefore converges to a Nash equilibrium as $t \rightarrow \infty$.

Theorem 4. *FLOP is a form of Generalized Weakened Fictitious Play.*

Proof. Assume that the range of payoffs in the game is M . Since $\pi_i^t = \arg\max_{\pi_i} v_i(\pi_i, \frac{t}{t+2} \bar{\pi}_{-i}^{t-1} + \frac{2}{t+2} \pi_{-i}^{t-1})$, so π_i^t is an ϵ_t -best response to $\bar{\pi}_{-i}^{t-1}$ where $\epsilon_t < M \frac{2}{t+2}$ and $\epsilon_t \rightarrow 0$ as $t \rightarrow \infty$. Thus, FLOP is a form of GWFP with $\alpha_t = \frac{2}{t}$. □

I CFR-AVG: CFR Decomposition using Average Strategy

On each iteration t of CFR-D, the value of every leaf node z is set to $\hat{v}(s_i(z)|\beta_z^{\pi^t})$. Other than changing the values of leaf nodes every iteration, CFR-D is otherwise identical to CFR. If T iterations of CFR-D are conducted with a value network that has error at most δ for each infostate value, then $\bar{\pi}^T$ has exploitability of at most $k_1\delta + k_2/\sqrt{T}$ where k_1 and k_2 are game-specific constants [40].

Since it is the *average* policy profile $\bar{\pi}^t$, not π^t , that converges to a Nash equilibrium as $t \rightarrow \infty$, and since the leaf PBSs are set based on π^t , the input to the value network $\hat{v}$ may span the entire domain of inputs even as $t \rightarrow \infty$. For example, suppose in a Nash equilibrium π^* the probability distribution at $\beta_z^{\pi^*}$ was uniform. Then the probability distribution at $\beta_z^{\pi^t}$ for any individual iteration t could be *anything*, because regardless of what the probability distribution is, the average over all iterations could still be uniform in the end. Thus, $\hat{v}$ may need to be accurate over the entire domain of inputs rather than just the subspace near $\beta_z^{\pi^*}$.

In **CFR-AVG**, leaf values are instead set according to the *average policy* $\bar{\pi}^t$ on iteration t . When a leaf PBS is sampled, the leaf node is sampled with probability determined by π^t , but the PBS itself is defined using $\bar{\pi}^t$.

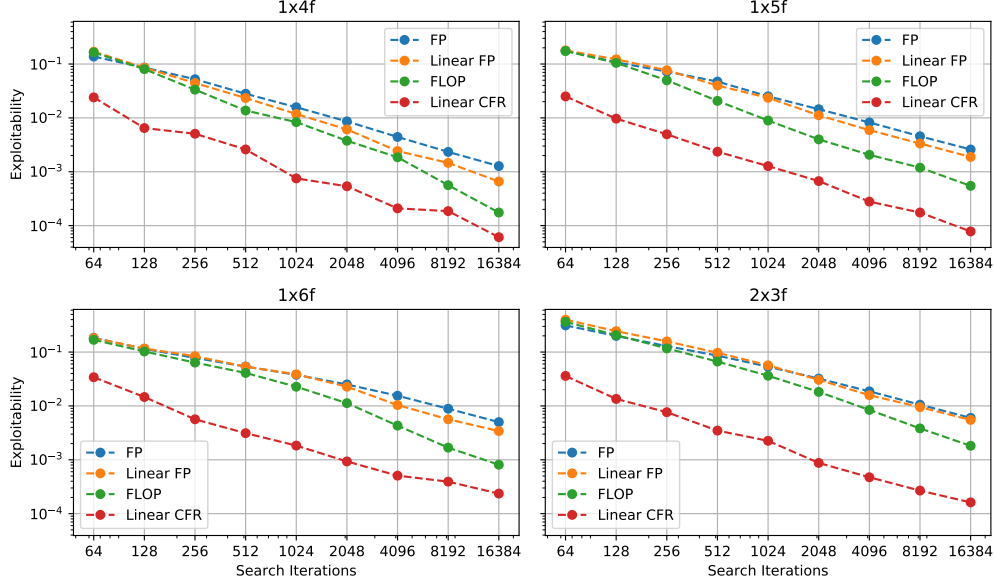


Figure 4: Exploitability of different algorithms of 4 variants of Liar’s Dice: 1 die with 4, 5, or 6 faces and 2 dice with 3 faces. For all games FLOP outperforms Linear FP, but does not match the convergence of Linear CFR.

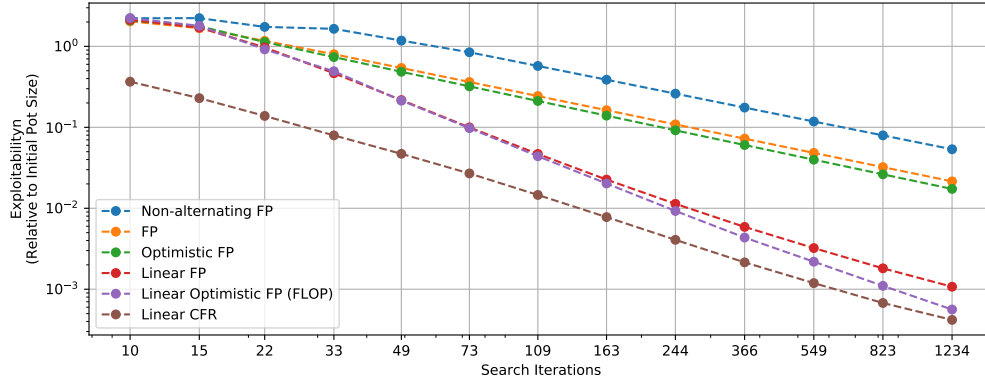


Figure 5: Exploitability of different algorithms for Turn Endgame Hold’em.

We first describe the tabular form of CFR-D [16]. Consider a game G' and a depth-limited subgame G , where both G' and G share a root but G extends only a limited number of actions into the future. Suppose that T iterations of a modified form of CFR are conducted in G' . On each iteration $t \leq T$, the policy $\pi(s_i)$ is set according to CFR for each $s_i \in G$. However, for every infostate $s'_i \in G' \setminus G$, the policy is set differently than what CFR would call for. At each leaf public state s'_{pub} of G , we solve a subgame rooted at $\beta_{s'_{\text{pub}}}^{\pi^t}$ by running T' iterations of CFR. For each s'_i in the subgame rooted at $\beta_{s'_{\text{pub}}}^{\pi^t}$, we set $\pi^t(s'_i) = \bar{\pi}^T(s'_i)$ (where $\pi^t(s'_i)$ is the policy for the infostate in G and $\bar{\pi}^T(s'_i)$ is the policy for the infostate in the subgame rooted at $\beta_{s'_{\text{pub}}}^{\pi^t}$). It is proven that as $T' \rightarrow \infty$, CFR-D converges to a $O(\frac{1}{\sqrt{T}})$ -Nash equilibrium [16].

CFR-AVG is identical to CFR-D, except the subgames that are solved on each iteration t are rooted at $\beta_{s'_{\text{pub}}}^{\bar{\pi}^t}$ rather than $\beta_{s'_{\text{pub}}}^{\pi^t}$. Theorem 5 proves that CFR-AVG achieves the same bound on convergence to a Nash equilibrium as CFR-D.

Theorem 5. *Suppose that T iterations of CFR-AVG are run in a depth-limited subgame, where on each iteration $t \leq T$ the subgame rooted at each leaf PBS $\beta_{s'_{\text{pub}}}^{\bar{\pi}^t}$ is solved completely. Then $\bar{\pi}^T$ is a $\frac{C}{\sqrt{T}}$ -Nash equilibrium for a game-specific constant C .*

CFR-AVG has a number of potential benefits over CFR-D:

- Since $\bar{\pi}^t$ converges to a Nash equilibrium as $t \rightarrow \infty$, CFR-AVG allows $\hat{v}$ to focus on being accurate over a more narrow subspace of inputs.
- When combined with a policy network (as introduced in Section 5.3), CFR-AVG may allow $\hat{v}$ to focus on an even more narrow subspace of inputs.
- Since $\bar{\pi}^{t+1}$ is much closer to $\bar{\pi}^t$ than π^{t+1} is to π^t , in practice as t becomes large one can avoid querying the value network on every iteration and instead recycle the values from a previous iteration. This may be particularly valuable for Monte Carlo versions of CFR.

While CFR-AVG is theoretically sound, we modify its implementation in our experiments to make it more efficient in a way that has not been proven to be theoretically sound. The reason for this is that while the *input* to the value network is $\beta_{s'_{\text{pub}}}^{\bar{\pi}^t}$ (i.e., the leaf PBS corresponding to $\bar{\pi}^t$ being played in G), the *output* needs to be the value of each infostate s_i given that π^t is played in G . Thus, unlike CFR-D and FP, in CFR-AVG there is a mismatch between the input policy and the output policy.

One way to cope with this is to have the input consist of both $\beta_{s'_{\text{pub}}}^{\bar{\pi}^t}$ and $\beta_{s'_{\text{pub}}}^{\pi^t}$. However, we found this performed relatively poorly in preliminary experiments when trained through self play.

Instead, on iteration $t - 1$ we store the output from $\hat{v}(s_i | \beta_{s'_{\text{pub}}}^{\bar{\pi}^{t-1}})$ for each s_i and on iteration t we set $v^t(s_i)$ to be $t\hat{v}(s_i | \beta_{s'_{\text{pub}}}^{\bar{\pi}^t}) - (t - 1)\hat{v}(s_i | \beta_{s'_{\text{pub}}}^{\bar{\pi}^{t-1}})$ (in vanilla CFR). The motivation for this is that $\pi^t = t\bar{\pi}^t - (t - 1)\bar{\pi}^{t-1}$. If $v^t(h) = v^{t-1}(h)$ for each history h in the leaf PBS, then this modification of CFR-AVG is sound. Since $v^t(h) = v^{t-1}(h)$ when h is a full-game terminal node (i.e., it has no actions), this modified form of CFR-AVG is identical to CFR in a non-depth-limited game. However, that is not the case in a depth-limited subgame, and it remains an open question whether this modified form of CFR-AVG is theoretically sound in depth-limited subgames. Empirically, however, we found that it converges to a Nash equilibrium in turn endgame hold'em for every set of parameters (e.g., bet sizes, stack sizes, and initial beliefs) that we tested.

Figure 6 shows the performance of CFR-D, CFR-AVG, our modified form of CFR-AVG, and FP in TEH when using an oracle function for the value network. It also shows the performance of CFR-D, our modified form of CFR-AVG, and FP in TEH when using a value network trained through self-play. Surprisingly, the theoretically sound form of CFR-AVG does worse than CFR-D when using an oracle function. However, the modified form of CFR-AVG does better than CFR-D when using an oracle function and also when trained through self play.

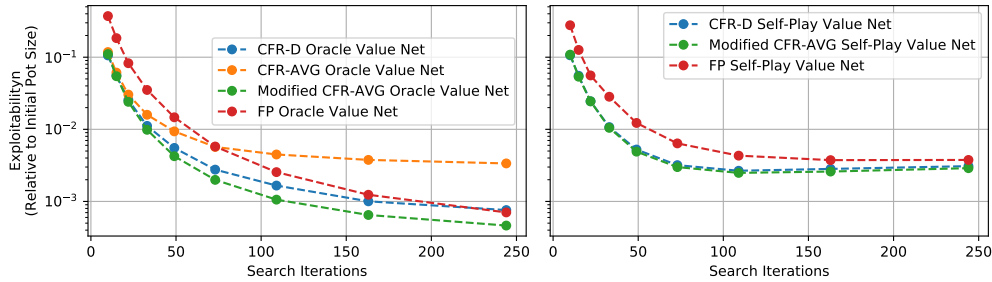


Figure 6: **Left:** comparison of CFR-D, CFR-AVG, modified CFR-AVG, and FP using an oracle value network which returns exact values for leaf PBSs. **Right:** comparison of CFR-D, modified CFR-AVG, and FP using a value network learned through 300 epochs of self play.

We also trained a model on HUNL with training parameters that were identical to the one reported in Section 8, but using CFR-D rather than CFR-AVG. That model lost to BabyTartanian8 by 10 ± 3 whereas the CFR-AVG model won by 9 ± 4 . The CFR-D model also beat Slumbot by 39 ± 6 whereas the CFR-AVG model won by 45 ± 5 .

I.1 Proof of Theorem 5

Our proof closely follows from [16] and [40].

Proof. Let $R^t(s_i)$ be the (cumulative) regret of infostates s_i on iteration t . We show that the regrets of all infostates in G' are bounded by $O(\sqrt{T})$ and therefore the regret of the entire game is bounded by $O(\sqrt{T})$.

First, consider the infostates in G . Since their policies are chosen according to CFR each iteration, their regrets are bounded by $O(\sqrt{T})$ regardless of the policies played in descendant infostates.

Next consider an infostate $s_i \in G' \setminus G$. We prove inductively that $R^t(s_i) \leq 0$. Let β^{π^t} be the PBS at the root of the subgame containing s_i in CFR-D, and $\beta^{\bar{\pi}^t}$ be the PBS at the root of the subgame containing s_i in CFR-AVG. On the first iteration, $\beta^{\pi^t} = \beta^{\bar{\pi}^t}$. Since we assume CFR-AVG computes an exact equilibrium in the subgame rooted at $\beta^{\bar{\pi}^t} = \beta^{\pi^t}$, so $R^t(s_i) = 0$ on the first iteration.

Next, we prove $R^{t+1}(s_i) \leq R^t(s_i)$. We define $a_i^{*,t}$ as

$$a_i^{*,t} = \operatorname{argmax}_{a_i} \sum_{t'=0}^t v^{t'}(s_i, a_i) \quad (13)$$

By definition of regret,

$$R^{t+1}(s_i) = \sum_{t'=0}^{t+1} (v^{t'}(s_i, a_i^{*,t+1}) - v^{t'}(s_i)) \quad (14)$$

Separating iteration $t + 1$ from the summation we get

$$R^{t+1}(s_i) = \sum_{t'=0}^t (v^{t'}(s_i, a_i^{*,t+1}) - v^{t'}(s_i)) + (v^{t+1}(s_i, a_i^{*,t+1}) - v^{t+1}(s_i)) \quad (15)$$

By definition of $a_i^{*,t}$ we know $\sum_{t'=0}^t v^{t'}(s_i, a_i^{*,t+1}) \leq \sum_{t'=0}^t v^{t'}(s_i, a_i^{*,t})$, so

$$R^{t+1}(s_i) \leq \sum_{t'=0}^t (v^{t'}(s_i, a_i^{*,t}) - v^{t'}(s_i)) + (v^{t+1}(s_i, a_i^{*,t+1}) - v^{t+1}(s_i)) \quad (16)$$

Since $\sum_{t'=0}^t (v^{t'}(s_i, a_i^{*,t}) - v^{t'}(s_i))$ is the definition of $R^t(s_i)$ we get

$$R^{t+1}(s_i) \leq R^t(s_i) + (v^{t+1}(s_i, a_i^{*,t+1}) - v^{t+1}(s_i)) \quad (17)$$

Since $\pi^{t+1} = \pi^{*,t+1}$ in the subgame where $\pi^{*,t+1}$ is an exact equilibrium of the subgame rooted at $\beta^{\bar{\pi}^{t+1}}$, so π^{t+1} is a best response to $\bar{\pi}^{t+1}$ in the subgame and therefore $v^{t+1}(s_i, a_i^{*,t+1}) = v^{t+1}(s_i)$. Thus,

$$R^{t+1}(s_i) \leq R^t(s_i) \quad (18)$$

□

J CFR Warm Start Algorithm Used

Our warm start technique for CFR is based on [10], which requires only a policy profile to warm start CFR soundly. That technique computes a “soft” best response to the policy profile, which results in instantaneous regrets for each infostate. Those instantaneous regrets are scaled up to be equivalent to some number of CFR iterations. However, that technique requires careful parameter tuning to achieve good performance in practice.

We instead use a simplified warm start technique in which an exact best response to the policy profile is computed. That best response results in instantaneous regrets at each infostate. Those regrets are scaled up by a factor of 15 to imitate 15 CFR iterations. Similarly, the average policy effectively assumes that the warm start policy was played for the first 15 iterations of CFR. CFR then proceeds as if 15 iterations have already occurred.

PerfectDou: Dominating DouDizhu with Perfect Information Distillation

Guan Yang^{1*} Minghuan Liu^{2*} Weijun Hong¹
 Weinan Zhang² Fei Fang³ Guangjun Zeng¹ Yue Lin^{1†}
¹ NetEase Games AI Lab, ² Shanghai Jiao Tong University, ³ Carnegie Mellon University
 {yangguan, gzzengguangjun, gzlinyue}@corp.netease.com,
 {minghuanliu, wnzhang}@sjtu.edu.cn, feif@cs.cmu.edu

Abstract

As a challenging multi-player card game, DouDizhu has recently drawn much attention for analyzing competition and collaboration in imperfect-information games. In this paper, we propose PerfectDou, a state-of-the-art DouDizhu AI system that dominates the game, in an actor-critic framework with a proposed technique named perfect information distillation. In detail, we adopt a perfect-training-imperfect-execution framework that allows the agents to utilize the global information to guide the training of the policies as if it is a perfect information game and the trained policies can be used to play the imperfect information game during the actual gameplay. To this end, we characterize card and game features for DouDizhu to represent the perfect and imperfect information. To train our system, we adopt proximal policy optimization with generalized advantage estimation in a parallel training paradigm. In experiments we show how and why PerfectDou beats all existing AI programs, and achieves state-of-the-art performance.

1 Introduction

With the fast development of Reinforcement Learning (RL), game AI has achieved great success in many types of games, including board games (e.g., Go [22], chess [21]), card games (e.g., Texas Hold'em [3], Mahjong [15]), and video games (e.g., Starcraft [26], Dota [1]). As one of the most popular card games in China, DouDizhu has not been studied in depth until very recently. In perfect-information games such as Go, agent can observe all the events occurred previously including initial hand of each agent and all agents' actions. In contrast, DouDizhu is an imperfect-information game with special structure, and an agent does not know other agents' initial hands but can observe all agents' actions. One challenge in DouDizhu is that it is a three-player game with both competition and collaboration: the two *Peasant* players need to cooperate as a team to compete with the third *Landlord* player. In addition, DouDizhu has a large action space that is hard to be abstracted for search-based methods [31].

Although various methods have been proposed for tackling these challenges [29, 10], they are either computationally expensive or far from optimal, and highly rely on abstractions with human knowledge [31]. Recently, Zha et al. [31] proposed DouZero, which applies simple Deep Monte-Carlo (DMC) method to learn the value function with pre-designed features and reward function. DouZero is regarded as the state-of-the-art (SoTA) AI system of DouDizhu for its superior performance and training efficiency compared with previous works.

*Equal contribution. Yang is responsible for the basic idea, system design and implementation details; Liu mainly contributes to the methodology, writing and experimental design. †Corresponding author. Project page at <https://github.com/Netease-Games-AI-Lab-Guangzhou/PerfectDou/>.

Unfortunately, we find DouZero still has severe limitations in many battle scenarios, which will be characterized in Section D.3. To establish a stronger and more robust DouDizhu bot, in this paper, we present a new AI system named PerfectDou, and show that it leads to significantly better performance than existing AI systems including DouZero. The name of our program follows the key technique we use – perfect information distillation. The proposed technique utilizes a Perfect-Training-Imperfect-Execution (PTIE) framework, a variant of the popular Centralized-Training-Decentralized-Execution (CTDE) paradigm in multi-agent RL literature [5, 11]. Namely, we feed perfect-information to the agent in the training phase to guide the training of the policy, and only imperfect-information can be used when deploying the learned policy for actual game play. Correspondingly, we further design the card and game features to represent the perfect and imperfect information. To train PerfectDou, we utilize Proximal Policy Optimization (PPO) [19] with Generalized Advantage Estimation (GAE) [18] by self-play in a distributed training system.

In experiments, we show PerfectDou beats all the existing DouDizhu AI systems and achieves the SoTA performance in a 10k-decks tournament; moreover, PerfectDou is the most training efficient, such that the number of samples required is an order of magnitude lower than the previous SoTA method; for application usage, PerfectDou can be deployed in online game environment due to its low inference time.

2 Preliminaries

Imperfect-Information Extensive-Form Games. An imperfect-information extensive-form (or tree-form) game can be described as a tuple $G = (\mathcal{P}, \mathcal{H}, \mathcal{Z}, \mathcal{A}, \mathcal{T}, \chi, \rho, r, \mathcal{I})$, where $\mathcal{P}$ denotes a finite set of *players*, $\mathcal{A}$ is a finite set of actions, and $\mathcal{H}$ is a finite set of *nodes* at which players can take actions and are similar to states in an RL problem. At a node $h \in \mathcal{H}$, $\chi : \mathcal{H} \rightarrow 2^{\mathcal{A}}$ is the action function that assigns to each node $h \in \mathcal{H}$ a set of possible actions, and $\rho : \mathcal{H} \rightarrow \mathcal{P}$ represents the unique acting player. An action $a \in \mathcal{A}(h)$ that leads from h to h' is denoted by the successor function $\mathcal{T} : \mathcal{H} \times \mathcal{A} \rightarrow \mathcal{H}$ as $h' = \mathcal{T}(h, a)$. $\mathcal{Z} \subseteq \mathcal{H}$ are the sets of terminal nodes for which no actions are available. For each player $p \in \mathcal{P}$, there is a reward function $r_p \in r = r_1, r_2, \dots, r_{|\mathcal{P}|} : \mathcal{Z} \rightarrow \mathbb{R}$. Furthermore, $\mathcal{I} = \{\mathcal{I}_p | p \in \mathcal{P}\}$ describes the information sets (infosets) in the game where $\mathcal{I}_p$ is a partition of all the nodes with acting player p . If two nodes h, h' belong to the same infoset I of player p , i.e., $h, h' \in I \in \mathcal{I}_p$, these two nodes are indistinguishable to p and will share the same action set. We use $I(h)$ to denote the infoset of node h . Upon a certain infoset, a policy (or a behavior strategy) π_p for player p describes which action the player would take at each infoset. A policy can be stochastic, and we use $\pi_p(I)$ to denote the probability vector over player p 's available actions at infoset I . With a slight abuse of notation, we use $\pi_p(h)$ to denote the stochastic action player p will take at node h . For two nodes h, h' that belong to the same infoset I , it is clear that $\pi_p(h) = \pi_p(h') = \pi_p(I)$. Therefore, the objective for each player p is to maximize its own total expected return at the end of the game: $R_p \triangleq \mathbb{E}_{Z \sim \pi}[r_p(Z)]$, $Z \in \mathcal{Z}$.

The Three-Players DouDizhu Game. DouDizhu (a.k.a. Fight the Landlord) is a three-player card game that is popular in China and is played by hundreds of millions of people. Among the three players, two of them are called the *Peasants*, and they need to cooperate as a team to compete against another player called the *Landlord*. The standard game consists of two phases, bidding and cardplay. The bidding phase designates the roles to the players and deals leftover cards to the *Landlord*. In the cardplay phase, the three players play cards in turn in clock-wise order. Within a game episode, there are several *rounds*, and each begins with one player showing a legal combination of cards (solo, pair, etc.). The subsequent players must either choose to pass or beat the previous hand by playing a more superior combination of cards, usually in the same category. The round continues until two consecutive players choose to pass and the player who played the last hand initiates to the next round. DouDizhu is in the genre of shedding where the player wins by emptying his's hand, or loses vice versa. Therefore, in this game, the suit does not matter but the rank does. The score of a game is calculated as the base score multiplied by a multiplier determined by specialized categories of cards (Appendix B.2 shows the details). In this paper, we only consider the cardplay phase, which can be formulated as an imperfect-information game. More detailed information about the game can be referred to [31].

The key challenges of DouDizhu include how the *Peasants* work as a team to beat the *Landlord* with card number advantage using only imperfect information. For example, one *Peasant* can try

to help his teammate to win by always trying the best to beat the *Landlord*'s cards and play cards in a category where the teammate has an advantage. In addition, the action space of DouDizhu is particularly large, and there are 27,472 possible combinations of cards that can be played in total with hundreds of legal actions in a hand. Furthermore, the action space cannot be easily abstracted since improperly playing a card may break other potential card combinations in the following rounds and lead to losing the game.

3 Methodology

In this section, we first introduce how perfect-training-imperfect-execution works for a general imperfect-information game. Then, we formulate the DouDizhu game as an imperfect-information game to solve.

3.1 Perfect Information Distillation

In card games as DouDizhu, the imperfect-information property comes from the fact that players do not show their hand cards to the others. And therefore the critical challenge for each player is to deal with the indistinguishable nodes from the same info-set. For such games, consider we can construct a strategically identical perfect-information game and allow one player to observe distinguishable nodes, then, the decisions at each node can rely on the global information and he may have more chances to win the game, like owning a cheating plug-in. This motivates us to utilize the distinguishable nodes for training the agents of imperfect-information games, and therefore we propose the technique of perfect information distillation.

In general, the perfect information distillation is a framework trained in perfect-training-imperfect-execution (PTIE) paradigm, a variant of centralized-training-decentralized-execution (CTDE) [5, 11], as illustrated in Fig. 1. Particularly, CTDE constructs the value function with all agents' observations and actions for general multi-agent tasks. By comparison, our proposed PTIE is designed for imperfect-information games where additional perfect information is introduced in the training stage. In this work, we consider to initiate PTIE with actor-critic [25] (but PTIE is not limited to actor-critic), which is a template of policy gradient (PG) methods, proposed in the RL literature towards maximizing the expected reward of the policy function through PG with a value function:

$$\nabla_{\theta_p} J = \mathbb{E}_{\pi} [\nabla_{\theta_p} \log \pi_{\theta_p}(a|s) Q_{\pi}(s, a)] , \quad (1)$$

where s denotes a state in an RL problem, Q is the state-action value function learned by a function approximator, usually called the critic. Notice that the critic is playing the role of evaluating how good an action is taken at a specific situation, but only at the training time. When the agent is deployed into inference, only the policy π can be used to inferring feasible actions. Therefore, for imperfect-information games, we can provide additional information about the exact node the player is in to train the critic with self-play, as long as the actor does not take such information for decision making. Intuitively, we are distilling the perfect information into the imperfect policy.

Formally, for each node h , we construct a distinguishable node $D(h)$ for the strategically identical perfect-information game. Then, we define the value function at $D(h)$, $V_{\pi_p}(D(h)) = \mathbb{E}_{a \sim \pi_p, h^0=h} [Q_{\pi_p}(D(h), a)] = \mathbb{E}_{Z|\pi_p, h^0=h} [r_p(Z)]$ as the expected value of distinguishable nodes. In the sequel, we propose a simple extension of actor-critic policy gradient considering parameterized policy π_{θ_p} for each player p :

$$\nabla_{\theta_p} J = \mathbb{E}_{\pi_p} [\nabla_{\theta_p} \log \pi_{\theta_p}(a|h) Q_{\pi_p}(D(h), a)] . \quad (2)$$

In practice, we use a policy network (actor) to represent the policy π_p for each player, which takes as input a vector describing the representation at an indistinguishable node h that the player can observe

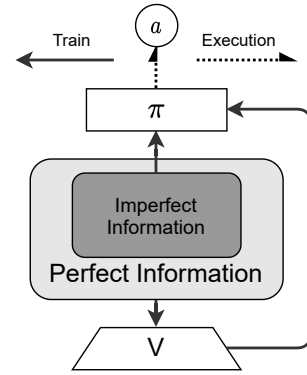


Figure 1: Overview of perfect information distillation within a perfect-training-imperfect-execution framework. The value network takes additional information (such as other players' cards in Poker games) as input, while the policy network does not.

during the game. For estimating the critic, a value network is utilized, which takes representation of the global information at the distinguishable node $D(h)$. In other words, the value network takes additional information as inputs (such as other players' cards in Poker games) or targets (such as immediate rewards computed upon others' hands), while the policy network does not. During the training, the value function updates the values for all distinguishable nodes; then, it trains the policy on every node on the same info set from sampled data, which implicitly gives an expected value estimation on each info set. In practice, the generalization ability of neural networks enables the policy to find a better solution, which is also the advantage for using the proposed PTIE framework.

PTIE is a general way for training imperfect-information game agents using RL. In addition, the optimality point of independent RL in multi-agent learning is exactly one of an Nash equilibria (NE) and it has been proved that training RL algorithms with self-play can converge to an NE in two-player cases (although no convergence guarantees for three-player games) [9, 12], which may provide insights why PTIE works. We expect that with PTIE, players can leverage the perfect information during inference to derive coordination and strategic policies. In experiments, we show that this allows PerfectDou to cooperate with each other (as *Peasants*) or compete against the team (as the *Landlord*).

3.2 DouDizhu as An Imperfect-Information Game

As is mentioned in Section 2, the cardplay phase of DouDizhu can be regarded as an imperfect-information game with three players. At each node h , its info set I_p contains all combinations of the other's invisible handcards. Then at each level of the game tree, the three players take clockwise turns to choose an available action with policies π_p depending on the info set $I_p(h)$ of the current node h with the reward function r_p . The path from the root of the game tree to a node h contains the initial hand of all players and all the historical moves of all players. The reward functions at leaf nodes are set to be the score the players win or lose at the end of the game.

4 PerfectDou System Design

In this section, we explain how we construct our PerfectDou system in detail, with the proposed perfect information distillation technique, and several novel components designed for DouDizhu that help it summit the game. Particularly, PTIE requires different representations as input layer for the policy and the value network by feeding the value function with perfect information (distinguishable nodes) and the policy with imperfect information (indistinguishable nodes).

4.1 Card Representation

In our system, we encode each feasible card combination with a 12×15 matrix, as shown in Fig. 2. Specifically, we first encode different ranks and numbers with a 4×15 matrix, where the columns correspond to the 15 ranks (including jokers) and similar to Zha et al. [31], the number of ones in the four rows of a single column represents the number of cards of that rank in the player's hand. Different from Zha et al. [31], we further propose to encode the legal combination of cards with the player's current hand, to help the agent realize the different property of various kinds of cards (see Appendix B.1). The feature sizes of each part are shown in Appendix C.1.

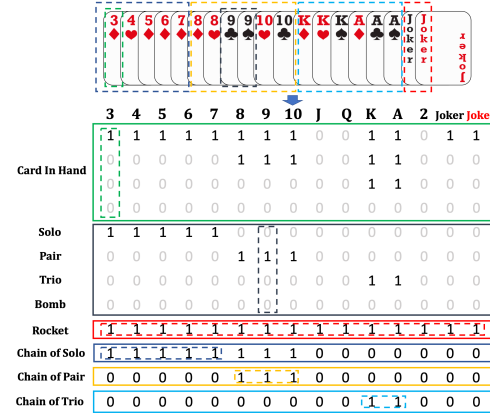


Figure 2: Card representation matrix. Columns stand for 15 different card ranks and rows stand for correspondingly designed features. The first 4 rows are the same as Zha et al. [31], and the last 8 rows are additional design for encoding the legal combination of cards.

Table 1: Feature design of perfect-information (distinguishable nodes) and imperfect-information (indistinguishable nodes) for the game. Perfect features include all imperfect features.

	FEATURE DESIGN	SIZE
IMPERFECT FEATURE	CURRENT PLAYER’S HAND	$1 \times 12 \times 15$
	UNPLAYED CARDS	$1 \times 12 \times 15$
	CURRENT PLAYER’S PLAYED CARDS	$1 \times 12 \times 15$
	PREVIOUS PLAYER’S PLAYED CARDS	$1 \times 12 \times 15$
	NEXT PLAYER’S PLAYED CARDS	$1 \times 12 \times 15$
	3 ADDITIONAL BOTTOM CARDS	$1 \times 12 \times 15$
	LAST 15 MOVES	$15 \times 12 \times 15$
	PREVIOUS PLAYER’S LAST MOVE	$1 \times 12 \times 15$
	NEXT PLAYER’S LAST MOVE	$1 \times 12 \times 15$
	MINIMUM PLAY-OUT STEPS OF HAND CARDS	1
	NUMBER OF CARDS IN CURRENT PLAYER’S HAND	1
	NUMBER OF CARDS IN PREVIOUS PLAYER’S HAND	1
	NUMBER OF CARDS IN NEXT PLAYER’S HAND	1
	NUMBER OF BOMBS	1
	FLAG OF GAME CONTROL BY CURRENT PLAYER	1
ADDITIONAL PERFECT FEATURE	PREVIOUS PLAYER’S HAND CARDS	$1 \times 12 \times 15$
	NEXT PLAYER’S HAND CARDS	$1 \times 12 \times 15$
	MINIMUM PLAY-OUT STEPS OF PREVIOUS PLAYER’S HAND CARDS	1
	MINIMUM PLAY-OUT STEPS OF NEXT PLAYER’S HAND CARDS	1

4.2 Node Representation

In the game of DouDizhu, the distinguishable node $D(h)$ should cover all players’ hand cards at h , along with the game and player status. Therefore, we propose to represent h with imperfect features and $D(h)$ with perfect feature designs, shown in Tab. 1. In detail, the imperfect features include a flatten matrix² of $23 \times 12 \times 15$ and a game state array of 6×1 . On the contrary, the perfect features consist of a flatten card matrix of $25 \times 12 \times 15$ and a game state array of 8×1 . Therefore, they are totally asymmetric, and the imperfect features are a subset of the perfect features.

4.3 Network Structure and Action Representation

The PerfectDou system follows the general actor-critic design, and we take PPO [19] with GAE [18] as the learning algorithm. Slightly different from Eq. (2), PPO estimates the advantage $A_p = R_p - V_{\pi_p}$ as the critic instead of Q_{π_p} . For value network, we use an MLP to handle encoded features (the detailed structure is shown in Appendix C.3). As for the policy network, we first utilize an LSTM to encode all designed features; to encourage the agent to pay attention to specific card types, the proposed network structure will encode all the available actions into feature vectors, as depicted in Tab. 6. The output of the legal action probability is then computed with the action and game features, as illustrated in Fig. 3. Formally, we concatenate the node representation e_s with each action representation e_{a^i} separately, and get the legal action distribution:

$$p(a) = \text{softmax}(f([e_s, e_{a^i}]_{i=1}^N)), \quad (3)$$

where a^i is the i -th action, $[\cdot]$ denotes the concatenation operation for N available actions, and f are layers of MLPs. This resembles the target attention mechanism in Ye et al. [28].

4.4 Perfect Reward Design

If we only care about the result at the end of the game, the reward at leaf nodes is rather sparse; in addition, players can only estimate their advantage of winning the game using imperfect information during the game, which could be inaccurate and fluctuated. Thanks to PTIE, we are allowed to impose an oracle reward function for DouDizhu at each node to enhance the perfect information modeled by the value function. In the training of PerfectDou, instead of estimating the advantage, we utilize an oracle³ for evaluating each player, particularly, the minimum steps needed to play out all cards, which can be treated as a simple estimation of the distance to win. The reward function is

²Short for a matrix flattened to a one-dimensional vector.

³Implemented as a dynamic programming algorithm, see Appendix E for details.

then defined as the advantage difference computed by the relative distance to win of the two camps in two consecutive timesteps. Formally, at timestep t , the reward function is:

$$r_t = \begin{cases} -1.0 \times (\text{Adv}_t - \text{Adv}_{t-1}) \times l, & \text{Landlord} \\ 0.5 \times (\text{Adv}_t - \text{Adv}_{t-1}) \times l, & \text{Peasant} \end{cases} \quad (4)$$

$$\text{Adv}_t = N_t^{\text{Landlord}} - \min(N_t^{\text{Peasant1}}, N_t^{\text{Peasant2}}), \quad (5)$$

where l is a scaling factor, and N_t is the minimum steps to play out all cards at timestep t .

For instance, in a round, at timestamp t , the distance of the *Landlord* to win is 5 and the distances of two *Peasants* are 3 and 7, which means *Peasants* have a larger advantage since the relative distance is 2 for *Peasants* and -2 for the *Landlord*. However, if the *Landlord* plays a good hand such that both *Peasants* can not suppress, the *Landlord* will in result get a positive reward due to the decreased relative distance of the *Landlord*, i.e., from 2 to 1. Correspondingly, the *Peasants* would get a negative reward as their relative distances are getting larger. Such a reward function can encourage the cooperation between *Peasants*, since the winning distance is defined by the minimum steps of both players. In our implementation, the computation of the rewards is carried out after a round of the game, hence to promote training efficiency.

4.5 Distributed Training Details

To further expedite the training procedure, we design a distributed training system represented in Fig. 4. Specifically, the system contains a set of rollout workers for collecting the self-play experience data and sending it to a pool of GPUs; these GPUs asynchronously receive the data and store it into their local buffers. Then, each GPU learner randomly samples mini-batches from its own buffer and compute the gradient separately, which is then synchronously averaged across all GPUs and back propagated to update the neural networks. After each round of updating, new parameters are sent to every rollout worker. And each worker will load the latest model after 24 (8 for each player) steps sampling. Such a decoupled training-sampling structure will allow PerfectDou to be extended to large scale experiments. Our design of the distributed system borrows a lot from IMPALA [4], which also keeps a set of rollout workers to receive the updated model, interact with the environment and send back rollout trajectories to learners. The main difference is derived from the learning algorithm where we use PPO with GAE instead of actor-critic with V-trace [4]. Moreover, we keep three different models for *Landlord* and two *Peasants* separately which are only updated by their own data against the latest opponent models.

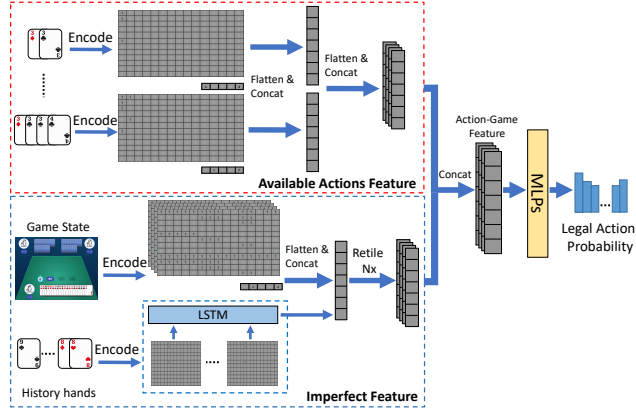


Figure 3: The policy network structure of PerfectDou system. The network predicts the action distribution given the current imperfect information of the game, including state information and available actions feature.

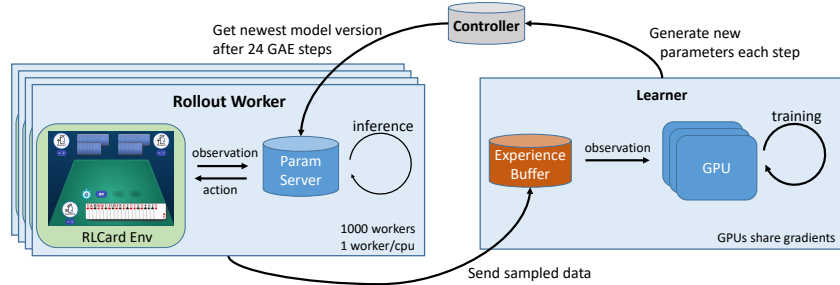


Figure 4: Illustration of the distributed training system.

Table 2: DouDizhu tournaments for existing AI programs by playing 10k randomly generated decks. Player A outperforms B if WP is larger than 0.5 or ADP is larger than 0 (highlighted in **boldface**). The algorithms are ranked according to the number of the other algorithms that they beat. We note that DouZero is the current SoTA DouDizhu bot and the gray rows highlight the comparison. Numerical results except marked * are directly borrowed from Zha et al. [31].

Rank	A \ B	PerfectDou		DouZero		DeltaDou		RHCP-v2		CQN		Random	
		WP	ADP	WP	ADP	WP	ADP	WP	ADP	WP	ADP	WP	ADP
1	PerfectDou (Ours)	-	-	0.543*	0.143*	0.584*	0.420*	0.543*	0.506*	0.862*	2.090*	0.994*	3.146*
2	DouZero (Paper)	-	-	-	-	0.586	0.258	0.764	1.671	0.810	1.685	0.989	3.036
-	DouZero (Public)	0.457*	-0.143*	-	-	0.585*	0.253*	0.451*	0.060*	0.828*	1.950*	0.986*	3.050*
3	DeltaDou	0.416*	-0.420*	0.414	-0.258	-	-	0.691*	1.528*	0.784	1.534	0.992	3.099
4	RHCP-v2	0.457*	-0.506*	0.549*	-0.060*	0.309*	-1.423*	-	-	0.770*	1.414*	0.990*	2.670*
5	CQN	0.138*	-2.090*	0.190	-1.685	0.216	-1.534	0.230*	-1.414*	-	-	0.889	1.912
6	Random	0.006*	-3.146*	0.011	-3.036	0.008	-3.099	0.010*	-2.670*	0.111	-1.912	-	-

5 Related Work

Imperfect-Information Games. Many popular card games are imperfect-information games and have attracted much attention. For instance, Li et al. [15] worked on the four-player game Mahjong and proposed a distributed RL algorithm combined with techniques like global reward prediction, oracle guiding, and run-time policy adaptation to win against most top human players; in addition, Lerer et al. [13] adopted search-based and imitation methods to learn the playing policy for Hanabi. Iterative algorithms such as Counterfactual Regret Minimization (CFR) and its variants [35, 17, 3] are also well-used for a fully competitive game, Hold'em Poker, whose action space is generally designed at the scale of tens (fold, call, check and kinds of bet) and the legal actions at each decision point are even less [17, 33]. In addition, CFR does not proven to converged to an Nash equilibrium in games with more than two players [14]. Consequently, they can be hardly designed for DouDizhu due to the large action space with difficult abstraction and the mixed game property (both cooperative and competitive, although there are few success cases for such a setting [16]).

DouDizhu AI systems. Besides the recent SoTA work DouZero [31], many researchers have made efforts on utilizing the power of RL into solving DouDizhu. However, simply applying RL algorithms such as DQN and A3C into the game can hardly make benefits [29]. Therefore, You et al. [29] proposed Combinational Q-Network (CQN) that reduces the action space by heuristics action decoupling; moreover, DeltaDou [10] utilized Monte-Carlo Tree Search (MCTS) for DouDizhu, along with Bayesian inference for the hidden information and a pre-trained kicker network for action abstraction. DeltaDou was also reported as reaching human-level performance. Zhang et al. [32] also relied on MCTS with predicting other players' actions. Recently, Zhao et al. [34] similarly proposed to model opponents' actions and train the policy model based on the DouZero architecture, reaching limited improvements. Nevertheless, as shown in this paper, we can instead distill such perfect-information knowledge like opponents' hand cards to the policy in a perfect-training-imperfect-execution style and reach a better performance.

6 Experiments

We conduct comprehensive experiments to investigate the following research questions. **RQ1:** How good is PerfectDou against SoTA DouDizhu AI? **RQ2:** What are the key ingredients of PerfectDou? **RQ3:** How is the inference efficiency of PerfectDou? To answer **RQ1**, we empirically evaluate the performance against existing DouDizhu programs. Regarding **RQ2**, we conduct ablation studies on key components in our design. And for **RQ3**, we calculate the average inference time for all algorithms involved. Finally, we conduct in-depth analysis, provide interesting case studies of PerfectDou. In the appendix, we report more results including a battle against skilled human players.

6.1 Experimental Setups

Baselines. We evaluate PerfectDou against the following algorithms under the open-source RL-Card Environment [30]: 1) **DouZero** [31]: A recent SoTA baseline method that had beaten every existing DouDizhu AI system using Deep Monte-Carlo algorithm. 2) **DeltaDou** [10]: An MCTS-based algorithm with Bayesian inference. It achieved comparable performance as human experts. 3) **Combinational Q-Network (CQN)** [29]: Based on card decomposition and Deep Q-Learning. 4)

Rule-Based Algorithms: Including the open-source heuristic-based program **RHCP-v2** [10, 31], the rule model in RLCARD and a **Random** program with uniform legal moves. For evaluation, we directly take their public (or provided) codes and pre-trained models.

Metrics. The performance of DouDizhu are mainly quantified following the same metrics in previous researches [10, 31]. Specifically, given two algorithms A against B, we calculate: 1) **WP** (Winning Percentage): The proportion of winning by A in a number of games. 2) **ADP** (Average Difference in Points): The per-game averaged difference of scores between A and B. In other words, positive ADP means gaining scores while the negative represents losing it. This is a more reasonable metric for evaluating DouDizhu AI systems, because in real games players are evaluated by the scores obtained instead of their winning rates, as further discussed in Appendix B.2.

In our experiments, we choose ADP as the basic reward for all experiments training PerfectDou, which is augmented with the proposed reward signal in Section 4.4 during the training stage.

6.2 Comparative Evaluations

We conduct a tournament to demonstrate the advantage of our PerfectDou, where each pair of the algorithms play 10,000 decks, shown in Tab. 2 (**RQ1**). Since the bidding performance in each algorithm varies and poor bidding would affect game results significantly, for fair comparison, we omit the bidding phase and focus on the phase of cardplay. In detail, all games are randomly generated and each game would be played two times, i.e., each competing algorithm is assigned as *Landlord* or *Peasant* once. We use WP and ADP as the basic reward respectively for comparing over these two metrics for all evaluating methods.

Overall, PerfectDou dominates the leaderboard by beating all the existing AI programs, no matter rule-based or learning based algorithms, with significant advantage on both WP and ADP. Specifically, as noted that DouDizhu has a large variance where the initial hand cards can seriously determine the advantage of the game; even though, PerfectDou still consistently outperforms the current SoTA baseline – DouZero. However, we find that PerfectDou is worse than the result published in DouZero paper [31] when competing against RHCP. To verify this problem, we test the public model of DouZero (denoted as DouZero (Public) with grey color). To our surprise, its performance can match most of the reported results in their paper except the one against RHCP, where it only takes a WP of 0.452 lower than 0.5, indicating that the public model of DouZero can not beat RHCP as suggested in the original paper, and in fact PerfectDou is the better one.

It is also observed that some competition outcome has a high WP and a negative ADP. A potential reason can be explained as such agents are reckless to play out the bigger cards without considering the left hand, leading to winning many games of low score, but losing high score in the other games when their hand cards are not good enough. From our statistics of online human matches, the WP of winner is usually in a range of $0.52 \sim 0.55$ when the player tries to maximize its ADP.

We further reveal the sample efficiency of PerfectDou by comparing the competing performance w.r.t. different training steps. As shown in Tab. 3, we compare two versions of PerfectDou (1e9 and 2.5e9 steps) against two versions of DouZero (roughly 1e9 and 1e10 steps). From the tournament in Tab. 2, we know the final version of PerfectDou (2.5e9 steps) outperforms the final version of DouZero ($\sim 1e10$ steps). However, to our surprise, an early stage of PerfectDou is able to beat DouZero. With the same 1e9 training samples, PerfectDou wins DouZero with a large gap (WP of 0.732 and ADP of 1.270), which is even better than the 1e10 sample-trained DouZero. This indicates that PerfectDou is not only the best performance but also the most training efficient. The related training curves are shown in Appendix D.5.

Table 3: Training efficiency comparison over 100k decks.

A \ B	DouZero ($\sim 1e9$)		DouZero ($\sim 1e10$)	
	WP	ADP	WP	ADP
PerfectDou (2.5e9)	-	-	0.541	0.130
PerfectDou (1e9)	0.732	1.270	0.524	0.014
DouZero ($\sim 1e10$)	0.698	1.150	-	-

6.3 Ablation Studies

We want to further investigate the key to the success of our AI system (**RQ2**). Specifically, we would like to analyse how our design of the feature and the training framework help PerfectDou dominate the tournament of DouDizhu. To this end, we evalu-

ate different variants of PerfectDou and the previous SoTA AI system – DouZero, including: a) *ImperfectDouZero*⁴: DouZero with our proposed imperfect-information features. b) *ImperfectDou*: PerfectDou with only imperfect-information features as inputs for the value function. c) *RewardlessDou*: PerfectDou without node reward. d) *Vanilla PPO*: Naive actor-critic training with imperfect-information features only and without additional reward.

The ablation experiments are designed as competitions among ImperfectDou, RewardlessDou and PerfectDou against DouZero for comparing the effectiveness of perfect information distillation and perfect intermediate reward separately; while the

Table 4: Ablation studies over 100k decks.

A	B	DouZero ($\sim 1e9$)		DouZero ($\sim 1e10$)		ImperfectDouZero ($\sim 1e9$)	
		WP	ADP	WP	ADP	WP	ADP
PerfectDou (1e9)		0.732	1.270	0.524	0.014	0.731	1.350
ImperfectDou (1e9)		0.717	1.180	0.486	-0.057	0.723	1.320
RewardlessDou (1e9)		0.738	0.490	0.540	-0.201	0.659	0.587
Vanilla PPO(1e9)		0.509	-0.307	0.346	-0.709	0.433	-0.023

battle of ImperfectDouZero and DouZero against PerfectDou are designed for excluding the benefit from feature engineering. Results for all comparisons are shown in Tab. 4. Even with the imperfect features only, ImperfectDou can still easily beat DouZero with the same training steps; however, DouZero turns the tide with much more training data. Furthermore, our proposed node features seem not appropriate for DouZero to achieve a better results compared with its original design. Additionally, without the node reward, PerfectDou still beats DouZero with higher WP (in spite of sacrificing a lot of ADP), indicating the effectiveness of perfect reward in training, without which it would risk losing points to win one game. Finally, without both node reward and perfect feature design for the value function, vanilla PPO simply can not perform well. Therefore, we can conclude that our actor-critic based algorithm along with the PTIE training provides a high sample efficiency under our feature design, and the node reward benefits the rationality of our AI.

6.4 Runtime Analysis

We further conduct runtime analysis to show the efficiency of PerfectDou w.r.t. the inference time (RQ3), which is reported in Fig. 5. All evaluations are conducted on a single core of Intel(R) Xeon(R) Gold 6130 CPU @ 2.10GHz. The inference time of each AI could be attributed to its pipeline and implementation in the playing time. CQN uses a large Q network (nearly $10\times$ parameters larger than ours) with a complex card decomposer to derive reasonable hands. As a result, the inference time of CQN is the longest. Besides, both DeltaDou and RHCP-V2 contain lots of times of Monte Carlo simulations, thus slowing down the inference time. As comparisons, DouZero and PerfectDou only require one network forward inference time with a similar number of parameters. For RLCard, only handcraft rules are computed. Therefore, we can notice that PerfectDou is significantly faster than previous programs like DeltaDou, CQN and RHCP, yet is slightly slower than DouZero. To be more accurate, the average inference time of DouZero is 2 milliseconds compared with 6 milliseconds of PerfectDou. And the reason why PerfectDou is a bit slower than DouZero may due to the more complex feature processing procedure. Note that this can be further optimized in practice such as changing the implementation from Python to C++, which is common for AI deployment. In addition, the model inference time of DouZero we test is 1.9 ms while PerfectDou is 4 ms, the difference is slight. The above analysis suggests that PerfectDou is applicable and affordable to real-world applications such as advanced game AI.

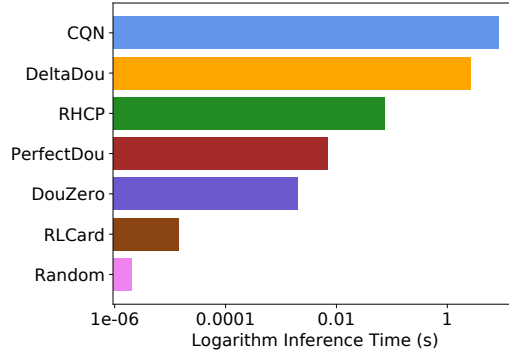


Figure 5: Comparison of the inference time.

6.5 In-depth Analysis and Case Studies

In our experiments, we find that DouZero is leaky and unreasonable in many battle scenarios, while PerfectDou performs better therein. To quantitatively evaluate whether PerfectDou is stronger and more reasonable, we conduct an in-depth analysis by collecting the statistics among the games, and

⁴Note that DouZero cannot acquire any perfect-information feature since it will play in a cheating style if so.

additionally illustrate some of the observations for qualitatively comparing the behavior of DouZero and PerfectDou. Detailed results can be further referred to Appendix D.2 and Appendix D.3, and here we list the key conclusions below:

The statistic analysis claims the rationality of PerfectDou: (i) when playing as the *Landlord*, PerfectDou plays fewer bombs to avoid losing scores and tends to control the game even when the *Peasants* play more bombs; (ii) when playing as the *Peasant*, two PerfectDou agents cooperate better with more bombs to reduce the control time of the *Landlord* and its chance to play bombs; (iii) when playing as the *Peasant*, the right-side *Peasant* agent (play after the *Landlord*) of PerfectDou throws more bombs to suppress the Landlord than DouZero, which is more like human strategy. From behavior observations, we also find: 1) DouZero is more aggressive but less thinking. 2) PerfectDou is better at guessing and suppressing. 3) PerfectDou is better at card combination. 4) PerfectDou is more calm. Beyond these, we also include battle results against skilled human players, additional training results and complete tournament results in the appendix.

7 Conclusion

In this paper, we propose PerfectDou, a SoTA DouDizhu AI system that dominates the game. PerfectDou takes the advantage of the perfect-training-imperfection-execution (PTIE) training paradigm, and is trained within a distributed training framework. In experiments we extensively investigate how and why PerfectDou can achieve the SoTA performance by beating all existing AI programs with reasonable strategic actions. In fact, the PTIE paradigm is actually a variant of centralized-training-decentralized-execution (CTDE), applied for imperfect-information games in particular. Intuitively, PTIE is a general way for training imperfect-information game AI, with which we expect the value function can distill the perfect information to the policy which can only receive imperfect information. Although in this paper we only discuss its success on one of the hard poker games, DouDizhu, we believe it has the power to further improve the ability for other imperfect-information games, remaining a space of imagination to be explored by more future works.

Acknowledgement

The SJTU team is partially supported by “New Generation of AI 2030” Major Project (2018AAA0100900) and National Natural Science Foundation of China (62076161). Minghuan Liu is also supported by Wu Wen Jun Honorary Doctoral Scholarship, AI Institute, SJTU. We thank Ming Zhou for helpful discussions on game theory, and anonymous reviewers for constructive suggestions.

Appendix

A Additional Related Work

Train with global information, test with local one. Utilizing global information to reduce the complexity of imperfect-information games has also been investigated in some works. For example, AlphaStar [26], a grand-master level AI system for StarCraft II. In their implementation, the value network of the agent can observe the full information about the game state, including those that are hidden from the policy. They argue that such a training style improves training performance. In our work, we formulate the idea as Perfect-Training-Imperfect-Execution (PTIE) or perfect information distillation technique for imperfect-information games, and show the effectiveness on complicated card games like DouDizhu. Moreover, in Suphx [15], a strong Mahjong AI system, they used a similar method namely oracle guiding. Particularly, in the beginning of the training stage, all global information is utilized; then, as the training goes, the additional information would be dropped out slowly to none, and only the information that the agent is allowed to observe is reserved in the subsequent training stage. However, there are obvious difference between Suphx and PerfectDou. In Suphx, the perfect information is used by the actor and thus has to be dropped before the inference stage; on the contrary, PerfectDou feeds the critic with additional observations and distill the global information to the actor. Beyond games, Fang et al. [6] worked on trading for order execution and proposed a different technique using global information other than PTIE, which trained a student policy with imperfect (real) market information and policy distillation from a teacher policy trained with perfect (oracle) market information.

Relation to sample-based CFR. CFR aims to minimize the total regret of policy by minimizing the cumulative counterfactual regret in each infoset. The definition of regret highly relates to the definition of advantage used in RL community, which has been shown in lots of previous works [23, 7]. Vanilla CFR [35] and many variants [27, 2] apply model-based approach to calculated all the weights of the game tree to update and obtain a good strategy (policy). However, when the game has long episodes and is hard for searching across the game tree, it is necessary to compute through trajectory samples, called sampled-based CFR methods [24, 8]. This resembles the learning procedure of RL algorithms. Recently, Fu et al. [7] proposed a new form of sample-based CFR algorithm, and shown that PPO is exactly a practical implementation of it (but not PTIE), revealing close connections between CFR and RL.

B More About DouDizhu

B.1 Term of Categories

In the work of Zha et al. [31], they had shown a comprehensive introduction of DouDizhu game, so we think it may be wordy to repeat the stereotyped rules. However, for better understanding the cases shown in this paper, we introduce the typical term of categories in DouDizhu that are commonly used as follows. Note that all cards can suppress the cards in the same category with a higher rank, yet bomb can suppress any categories except the bomb with a higher rank. Rocket is the highest-rank bomb. **Kicker** refer to the unrelated or useless cards that players can deal out when playing some kind of categories of **main cards** (see below), which can be either a solo or a pair.

1. **Solo** : Any single card.
2. **Pair** : Two matching cards of equal rank.
3. **Trio** : Three individual cards of equal rank.
4. **Trio with Solo** : Three individual cards of equal rank with a Solo as the kicker.
5. **Trio with Pair** : Three individual cards of equal rank with a Pair as the kicker.
6. **Chain of Solo** : Five or more consecutive individual cards.
7. **Chain of Pair** : Three or more consecutive Pairs.
8. **Chain of Trio (Plane)** : Two or more consecutive Trios.
9. **Plane with Solo**: Two or more consecutive Trios with each has a distinct individual kicker card and Plane as the main cards.

10. **Quad with Solo** : Four-of-a-kind with two Solos as the kicker and Four-of-a-kind as the main cards.
11. **Quad with Pair** : Four-of-a-kind with a set of Pair as the kicker and Quad with Pair as the main cards.
12. **Quad with Pairs** : Four-of-a-kind with two sets of Pair as the kicker and Quad with Pairs as the main cards.
13. **Bomb** : Four-of-a-kind.
14. **Rocket** : Red and black jokers.

B.2 Scoring Rules

In Zha et al. [31], they pay more attention to the win/lose result of the game but care less about the score. However, in real competitions, players must play for numbers of games and are ranked by the scores they win. And that is why we think ADP is a better metric for evaluating DouDizhu AI systems because a bad AI player can win a game with few scores but lose with much more scores.

Specifically, in each game, the *Landlord* and the *Peasants* have base scores of 2 and 1 respectively. When there is a bomb shown in a game, the score of each player doubles. For example, a *Peasant* player first shows a bomb of 4 and then the *Landlord* player suppresses it with a rocket, then the base score of each *Peasant* becomes 4 and the *Landlord* becomes 8. A player will win all his scores after winning the game, or loses all of them vice versa.

C Additional System Design Details

C.1 Card Representation Details

In the system of PerfectDou, we augment the basic card in hand matrix with explicitly encoded card types as additional features, in order to allow the agent realizing the different properties of different kind of cards. The size details are shown in Tab. 5.

Table 5: Card representation design.

CARD MATRIX FEATURE	SIZE
CARD IN HAND	4×15
SOLO	1×15
PAIR	1×15
TRIO	1×15
BOMB	1×15
ROCKET	1×15
CHAIN OF SOLO	1×15
CHAIN OF PAIR	1×15
CHAIN OF TRIO	1×15

C.2 Action Feature Details

Table 6: Action feature design.

FEATURE DESIGN	SIZE
CARD MATRIX OF ACTION	12×15
IF THIS ACTION IS BOMB	1
IF THIS ACTION IS THE LARGEST ONE	1
IF THIS ACTION EQUALS THE NUMBER OF LEFT PLAYER'S CARDS IN HAND	1
IF THIS ACTION EQUALS THE NUMBER OF RIGHT PLAYER'S CARDS IN HAND	1
THE MINIMUM STEPS TO PLAY-OUT ALL LEFT CARDS AFTER THIS ACTION PLAYED	1
IF THIS ACTION IS VALID	1
ACTION ID	1

The action features are a flatten matrix from 12×15 action card matrix plus 1×7 extra dimensions describing the property of the cards as shown in Tab. 6. Since the number of actions in each game state varies, which can lead to different lengths of action features, a fixed length matrix is flattened to store all action features where the non-available ones are marked as zero.

C.3 Value Network Structure

The value network of PerfectDou is designed to evaluate the current situation of players, and we expect that the value function can utilize the global information, in other words, know the exact node the player is in. Therefore, we should feed additional information that the policy is not allowed to see in our design. Specifically, as shown in Fig. 6, the imperfect feature for indistinguishable nodes is encoded using the shared network as in the policy network; besides, we also encode the perfect feature of distinguishable nodes that the policy cannot observe during its game playing. The encoded vector are then concatenated to a simple MLP to get the scalar value output.

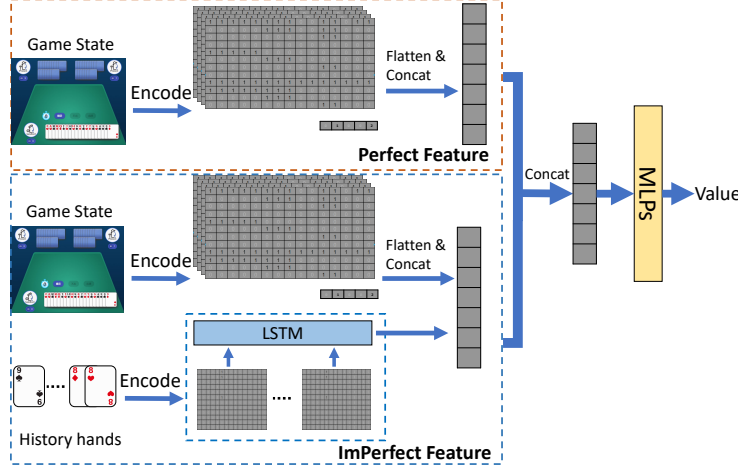


Figure 6: The value network structure of the proposed PerfectDou system. The network predicts values using both the imperfect feature and the perfect feature and distill the knowledge into the policy in the training.

D Experiments

D.1 Setups, Hyperparameters and Training Details

In our implementation, a small distributed training cluster is built using 880 CPUs cores and 8 GPUs. Horovod [20] is used to synchronize gradients between GPUs, the total batch size is 1024, 128 for each GPU. The most important hyperparameters in our experiment are shown in Tab. 7. Specifically, in our design, we simplify the discrete action space from 27472 (include all possible combinations) into an abstract action space of 621 for learning the actor, followed by an decoding strategy to get the final action (see Appendix E.2 for more details).

During self-play training, we find a better practical solution for DouDizhu is to keep three different models for *Landlord* and two *Peasants* separately which is only updated by their own data against the latest opponent model. In the main training stage, the total reward function will be a basic reward (in this paper we use ADP all the time) augmented with the designed oracle reward as shown in Section 4.4, which is found to be extremely useful for accelerating convergence. In the final stage, the oracle reward is removed and only the ADP reward is used to fine-tune the model for reaching a better performance measured by the ADP metric.

D.2 In-Depth Statistical Analysis

In our experiments, we find that DouZero is leaky and unreasonable in many battle scenarios, while PerfectDou performs better therein. To quantitatively evaluate whether PerfectDou is stronger and more reasonable, we conduct an in-depth analysis and collect the statistics among the games between DouZero and PerfectDou. Particularly, we organize games between PerfectDou and Douzero to play in different roles for 100,000 decks in each setting. Since the roles are assigned randomly instead of opting by agents themselves in our experiments, and the *Landlord* has a higher base score with three extra cards, we observe that playing as a *Landlord* is always harder to win and leads to negative ADPs. From the statistics shown in Tab. 8, we learn many lessons about the rationality of PerfectDou: (i) when playing as the *Landlord*, PerfectDou plays fewer bombs to avoid losing

Table 7: Hyperparameters. * refers to the maximum version gap allowed between the models used for sampling and training.

Learning rate	3e-4
Optimizer	Adam
Discount factor γ	1.0
λ of GAE	0.95
Step of GAE	24 (8 for each player)
Batch size	1024
Entropy weight of PPO	0.1
Length of LSTM	15 (5 for each player)
Max model lag*	1
Intermediate reward scale	50
Policy MLP hidden sizes	[256, 256, 256, 512]
Policy MLP output size (action space size)	621
Value MLP hidden sizes	[256, 256, 256, 256]
Value MLP output size	1

Table 8: Average per game statistics of important behaviors over 100k decks: *Game Len* is the average number of rounds in a game; *% Bomb* represents the average percentage of bombs (a type of card can suppress any categories except the bomb with a higher rank, see Appendix B) played in the game; *Left* and *Right* are the relative position to the *Landlord*; and *Landlord Control Time* measures the number of rounds that the landlord plays an action suppressing all other players.

<i>Landlord Agent</i>	WP	ADP	<i>Game Len</i>	<i>% Bomb of Left Peasant</i>	<i>% Bomb of Landlord</i>	<i>% Bomb of Right Peasant</i>	<i>Landlord Control Time</i>	<i>Peasant Agent</i>
PerfectDou (2.5e9)	0.446	-0.407	33.347	68.05	28.46	74.90	12.993	DouZero (~1e10)
DouZero (~1e10)	0.421	-0.461	33.911	66.24	28.73	75.29	9.005	
PerfectDou (2.5e9)	0.387	-0.608	31.157	66.13	26.67	79.68	10.518	PerfectDou (2.5e9)
DouZero (~1e10)	0.360	-0.686	31.267	64.80	26.72	79.29	7.123	

scores and tends to control the game even the *Peasants* play more bombs; (ii) when playing as the *Peasant*, two PerfectDou agents cooperate better with more bombs to reduce the control time of the *Landlord* and its chance to play bombs; (iii) when playing as the *Peasant*, the right-side *Peasant* agent (play after the *Landlord*) of PerfectDou throws more bombs to suppress the *Landlord* than DouZero, which is more like human strategy.

D.3 Case Study: Behavior of DouZero vs PerfectDou

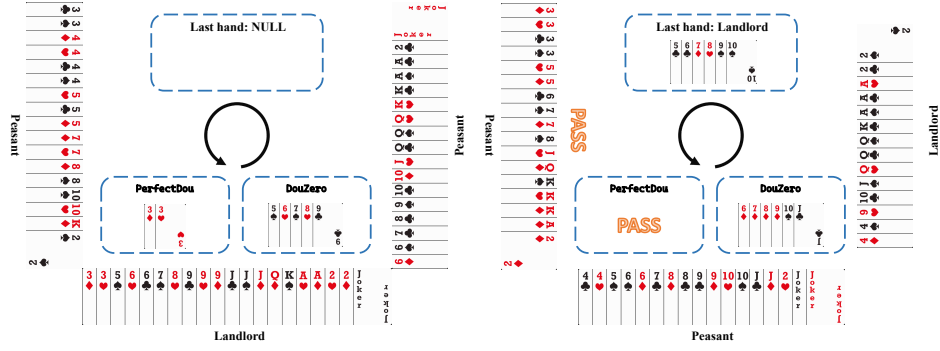
In this section, we list some of the observations during the games for comparing the behavior of DouZero and PerfectDou to qualitatively support our analysis.

DouZero is more aggressive but less thinking. The first observation is that DouZero is extremely aggressive without considering the left hands. For instance, as shown in Fig. 7(a), in the beginning DouZero chooses a chain of solo but leaves the pair of 3, which can be dangerous since the pair of 3 is the one of the minimum cards and cannot suppress any card; Fig. 7(b) illustrates another strong case, where DouZero also chooses a chain of solo to suppress the opponent without considering the consequence of leaving a hand of solos. On the contrary, PerfectDou is more conservative and steady. We believe the proposed perfect information distillation mechanism helps PerfectDou to infer global information in a more reasonable way.

PerfectDou is better at guessing and suppressing. We observe another fact that the usage of perfect information distillation within the PTIE framework benefits PerfectDou a lot by suppressing the opponents in advance. In Fig. 7(c) shows a case when the teammate puts a pair of T^5 , DouZero chooses to pass; on the contrary, PerfectDou chooses suppressing by a pair of Q – the minimal pair of the *Landlord*.

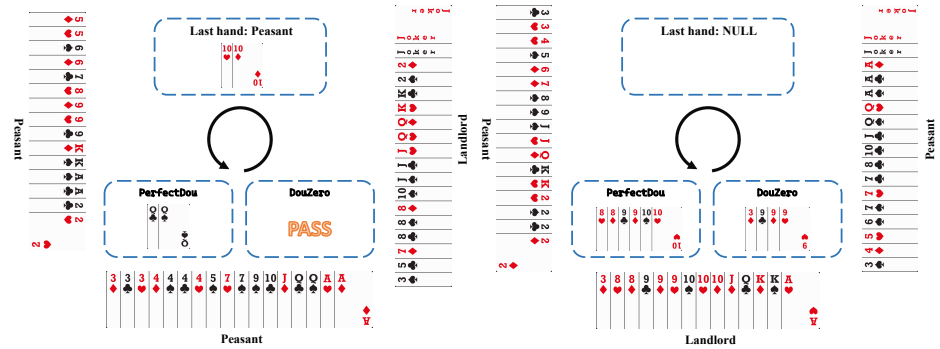
PerfectDou is better at card combination. In the battle shown in Fig. 7(d), PerfectDou shows the better ability on the strategy of card combination. Specifically, PerfectDou chooses to split the plane (999, TTT) since it considers there is a chain of solo ($9TJQK$) left. However, DouZero only

⁵We denote $T(\text{en})$ as the card 10 for simplicity.



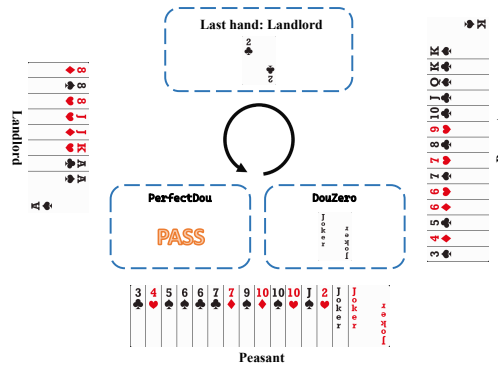
(a) Case study: DouZero is more aggressive by choosing a chain of solo in the beginning but leaves the pair of 3 in the hand.

(b) Case study: DouZero is more aggressive by suppressing the *Landlord* but less thinking on the consequence of the left hands of solos.



(c) Case study: the teammate shows a pair of T and DouZero chooses to pass; on the contrary, PerfectDou chooses suppressing by a pair of Q – the minimal pair of the opponent.

(d) Case study: PerfectDou chooses to split the plane (999, *TTT*) since it considers there is a chain of solo (9*TJQK*) left.



(e) Case study: DouZero splits the rocket bomb while PerfectDou chooses to keep it.

Figure 7: Case studies.

takes the trio, which will be easily suppressed by the opponent. This benefits from the proper design of the card representation and the action feature of PerfectDou.

PerfectDou is more calm. Fig. 7(e) depicts a typical and interesting scenario where PerfectDou shows its calm and careful consideration over the whole. In the game, the last hand is of the *Landlord* with a solo 2, and it only has 8 cards left in the hand. DouZero seems afraid and splits the rocket

bomb; however, PerfectDou benefits from the advantage reward design and is calm considering there is a greater chance on winning the game with a higher score by keeping the bomb.

D.4 Battle Results Against Skilled Human Players

We further invite some skilled human players to play against PerfectDou. Particularly, each human player plays with two AI players. In other words, each game is involved with either two AI *Peasants* against one human *Landlord*, or one AI *Peasant* cooperating with one human *Peasant* against one AI *Landlord*. The results are shown in Tab. 9. One can easily observe that PerfectDou takes evident advantage during the game.

Table 9: Battle results against skilled human players for 1260 episodes of game.

A \ B	Skilled human	
	WP	ADP
PerfectDou (2.5e9)	0.625	0.590

D.5 Additional Training Results

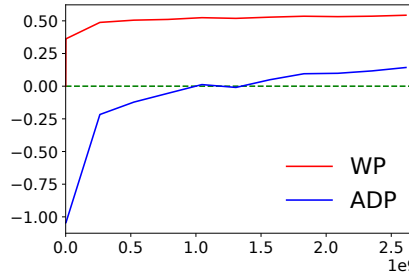


Figure 8: Learning curves of WP and ADP against the final model of DouZero w.r.t. timesteps for PerfectDou. Every evaluation contains 10,000 decks. PerfectDou is able to beat DouZero without considering the scores at the beginning of the training, around 1.5e6 steps.

Fig. 8 shows the learning curves of WP and ADP against DouZero for PerfectDou with a single run, and every evaluation contains 10,000 decks. As shown in the figure, PerfectDou can easily beat DouZero (on WP) without considering the scores (ADP) at the beginning of the training; but after 1.5e9 steps of training, PerfectDou is able to fully beat DouZero (both WP and ADP).

D.6 Extended Competition against DouZero

Cycling Different Hands for the *Landlord*. In our main competition conducted in Section 6.2, all games are randomly generated and played twice with the same assigned hand cards for the *Landlord*, once each algorithm controlling the *Landlord* and once two *Peasants*. In this section, we test cycling the 3 hand cards of every randomly generated game for the *Landlord*, and test PerfectDou against DouZero by controlling the *Landlord* separately, which leads to 6 times of battle. In this situation, we obtain the results as follows showing that PerfectDou has consistent advantage (Tab. 10).

Table 10: Results of cycling different hands for Landlord by playing 100k decks.

A \ B	PerfectDou		DouZero	
	WP	ADP		
PerfectDou	-	-	0.544	0.150
DouZero	0.456	-0.150	-	-

Peasants Paired with Different AIs. We also include an interesting battle by pairing the peasants with different algorithms. Specifically, DouZero or PerfectDou plays the *Landlord* while one *Peasant* is assigned with DouZero and the other *Peasant* is played by PerfectDou. The results are concluded in Tab. 11, which reveal that when playing as the peasant, PerfectDou can better cooperate with its teammate than DouZero. And when playing as the landlord, PerfectDou outperforms DouZero against all types of opponents.

Table 11: Results of battles that the *Peasants* are paired with different AIs by 100k decks and 6 times of battle per deck. The results are evaluated from the *Landlord* side.

Landlord \ Peasant	PerfectDou + DouZero		PerfectDou		DouZero	
	WP	ADP	WP	ADP	WP	ADP
PerfectDou	0.424	-0.448	0.389	-0.606	0.452	-0.375
DouZero	0.395	-0.534	0.363	-0.676	0.421	-0.465

D.7 Complete Tournament Results of ADP for *Landlord* and *Peasants*

We report the complete tournament results of ADP and WP for *Landlord* and *Peasants* in Tab. 12 and Tab. 13. PerfectDou tends to have more advantage of *Peasants* than that of *Landlord*, especially when competes against stronger baselines. We believe that the proposed perfect information distillation technique allows for better cooperation between two *Peasants*. In addition, since the roles are assigned instead of opting according to hand in our competition, and the *Landlord* has extra three cards and can lose a higher base score, the *Peasants* seems having more chance to win the game. Therefore, almost all methods can play better results as a *Peasant* than that as a *Landlord*.

Table 12: ADP results of DouDizhu tournaments for existing AI programs by playing 10k decks. L: ADP of A as Landlord; P: ADP of A as Peasants. Algorithm A outperforms B if the ADP of L or P is larger than 0 (highlighted in boldface). We note that DouZero is the current SoTA DouDizhu bot. Numerical results except marked * are directly borrowed from Zha et al. [31].

Rank	A \ B	PerfectDou		DouZero		DeltaDou		RHCP-v2		CQN		Random	
		P	L	P	L	P	L	P	L	P	L	P	L
1	PerfectDou (Ours)	0.656*	-0.656*	0.686*	-0.407*	0.980*	-0.145*	0.872*	0.138*	2.020*	2.160*	3.008*	3.283*
2	DouZero (Public)	0.407*	-0.686*	0.435*	-0.435*	0.858*	-0.342*	0.166*	-0.046*	2.001*	1.368*	2.818*	3.254*
3	DeltaDou	0.145*	-0.980*	0.342*	-0.858*	0.476	-0.476	1.878*	0.974*	1.849	1.218	2.930	3.268
4	RHCP-v2	-0.138*	-0.872*	0.046*	-0.166*	-0.974*	-1.878*	0.182*	-0.182*	1.069*	1.758*	2.560*	2.780*
5	CQN	-2.160*	-2.020*	-1.368*	-2.001*	-1.218	-1.849	-1.758*	-1.069*	0.056	-0.056	1.992	1.832
6	Random	-3.283*	-3.008*	-3.254*	-2.818*	-3.268	-2.930	-2.780	-2.560*	-1.832	-1.991	0.883	-0.883

Table 13: WP results of DouDizhu tournaments for existing AI programs by playing 10k decks. L: WP of A as Landlord; P: WP of A as Peasants. Algorithm A outperforms B if the WP of L or P is larger than 0.5 (highlighted in boldface). Numerical results except marked * are directly borrowed from Zha et al. [31].

Rank	A \ B	PerfectDou		DouZero		DeltaDou		RHCP-v2		CQN		Random	
		P	L	P	L	P	L	P	L	P	L	P	L
1	PerfectDou (Ours)	0.622*	0.378*	0.640*	0.446*	0.693*	0.474*	0.609*	0.478*	0.894*	0.830*	0.998*	0.990*
2	DouZero (Public)	0.554*	0.360*	0.584*	0.416*	0.684*	0.487*	0.427*	0.475*	0.851*	0.769*	0.992*	0.986*
3	DeltaDou	0.526*	0.307*	0.513*	0.317*	0.588	0.412	0.768*	0.614*	0.835	0.733	0.996	0.987
4	RHCP-v2	0.522*	0.391*	0.525*	0.573*	0.386*	0.232*	0.536*	0.434*	0.687*	0.853*	0.994*	0.985*
5	CQN	0.170*	0.106*	0.231*	0.149*	0.267	0.165	0.147*	0.313*	0.476	0.524	0.921	0.857
6	Random	0.010*	0.002*	0.014*	0.008*	0.013	0.004	0.015*	0.006*	0.143	0.080	0.654	0.346

E More Implementation Details

E.1 The Oracle for Minimum Steps to Play Out All cards

In our paper, as mentioned in Section 4.4, we utilize an oracle for evaluating the minimum steps to play out all cards. Particularly, the oracle is implemented by a dynamic programming algorithm combined with depth-first-search, which can be referred to <https://www.cnblogs.com/SYCstudio/p/7628971.html> (which is also a competition problem of National Olympiad in Informatics in Provinces (NOIP) 2015). For completeness, we summarize the pseudocode for implementing such an algorithm in Algo. 1.

E.2 Detailed Action Space

In our paper, we utilize a simplified discrete action space of 621 for learning the actor, since we observe that the original action space of 27472 (include all possible combinations) contains a large number of actions that can be abstract. For instance, actions like Bomb with kickers and Trio with kickers occupy the action space most due to the large number of combinations of kickers. To this end, we abstract actions with the same main cards into one action, and significantly reduce the action space. When the policy chooses an abstract action, we further deploy a simple decoding function to obtain the most preferred action in the original action space, as illustrated in Algo. 2. We note that this part is similar to an old implementation of RLCard [30]: <https://github.com/datamllab/rlcard/blob/d100952f144e4b0fd7186cc06e79ef277cda9722/rlcard/envs/doudizhu.py#L67>.

Below we list all the 621 discrete actions of PerfectDou in 15 categories, where the notation * in category (9) (10) (11) and (12) denotes the kicker.

(1) Solo (15 actions): 3, 4, 5, 6, 7, 8, 9, T⁶, J, Q, K, A, 2, B, R

(2) Pair (13 actions): 33, 44, 55, 66, 77, 88, 99, TT, JJ, QQ, KK, AA, 22

(3) Trio (13 actions): 333, 444, 555, 666, 777, 888, 999, TTT, JJJ, QQQ, KKK, AAA, 222

(4) Trio with Solo (182 actions): 3334, 3335, 3336, 3337, 3338, 3339, 333T, 333J, 333Q, 333K, 333A, 3332, 333B, 333R, 3444, 4445, 4446, 4447, 4448, 4449, 444T, 444J, 444Q, 444K, 444A, 4442, 444B, 444R, 3555, 4555, 5556, 5557, 5558, 5559, 555T, 555J, 555Q, 555K, 555A, 5552, 555B, 555R, 3666, 4666, 5666, 6667, 6668, 6669, 666T, 666J, 666Q, 666K, 666A, 6662, 666B, 666R, 3777, 4777, 5777, 6777, 7778, 7779, 777T, 777J, 777Q, 777K, 777A, 7772, 777B, 777R, 3888, 4888, 5888, 6888, 7888, 8889, 888T, 888J, 888Q, 888K, 888A, 8882, 888B, 888R, 3999, 4999, 5999, 6999, 7999, 8999, 999T, 999J, 999Q, 999K, 999A, 9992, 999B, 999R, 3TTT, 4TTT, 5TTT, 6TTT, 7TTT, 8TTT, 9TTT, TTTJ, TTTQ, TTTK, TTTA, TTT2, TTTB, TTTR, 3JJJ, 4JJJ, 5JJJ, 6JJJ, 7JJJ, 8JJJ, 9JJJ, TJJJ, JJJQ, JJJK, JJJ A, JJJ2, JJJ B, JJJR, 3QQQ, 4QQQ, 5QQQ, 6QQQ, 7QQQ, 8QQQ, 9QQQ, TQQQ, JQQQ, QQQK, QQQA, QQQ2, QQQB, QQQR, 3KKK, 4KKK, 5KKK, 6KKK, 7KKK, 8KKK, 9KKK, TKKK, JK KK, QKKK, KKKA, KKK2, KKKB, KKKR, 3AAA, 4AAA, 5AAA, 6AAA, 7AAA, 8AAA, 9AAA, TAAA, JAAA, QAAA, KAAA, AAA2, AAAB, AAAR, 3222, 4222, 5222, 6222, 7222, 8222, 9222, T222, J222, Q222, K222, A222, 222B, 222R

(5) Trio with Pair (156 actions): 33344, 33355, 33366, 33377, 33388, 33399, 333TT, 333JJ, 333QQ, 333KK, 333AA, 33322, 33444, 44455, 44466, 44477, 44488, 44499, 444TT, 444JJ, 444QQ, 444KK, 444AA, 44422, 33555, 44555, 55566, 55577, 55588, 55599, 555TT, 555JJ, 555QQ, 555KK, 555AA, 55522, 33666, 44666, 55666, 66677, 66688, 66699, 666TT, 666JJ, 666QQ, 666KK, 666AA, 66622, 33777, 44777, 55777, 66777, 77788, 77799, 777TT, 777JJ, 777QQ, 777KK, 777AA, 77722, 33888, 44888, 55888, 66888, 77888, 88899, 888TT, 888JJ, 888QQ, 888KK, 888AA, 88822, 33999, 44999, 55999, 66999, 77999, 88999, 999TT, 999JJ, 999QQ, 999KK, 999AA, 99922, 33TTT, 44TTT, 55TTT, 66TTT, 77TTT, 88TTT, 99TTT, TTTJJ, TTTQQ, TTTKK, TTTAA, TTT22, 33JJJ, 44JJJ, 55JJJ, 66JJJ, 77JJJ, 88JJJ, 99JJJ, TTJJJ, JJJQQ, JJJKK, JJJAA, JJJ22, 33QQQ, 44QQQ, 55QQQ, 66QQQ, 77QQQ, 88QQQ, 99QQQ, TTQQQ, JJQQQ, QQQKK, QQQAA, QQQ22, 33KKK, 44KKK, 55KKK, 66KKK, 77KKK, 88KKK, 99KKK, TTKKK, JJKKK, QQKKK, KKKA, KKK22, 33AAA, 44AAA, 55AAA, 66AAA, 77AAA,

⁶T for Ten (10).

Algorithm 1 Calculate Minimum Step to Play Out All Cards

```
1: function MAIN
2:    $N_1 \leftarrow$  all possible number of single card
3:    $N_2 \leftarrow$  all possible number of pair card
4:    $N_3 \leftarrow$  all possible number of trio card
5:    $N_4 \leftarrow$  all possible number of bomb card
6:   function INITMATRIX( $F$ )
7:     for  $action \in \{\text{Solo}, \text{Pair}, \text{Trio}, \text{Bomb}, \text{Chain-of-Trio}, \text{Trio-with-Pair}, \text{Quad-with-Solos}, \text{Quad-with-Pair}, \text{Quad-with-Pairs}\}$  do
8:        $d_1 \leftarrow$  the number of Solo card in  $action$ 
9:        $d_2 \leftarrow$  the number of Pair card in  $action$ 
10:       $d_3 \leftarrow$  the number of Trio card in  $action$ 
11:       $d_4 \leftarrow$  the number of Bomb card in  $action$ 
12:       $F[N_1, N_2, N_3, N_4] \leftarrow \min(F[N_1, N_2, N_3, N_4], F[N_1 - d_1, N_2 - d_2, N_3 - d_3, N_4 - d_4] + 1)$ 
13:      if  $action$  is Trio then
14:         $F[N_1, N_2, N_3, N_4] \leftarrow \min(F[N_1, N_2, N_3, N_4], F[N_1 + 1, N_2 + 2, N_3 - 1, N_4])$ 
15:      end if
16:      if  $action$  is Bomb then
17:         $F[N_1, N_2, N_3, N_4] \leftarrow \min(F[N_1, N_2, N_3, N_4], F[N_1 + 1, N_2 + 2, N_3 - 1, N_4])$ 
18:      end if
19:    end for
20:  end function
21:  function NOWSTEP( $Cards$ )
22:    if Rocket  $\in Cards$  then
23:       $left\_cards \leftarrow$  left cards after playing out Rocket
24:      return  $\min(\text{NowStep}(left\_cards) + 1, \text{NowStep}(left\_cards) + 2)$ 
25:    end if
26:    if only one Joker  $\in Cards$  then
27:       $left\_cards \leftarrow$  left cards after playing out Joker
28:      return  $\text{NowStep}(left\_cards) + 1$ 
29:    end if
30:    if no Joker  $\in Cards$  then
31:      calculate number of Solo  $N_1$ , Pair  $N_2$ , Trio  $N_3$  and Bomb  $N_4$  of  $Cards$ 
32:      return  $F[N_1, N_2, N_3, N_4]$ 
33:    end if
34:  end function
35:  function DFS( $step, ans, Cards$ )
36:    if  $step > ans$  then
37:      return  $ans$ 
38:    end if
39:     $ans \leftarrow \min(step, \text{NowStep}(Cards))$ 
40:    for Chain-of-Solo  $\in Cards$  do
41:       $left\_cards \leftarrow$  left cards after playing out Chain-of-Solo
42:       $\text{DFS}(step + 1, ans, left\_cards \text{ of } Cards)$ 
43:    end for
44:    for Chain-of-Pair in  $Cards$  do
45:       $left\_cards \leftarrow$  left cards after playing out Chain-of-Pair
46:       $\text{DFS}(step + 1, ans, left\_cards \text{ of } Cards)$ 
47:    end for
48:    for Plane-with-Solo  $\in Cards$  do
49:       $left\_cards \leftarrow$  left cards after playing out Plane-with-Solo
50:       $\text{DFS}(step + 1, ans, left\_cards \text{ of } Cards)$ 
51:    end for
52:  end function
53:  Create a matrix  $F$  of size  $[N_1, N_2, N_3, N_4]$ 
54:   $\text{InitMatrix}(F)$ 
55:   $step \leftarrow 0, ans \leftarrow +\infty, Cards \leftarrow$  all  $Cards$  to be calculated
56:   $\text{DFS}(0, ans, Cards)$ 
57: end function
```

Algorithm 2 Decode action

```
1: function DECODE( $M$ )
2:    $A \leftarrow$  get all available actions from current hand
3:    $K \leftarrow$  get all kickers using the main card  $M$  from  $A$ 
4:   for  $k$  in  $K$  do
5:     calculate score  $s$  of each  $k$ 
6:      $N \leftarrow$  number of actions contains  $k$  in  $A$ ,  $rank_k \leftarrow$  card rank of  $k$ 
7:      $s \leftarrow 1.0 * N + 0.1 * rank_k$ 
8:   end for
9:   return  $k$  with minimum  $s$ 
10: end function
```

88AAA, 99AAA, TTAAA, JJAAA, QQAAA, KAAAA, AAA22, 33222, 44222, 55222, 66222, 77222, 88222, 99222, TT222, JJ222, QQ222, KK222, AA222

(6) Chain of Solo (36 actions): 34567, 45678, 56789, 6789T, 789TJ, 89TJQ, 9TJQK, TJQKA, 345678, 456789, 56789T, 6789TJ, 789TJQ, 89TJQK, 9TJQKA, 3456789, 456789T, 56789TJ, 6789TJQ, 789TJQK, 89TJQKA, 3456789T, 456789TJ, 56789TJQ, 6789TJQK, 789TJQKA, 3456789TJ, 456789TJQ, 56789TJQK, 6789TJQKA, 3456789TJQ, 456789TJQK, 56789TJQKA, 3456789TJQK, 456789TJQKA, 3456789TJQKA

(7) Chain of Pair (52 actions): 334455, 445566, 556677, 667788, 778899, 8899TT, 99TTJJ, TTJJQQ, JJQQKK, QQKKAA, 33445566, 44556677, 55667788, 66778899, 778899TT, 8899TTJJ, 99TTJJQQ, TTJJQQKK, JJQQKKAA, 3344556677, 4455667788, 5566778899, 66778899TT, 778899TTJJ, 8899TTJJQQ, 99TTJJQQKK, TTJJQQKKAA, 334455667788, 445566778899, 5566778899TT, 66778899TTJJ, 778899TTJJQQ, 8899TTJJQQKK, 99TTJJQQKKAA, 33445566778899, 445566778899TT, 5566778899TTJJ, 66778899TTJJQQ, 778899TTJJQQKK, 8899TTJJQQKKAA, 33445566778899TT, 445566778899TTJJ, 5566778899TTJJQQ, 66778899TTJJQQKK, 778899TTJJQQKKAA, 33445566778899TTJJ, 445566778899TTJJQQ, 5566778899TTJJQQKK, 66778899TTJJQQKKAA, 33445566778899TTJJQQ, 445566778899TTJJQQKK, 5566778899TTJJQQKKAA

(8) Chain of Trio (45 actions): 333444, 444555, 555666, 666777, 777888, 888999, 999TTT, TTTJJJ, JJJQQQ, QQQKKK, KKKAAA, 333444555, 444555666, 555666777, 666777888, 777888999, 888999TTT, 999TTTJJ, TTTJJJQQ, JJJQQKKK, QQQKKKAAA, 333444555666, 444555666777, 555666777888, 666777888999, 777888999TTT, 888999TTTJJ, 999TTTJJQQ, TTTJJJQQKKK, JJJQQKKKAAA, 333444555666777, 444555666777888, 555666777888999, 666777888999TTT, 777888999TTTJJ, 888999TTTJJQQQ, 999TTTJJQQQKKK, TTTJJJQQKKKAAA, 333444555666777888, 444555666777888999, 555666777888999TTT, 666777888999TTTJJ, 777888999TTTJJQQQ, 888999TTTJJQQQKKK, 999TTTJJQQQKKKAAA

(9) Plane with Solo (38 actions): 333444**, 444555**, 555666**, 666777**, 777888**, 888999**, 999TTT**, TTTJJJ**, JJJQQQ**, QQQKKK**, KKKAAA**, 333444555**, 444555666**, 555666777**, 666777888**, 777888999**, 888999TTT**, 999TTTJJJ**, TTTJJJQQQ**, JJJQQQKKK**, QQQKKKAAA**, 333444555666**, 444555666777**, 555666777888**, 666777888999**, 777888999TTT**, 888999TTTJJJ**, 999TTTJJJQQQ**, TTTJJJQQQKKK**, JJJQQQKKKAAA**, 333444555666777**, 444555666777888**, 555666777888999**, 666777888999TTT**, 777888999TTTJJJ**, 888999TTTJJJQQQ**, 999TTTJJJQQQKKK**, TTTJJJQQQKKKAAA**

(10) Plane with Pair (30 actions): 333444****, 444555****, 555666****, 666777****, 777888****, 888999****, 999TTT****, TTTJJJ****, JJJQQQ****, QQQKKK****, KKKAAA****, 333444555****, 444555666****, 555666777****, 666777888****, 777888999****, 888999TTT****, 999TTTJJJ****, TTTJJJQQQ****, JJJQQQKKK****, QQQKKKAAA****, 333444555666****, 444555666777****, 555666777888****, 666777888999****, 777888999TTT****, 888999TTTJJJ****, 999TTTJJJQQQ****, TTTJJJQQQKKK****, JJJQQQKKKAAA****

(11) Quad with Solo (13 actions): 3333**, 4444**, 5555**, 6666**, 7777**, 8888**, 9999**, TTTT**, JJJJ**, QQQQ**, KKKK**, AAAA**, 2222**

(12) Quad with Pair (13 actions): 3333****, 4444****, 5555****, 6666****, 7777****, 8888****, 9999****, TTTT****, JJJJ****, QQQQ****, KKKK****, AAAA****, 2222****

(13) Bomb (13 actions): 3333, 4444, 5555, 6666, 7777, 8888, 9999, TTTT, JJJJ, QQQQ, KKKK, AAAA, 2222

(14) Rocket (1 action): BR

(15) Pass (1 action): PASS

A Closer Look at Invalid Action Masking in Policy Gradient Algorithms

Shengyi Huang and Santiago Ontañón *

College of Computing & Informatics, Drexel University
Philadelphia, PA 19104
{sh3397, so367}@drexel.edu

Abstract

In recent years, Deep Reinforcement Learning (DRL) algorithms have achieved state-of-the-art performance in many challenging strategy games. Because these games have complicated rules, an action sampled from the full discrete action distribution predicted by the learned policy is likely to be invalid according to the game rules (e.g., walking into a wall). The usual approach to deal with this problem in policy gradient algorithms is to “mask out” invalid actions and just sample from the set of valid actions. The implications of this process, however, remain under-investigated. In this paper, we 1) show theoretical justification for such a practice, 2) empirically demonstrate its importance as the space of invalid actions grows, and 3) provide further insights by evaluating different action masking regimes, such as removing masking after an agent has been trained using masking.

Introduction

Deep Reinforcement Learning (DRL) algorithms have yielded state-of-the-art game playing agents in challenging domains such as Real-time Strategy (RTS) games (Vinyals et al. 2017; Vinyals et al. 2019) and Multiplayer Online Battle Arena (MOBA) games (Berner et al. 2019; Ye et al. 2020). Because these games have complicated rules, the valid discrete action spaces of different states usually have different sizes. That is, one state might have 5 valid actions and another state might have 7 valid actions. To formulate these games as a standard reinforcement learning problem with a singular action set, previous work combines these discrete action spaces to a *full discrete action space* that contains available actions of all states (Vinyals et al. 2017; Berner et al. 2019; Ye et al. 2020). Although such a full discrete action space makes it easier to apply DRL algorithms, one issue is that an action sampled from this full discrete action space could be invalid for some game states, and this action will have to be discarded.

To make matters worse, some games have extremely large full discrete action spaces and an action sampled will typically be invalid. As an example, the full discrete action space of Dota 2 has a size of 1,837,080 (Berner et al. 2019),

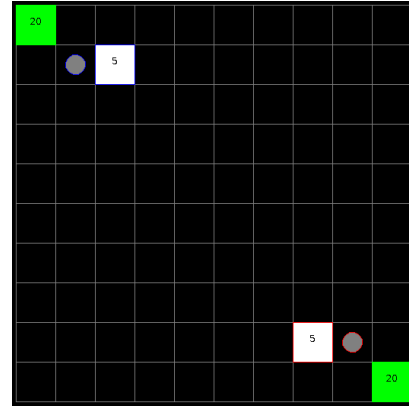


Figure 1: A screenshot of μ RTS. Square units are “bases” (light grey, that can produce workers), “barracks” (dark grey, that can produce military units), and “resources mines” (green, from where workers can extract resources to produce more units), the circular units are “workers” (small, dark grey) and military units (large, yellow or light blue), and on the right is the 10×10 map we used to train agents to harvest resources. The agents could control units at the top left, and the units in the bottom left will remain stationary.

and an action sampled might be to buy an item, which can be *valid* in some game states but will become *invalid* when there is not enough gold. To avoid repeatedly sampling invalid actions in full discrete action spaces, recent work applies policy gradient algorithms in conjunction with a technique known as invalid action masking, which “masks out” invalid actions and then just samples from those actions that are valid (Vinyals et al. 2017; Berner et al. 2019; Ye et al. 2020). To the best of our knowledge, however, the theoretical foundations of invalid action masking have not been studied and its empirical effect is under-investigated. In this paper, we take a closer look at invalid action masking in the context of games, pointing out the gradient produced by invalid action masking corresponds to a valid policy gradient. More interestingly, we show that in fact, invalid action masking can be seen as applying a *state-dependent differentiable function* during the calculation of the action probability distribution, to produce a behavior policy. Next, we de-

*Currently at Google

Copyright © 2022 by the authors. All rights reserved.

sign experiments to compare the performance of *invalid action masking* versus *invalid action penalty*, which is a common approach that gives negative rewards for invalid actions so that the agent learns to maximize reward by not executing any invalid actions. We empirically show that, when the space of invalid actions grows, invalid action masking scales well and the agent solves our desired task while invalid action penalty struggles to explore even the very first reward. Then, we design experiments to answer two questions: (1) What happens if we remove the invalid action mask once the agent was trained with the mask? (2) What is the agent’s performance when we implement the invalid action masking naively by sampling the action from the masked action probability distribution but updating the policy gradient using the unmasked action probability distribution? Finally, we made our source code available at GitHub for the purpose of reproducibility¹.

Background

In this paper, we use policy gradient methods to train agents. Let us consider the Reinforcement Learning problem in a Markov Decision Process (MDP) denoted as $(S, A, P, \rho_0, r, \gamma, T)$, where S is the state space, A is the discrete action space, $P : S \times A \times S \rightarrow [0, 1]$ is the state transition probability, $\rho_0 : S \rightarrow [0, 1]$ is the initial state distribution, $r : S \times A \rightarrow \mathbb{R}$ is the reward function, γ is the discount factor, and T is the maximum episode length. A stochastic policy $\pi_\theta : S \times A \rightarrow [0, 1]$, parameterized by a parameter vector θ , assigns a probability value to an action given a state. The goal is to maximize the expected discounted return of the policy:

$$J = \mathbb{E}_\tau \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]$$

where τ is the trajectory $(s_0, a_0, r_0, \dots, s_{T-1}, a_{T-1}, r_{T-1})$, $s_0 \sim \rho_0$, $s_t \sim P(\cdot | s_{t-1}, a_{t-1})$, $a_t \sim \pi_\theta(\cdot | s_t)$, $r_t = r(s_t, a_t)$

The core idea behind policy gradient algorithms is to obtain the *policy gradient* $\nabla_\theta J$ of the expected discounted return with respect to the policy parameter θ . Doing gradient ascent $\theta = \theta + \nabla_\theta J$ therefore maximizes the expected discounted reward. Earlier work proposes the following policy gradient estimate to the objective J (Sutton and Barto 2018):

$$\nabla_\theta J = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right], G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

Invalid Action Masking

Invalid action masking is a common technique implemented to avoid repeatedly generating invalid actions in large discrete action spaces (Vinyals et al. 2017; Berner et al. 2019; Ye et al. 2020). To the best of our knowledge, there is no literature providing detailed descriptions of the implementation of invalid action masking. Existing work (Vinyals et al. 2017; Berner et al. 2019) seems to treat invalid action masking as an auxiliary detail, usually describing it using only a

few sentences. Additionally, there is no literature providing theoretical justification to explain why it works with policy gradient algorithms. In this section, we examine how invalid action masking is implemented and prove it indeed corresponds to valid policy gradient updates (Sutton et al. 2000). More interestingly, we show it works by applying a *state-dependent differentiable function* during the calculation of action probability distribution.

First, let us see how a discrete action is typically generated through policy gradient algorithms. Most policy gradient algorithms employ a neural network to represent the policy, which usually outputs unnormalized scores (logits) and then converts them into an action probability distribution using a softmax operation or equivalent, which is the framework we will assume in the rest of the paper. For illustration purposes, consider an MDP with the action set $A = \{a_0, a_1, a_2, a_3\}$ and $S = \{s_0, s_1\}$, where the MDP reaches the terminal state s_1 immediately after an action is taken in the initial state s_0 and the reward is always +1. Further, consider a policy π_θ parameterized by $\theta = [l_0, l_1, l_2, l_3] = [1.0, 1.0, 1.0, 1.0]$ that, for the sake of this example, directly produces θ as the output logits. Then in s_0 we have:

$$\begin{aligned} \pi_\theta(\cdot | s_0) &= [\pi_\theta(a_0 | s_0), \pi_\theta(a_1 | s_0), \pi_\theta(a_2 | s_0), \pi_\theta(a_3 | s_0)] \\ &= \text{softmax}([l_0, l_1, l_2, l_3]) \\ &= [0.25, 0.25, 0.25, 0.25], \end{aligned} \quad (1)$$

$$\text{where } \pi_\theta(a_i | s_0) = \frac{\exp(l_i)}{\sum_j \exp(l_j)}$$

At this point, regular policy gradient algorithms will sample an action from $\pi_\theta(\cdot | s_0)$. Suppose a_0 is sampled from $\pi_\theta(\cdot | s_0)$, and the policy gradient can be calculated as follows:

$$\begin{aligned} g_{\text{policy}} &= \mathbb{E}_\tau \left[\nabla_\theta \sum_{t=0}^{T-1} \log \pi_\theta(a_t | s_t) G_t \right] \\ &= \nabla_\theta \log \pi_\theta(a_0 | s_0) G_0 \\ &= [0.75, -0.25, -0.25, -0.25] \\ (\nabla_\theta \log \text{softmax}(\theta)_j)_i &= \begin{cases} (1 - \frac{\exp(l_j)}{\sum_j \exp(l_j)}) & \text{if } i = j \\ -\frac{\exp(l_j)}{\sum_j \exp(l_j)} & \text{otherwise} \end{cases} \end{aligned}$$

Now suppose a_2 is invalid for state s_0 , and the only valid actions are a_0, a_1, a_3 . Invalid action masking helps to avoid sampling invalid actions by “masking out” the logits corresponding to the invalid actions. This is usually accomplished by replacing the logits of the actions to be masked by a large negative number M (e.g. $M = -1 \times 10^8$). Let us use $\text{mask} : \mathbb{R} \rightarrow \mathbb{R}$ to denote this masking process, and we can calculate the re-normalized probability distribution $\pi'_\theta(\cdot | s_0)$ as the following:

$$\pi'_\theta(\cdot | s_0) = \text{softmax}(\text{mask}([l_0, l_1, l_2, l_3])) \quad (2)$$

$$= \text{softmax}([l_0, l_1, M, l_3]) \quad (3)$$

$$\begin{aligned} &= [\pi'_\theta(a_0 | s_0), \pi'_\theta(a_1 | s_0), \epsilon, \pi'_\theta(a_3 | s_0)] \\ &= [0.33, 0.33, 0.0000, 0.33] \end{aligned} \quad (4)$$

where ϵ is the resulting probability of the masked invalid action, which should be a small number. If M is chosen to be

¹<https://github.com/vwxyzjn/invalid-action-masking>

sufficiently negative, the probability of choosing the masked invalid action a_2 will be virtually zero. After finishing the episode, the policy is updated according to the following gradient, which we refer to as the *invalid action policy gradient*.

$$g_{\text{invalid action policy}} = \mathbb{E}_{\tau} \left[\nabla_{\theta} \sum_{t=0}^{T-1} \log \pi'_{\theta}(a_t | s_t) G_t \right] \quad (5)$$

$$= \nabla_{\theta} \log \pi'_{\theta}(a_0 | s_0) G_0 \quad (6)$$

$$= [0.67, -0.33, 0.0000, -0.33]$$

This example highlights that invalid action masking appears to do more than just “renormalizing the probability distribution”; it in fact makes the gradient corresponding to the logits of the invalid action to zero.

Masking Still Produces a Valid Policy Gradient

The action selection process is affected by a process that seems external to π_{θ} that calculates the mask. It is therefore natural to wonder how does the policy gradient theorem (Sutton et al. 2000) apply. As a matter of fact, our analysis shows that the process of invalid action masking can be considered as a state-dependent differentiable function applied for the calculation of π'_{θ} , and therefore $g_{\text{invalid action policy}}$ can be considered as a policy gradient update for π'_{θ} .

Proposition 1. *$g_{\text{invalid action policy}}$ is the policy gradient of policy π'_{θ} .*

Proof. Let $s \in S$ to be arbitrary and consider the process of invalid action masking as a differentiable function $mask$ to be applied to the logits $l(s)$ outputted by policy π_{θ} given state s . Then we have:

$$\pi'_{\theta}(\cdot | s) = \text{softmax}(mask(l(s)))$$

$$mask(l(s))_i = \begin{cases} l_i & \text{if } a_i \text{ is valid in } s \\ M & \text{otherwise} \end{cases}$$

Clearly, $mask$ is either an identity function or a constant function for elements in the logits. Since these two kinds of functions are differentiable, π'_{θ} is differentiable to its parameters θ . That is, $\frac{\partial \pi'_{\theta}(a | s)}{\partial \theta}$ exists for all $a \in A, s \in S$, which satisfies the assumption of policy gradient theorem (Sutton et al. 2000). Hence, $g_{\text{invalid action policy}}$ is the policy gradient of policy π'_{θ} . $\square$

Note that $mask$ is *not* a piece-wise linear function. If we plot $mask$, it is either an identity function or a constant function, depending on the state s , going from $-\infty$ to $+\infty$. We therefore call $mask$ a state-dependent differentiable function. That is, given a vector x and two states s, s' with different number of invalid actions available in these states, $mask(s, x) \neq mask(s', x)$.

Experimental Setup

In the remainder of this paper, we provide a series of empirical results showing the practical implications of invalid action masking.

Table 1: Observation features and action components.

Observation Features	Planes	Description
Hit Points	5	0, 1, 2, 3, ≥ 4
Resources	5	0, 1, 2, 3, ≥ 4
Owner	3	player 1, -, player 2
Unit Types	8	-, resource, base, barrack, worker, light, heavy, ranged
Current Action	6	-, move, harvest, return, produce, attack
Action Components	Range	Description
Source Unit	$[0, h \times w - 1]$	the location of the unit selected to perform an action
Action Type	$[0, 5]$	NOOP, move, harvest, return, produce, attack
Move Parameter	$[0, 3]$	north, east, south, west
Harvest Parameter	$[0, 3]$	north, east, south, west
Return Parameter	$[0, 3]$	north, east, south, west
Produce Direction Parameter	$[0, 3]$	north, east, south, west
Produce Type Parameter	$[0, 6]$	resource, base, barrack, worker, light, heavy, ranged
Attack Unit	Target $[0, h \times w - 1]$	the location of unit that will be attacked

Evaluation Environment

We use μRTS^2 as our testbed, which is a minimalistic RTS game maintaining the core features that make RTS games challenging from an AI point of view: simultaneous and durative actions, large branching factors, and real-time decision-making. A screenshot of the game can be found in Figure 1. It is the perfect testbed for our experiments because the action space in μRTS grows combinatorially and so does the number of invalid actions that could be generated by the DRL agent. We now present the technical details of the environment for our experiments.

- **Observation Space.** Given a map of size $h \times w$, the observation is a tensor of shape (h, w, n_f) , where n_f is a number of feature planes that have binary values. The observation space used in this paper uses 27 feature planes as shown in Table 1, similar to previous work in μRTS (Stanescu et al. 2016; Yang and Onta  n 2018; Huang and Onta  n 2019). A feature plane can be thought of as a concatenation of multiple one-hot encoded features. As an example, if there is a worker with hit points equal to 1, not carrying any resources, the owner being Player 1, and currently not executing any actions, then the one-hot encoding features will look like the following:

$$[0, 1, 0, 0, 0], [1, 0, 0, 0, 0], [1, 0, 0],$$

$$[0, 0, 0, 0, 1, 0, 0, 0], [1, 0, 0, 0, 0, 0]$$

²<https://github.com/santiontanon/microrts>

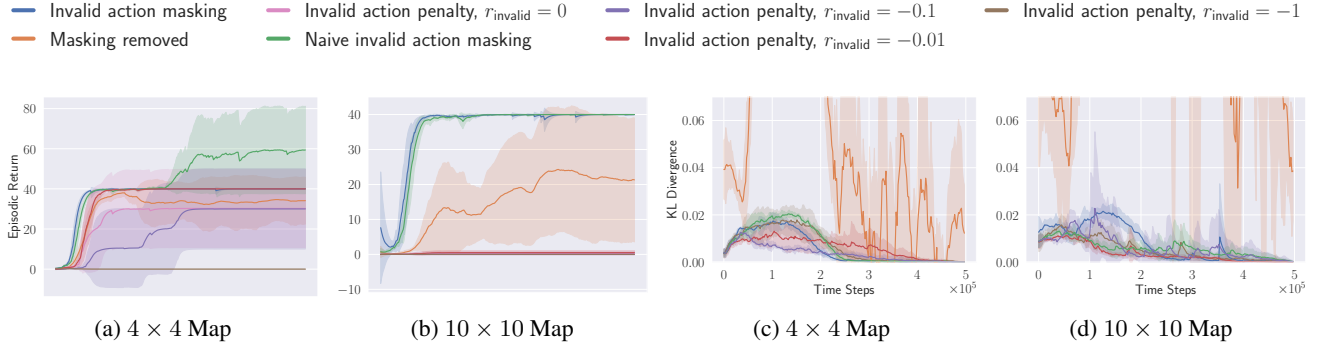


Figure 2: The first two figures show the episodic return over the time steps, and the remaining two show the Kullback–Leibler (KL) divergence between the target and current policy of PPO over the time steps. The shaded area represents one standard deviation of the data over 4 random seeds. Curves are exponentially smoothed with a weight of 0.9 for readability.

The 27 values of each feature plane for the position in the map of such worker will thus be the concatenation of the arrays above.

- **Action Space.** Given a map of size $h \times w$, the action is an 8-dimensional vector of discrete values as specified in Table 1. The action space is designed similar to the action space formulation by Hausknecht, et al., (Hausknecht and Stone 2016). The first component of the action vector represents the unit in the map to issue actions to, the second is the action type, and the rest of the components represent the different parameters different action types can take. Depending on which action type is selected, the game engine will use the corresponding parameters to execute the action.
- **Rewards.** We are evaluating our agents on the simple task of harvesting resources as fast as they can for Player 1 who controls units at the top left of the map. A +1 reward is given when a worker harvests a resource, and another +1 is received once the worker returns the resource to a base.
- **Termination Condition.** We set the maximum game length to be 200 time steps, but the game could be terminated earlier if all the resources in the map are harvested first.

Notice that the space of invalid actions becomes significantly larger in larger maps. This is because the range of the first and last discrete values in the action space, corresponding to *Source Unit* and *Attack Target Unit* selection, grows linearly with the size of the map. To illustrate, in our experiments, there are usually only two units that can be selected as the *Source Unit* (the base and the worker). Although it is possible to produce more units or buildings to be selected, the production behavior has no contribution to reward and therefore is generally not learned by the agent. Note the range of *Source Unit* is $4 \times 4 = 16$ and $24 \times 24 = 576$, in maps of size 4×4 and 24×24 , respectively. Selecting a valid *Source Unit* at random has a probability of $2/16 = 0.125$ in the 4×4 map and $2/576 = 0.0034$ in the 24×24 map. With such action space, we can examine the scalability of invalid action masking.

Training Algorithm

We use Proximal Policy Optimization (Schulman et al. 2017) as the DRL algorithm to train our agents.

Strategies to Handle Invalid Actions

To examine the empirical importance of invalid action masking, we compare the following four strategies to handle invalid actions.

1. **Invalid action penalty.** Every time the agent issues an invalid action, the game environment adds a non-positive reward $r_{\text{invalid}} \leq 0$ to the reward produced by the current time step. This technique is standard in previous work (Dietterich 2000). We experiment with $r_{\text{invalid}} \in \{0, -0.01, -0.1, -1\}$, respectively, to study the effect of the different scales on the negative reward.
2. **Invalid action masking.** At each time step t , the agent receives a mask on the *Source Unit* and *Attack Target Unit* features such that only valid units can be selected and targeted. Note that in our experiments, invalid actions still could be sampled because the agent could still select incorrect parameters for the current action type. We didn’t provide a feature-complete invalid action mask for simplicity, as the mask on *Source Unit* and *Attack Target Unit* already significantly reduce the action space.
3. **Naive invalid action masking.** At each time step t , the agent receives the same mask on the *Source Unit* and *Attack Target Unit* as described for invalid action masking. The action shall still be sampled according to the re-normalized probability calculated in Equation 4, which ensures no invalid actions could be sampled, but the gradient is updated according to the probability calculated in Equation 1. We call this implementation *naive invalid action masking* because its gradient does not replace the gradient of the logits corresponds to invalid actions with zero.
4. **Masking removed.** At each time step t , the agent receives the same mask on the *Source Unit* and *Attack Target Unit* as described for invalid action masking, and trains in the same way as the agent trained under invalid action mask-

Table 2: Results averaged over 4 random seeds. The symbol “-” means “not applicable”. Higher is better for r_{episode} and lower is better for a_{null} , a_{busy} , a_{owner} , t_{solve} , and t_{first} .

Strategies	Map size	r_{invalid}	r_{episode}	a_{null}	a_{busy}	a_{owner}	t_{solve}	t_{first}
Invalid action penalty	4×4	-1.00	0.00	0.00	0.00	0.00	-	0.53%
		-0.10	30.00	0.02	0.00	0.00	50.94%	0.52%
		-0.01	40.00	0.02	0.00	0.00	14.32%	0.51%
		0.00	30.25	2.17	0.22	2.70	36.00%	0.60%
	10×10	-1.00	0.00	0.00	0.00	0.00	-	3.43%
		-0.10	0.00	0.00	0.00	0.00	-	2.18%
		-0.01	0.50	0.00	0.00	0.00	-	1.57%
		0.00	0.25	90.10	0.00	102.95	-	3.41%
	16×16	-1.00	0.25	0.00	0.00	0.00	-	0.44%
		-0.10	0.75	0.00	0.00	0.00	-	0.44%
		-0.01	1.00	0.02	0.00	0.00	-	0.44%
		0.00	1.00	184.68	0.00	2.53	-	0.40%
	24×24	-1.00	0.00	49.72	0.00	0.02	-	1.40%
		-0.10	0.25	0.00	0.00	0.00	-	1.40%
		-0.01	0.50	0.00	0.00	0.00	-	1.92%
		0.00	0.50	197.68	0.00	0.90	-	1.83%
Invalid action masking	04x04	-	40.00	-	-	-	8.67%	0.07%
	10x10	-	40.00	-	-	-	11.13%	0.05%
	16x16	-	40.00	-	-	-	11.47%	0.08%
	24x24	-	40.00	-	-	-	18.38%	0.07%
Masking removed	04x04	-	33.53	63.57	0.00	17.57	76.42%	-
	10x10	-	25.93	128.76	0.00	7.75	94.15%	-
	16x16	-	17.32	165.12	0.00	0.52	-	-
	24x24	-	17.37	150.06	0.00	0.94	-	-
Naive invalid action masking	4×4	-	59.61	-	-	-	11.74%	0.07%
	10×10	-	40.00	-	-	-	13.97%	0.05%
	16×16	-	40.00	-	-	-	30.59%	0.10%
	24×24	-	38.50	-	-	-	49.14%	0.07%

ing. However, we then evaluate the agent without providing the mask. In other words, in this scenario, we evaluate what happens if we train with a mask, but then perform without it.

We evaluate the agent’s performance in maps of sizes 4×4 , 10×10 , 16×16 , and 24×24 . All maps have one base and one worker for each player, and each worker is located near the resources.

Evaluation Metrics

We used the following metrics to measure the performance of the agents in our experiments: r_{episode} is the average episodic return over the last 10 episodes. a_{null} is the average number of actions that select a *Source Unit* that is not valid over the last 10 episodes. a_{busy} is the average number of actions that select a *Source Unit* that is already busy executing other actions over the last 10 episodes. a_{owner} is the average number of actions that select a *Source Unit* that does not belong to Player 1 over the last 10 episodes. t_{solve} is the percentage of total training time steps that it takes for the agents’ moving average episodic return of the last 10 episodes to exceed 40. t_{first} is the percentage of the total training time step that it takes for the agent to receive the first positive reward.

Evaluation Results

We report the results in Figure 2 and in Table 2. Here we present a list of important observations:

Invalid action masking scales well. Invalid action masking is shown to scale well as the number of invalid actions increases; t_{solve} is roughly 12% and very similar across different map sizes. In addition, the t_{first} for invalid action masking is not only the lowest across all experiments (only taking about 0.05 – 0.08% of the total time steps), but also consistent against different map sizes. This would mean the agent was able to find the first reward very quickly regardless of the map sizes.

Invalid action penalty does not scale. Invalid action penalty is able to achieve good results in 4×4 maps, but it does not scale to larger maps. As the space of invalid action gets larger, sometimes it struggles to even find the very first reward. E.g. in the 10×10 map, agents trained with invalid action penalty with $r_{\text{invalid}} = -0.01$ spent 3.43% of the entire training time just discovering the first reward, while agents trained with invalid action masking take roughly 0.06% of the time in all maps. In addition, the hyper-parameter r_{invalid} can be difficult to tune. Although having a negative r_{invalid} did encourage the agents not to execute any invalid actions (e.g. a_{null} , a_{busy} , a_{owner} are usually very close to zero for these agents), setting $r_{\text{invalid}} = -1$

seems to have an adverse effect of discouraging exploration by the agent, therefore achieving consistently the worst performance across maps.

KL divergence explodes for naive invalid action masking. According to Table 2, the r_{episode} of naive invalid action masking is the best across almost all maps. In the 4×4 map, the agent trained with naive invalid action masking even learns to travel to the other side of the map to harvest additional resources. However, naive invalid action masking has two main issues: 1) As shown in the second row of Figure 3, the average Kullback–Leibler (KL) divergence (Kullback and Leibler 1951) between the target and current policy of PPO for naive invalid action masking is significantly higher than that of any other experiments. Since the policy changes so drastically between policy updates, the performance of naive invalid action masking might suffer when dealing with more challenging tasks. 2) As shown in Table 2, the t_{solve} of naive invalid action masking is more volatile and sensitive to the map sizes. In the 24×24 map, for example, the agents trained with naive invalid action masking take 49.14% of the entire training time to converge. In comparison, agents trained with invalid action masking exhibit a consistent $t_{\text{solve}} \approx 12\%$ in all maps.

Masking removed still behaves to some extent. As shown in Figures 3d, masking removed is still able to perform well to a certain degree. As the map size gets larger, its performance degrades and starts to execute more invalid actions by, most prominently, selecting an invalid *Source Unit*. Nevertheless, its performance is significantly better than that of the agents trained with invalid action penalty even though they are evaluated without the use of invalid action masking. This shows that the agents trained with invalid action masking can, to some extent, still produce useful behavior when the invalid action masking can no longer be provided.

Related Work

There have been other approaches to deal with invalid actions. Dulac-Arnold, Evans, et al. (Dulac-Arnold et al. 2015) suggest embedding discrete action spaces into a continuous action space, using nearest-neighbor methods to locate the nearest valid actions. In the field of games with natural language, others propose to train an Action Elimination Network (AEN) (Zahavy et al. 2018) to reduce the action set.

The purpose of avoiding executing invalid actions arguably is to boost exploration efficiency. Some less related work achieves this purpose by reducing the full discrete action space to a simpler action space. Kanervisto, et al. (Kanervisto, Scheller, and Hautamäki 2020) describes this kind of work as “action space shaping”, which typically involves 1) action removals (e.g. Minecraft RL environment removes non-useful actions such as “sneak” (Johnson et al. 2016)), and 2) discretization of continuous action space (e.g. the famous CartPole-v0 environment discretize the continuous forces to be applied to the cart (Brockman et al. 2016)). Although a well-shaped action space could help the agent efficiently explore and learn a useful policy, action space shaping is shown to be potentially difficult to tune and sometimes detrimental in helping the agent solve the desired tasks (Dulac-Arnold et al. 2015).

Lastly, Kanervisto, et al. (Kanervisto, Scheller, and Hautamäki 2020) and Ye, et al. (Ye et al. 2020) provide ablation studies to show invalid action masking could be important to the performance of agents, but they do not study the empirical effect of invalid action masking as the space of invalid action grows, which is addressed in this paper.

Conclusions

In this paper, we examined the technique of invalid action masking, which is a technique commonly implemented in policy gradient algorithms to avoid executing invalid actions. Our work shows that: 1) the gradient produced by invalid action masking is a valid policy gradient, 2) it works by applying a *state-dependent differentiable function* during the calculation of action probability distribution, 3) invalid action masking empirically scales well as the space of invalid action gets larger; in comparison, the common technique of giving a negative reward when an invalid action is issued fails to scale, sometimes struggling to find even the first reward in our environment, 4) the agent trained with invalid action masking was still able to produce useful behaviors with masking removed.

Given the clear effectiveness of invalid action masking demonstrated in this paper, we believe the community would benefit from wider adoption of this technique in practice. Invalid action masking empowers the agents to learn more efficiently, and we ultimately hope that it will accelerate research in applying DRL to games with large and complex discrete action spaces.

Appendices

Details on the Training Algorithm Proximal Policy Optimization

The DRL algorithm that we use to train the agent is Proximal Policy Optimization (PPO) (Schulman et al. 2017), one of the state of the art algorithms available. There are two important details regarding our PPO implementation that warrants explanation, as those details are not elaborated in the original paper. The first detail concerns how to generate an action in the `MultiDiscrete` action space as defined in the OpenAI Gym environment (Brockman et al. 2016) of gym-microrts (Huang and Ontañón 2019), while the second detail is about the various code-level optimizations utilized to augment performance. As pointed out by Engstrom, Ilyas, et al. (Engstrom et al. 2020), such code-level optimizations could be critical to the performance of PPO.

Multi Discrete Action Generation To perform an action a_t in μ RTS, according to Table 1, we have to select a Source Unit, Action Type, and its corresponding action parameters. So in total, there are $hw \times 6 \times 4 \times 4 \times 4 \times 4 \times 6 \times hw = 9216(hw)^2$ number of possible discrete actions (including invalid ones), which grows exponentially as we increase the map size. If we apply the PPO directly to this discrete action space, it would be computationally expensive to generate the distribution for $9216(hw)^2$ possible actions. To simplify this combinatorial action space, `openai/baselines` (Dhariwal et al. 2017) library proposes an idea to consider this discrete action to be composed from some smaller *independent* discrete actions. Namely, a_t is composed of smaller actions

$$\begin{aligned} & a_t^{\text{Source Unit}}, a_t^{\text{Action Type}}, a_t^{\text{Move Parameter}}, a_t^{\text{Harvest Parameter}}, \\ & a_t^{\text{Return Parameter}}, a_t^{\text{Produce Direction Parameter}}, a_t^{\text{Produce Type Parameter}}, a_t^{\text{Attack Target Unit}} \end{aligned}$$

And the policy gradient is updated in the following way (without considering the PPO’s clipping for simplicity)

$$\begin{aligned} \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t &= \sum_{t=0}^{T-1} \nabla_{\theta} \left(\sum_{d \in D} \log \pi_{\theta}(a_t^d | s_t) \right) G_t \\ &= \sum_{t=0}^{T-1} \nabla_{\theta} \log \left(\prod_{d \in D} \pi_{\theta}(a_t^d | s_t) \right) G_t \end{aligned}$$

$$D = \{\text{Source Unit, Action Type, Move Parameter, Harvest Parameter, Return Parameter, Produce Direction Parameter, Produce Type Parameter, Attack Target Unit,}\}$$

Implementation wise, for each Action Component of range $[0, x - 1]$, the logits of the corresponding shape x is generated, which we call Action Component logits, and each a_t^d is sampled from this Action Component logits. Because of this idea, the algorithm now only has to generate $hw + 6 + 4 + 4 + 4 + 4 + 6 + hw = 2hw + 36$ number of logits, which is significantly less than $9216(hw)^2$. To the best of our knowledge, this approach of handling large multi discrete action space is only mentioned by Kanervisto et, al (Kanervisto, Scheller, and Hautamäki 2020).

Code-level Optimizations Here is a list of code-level optimizations utilized in this experiments. For each of these optimizations, we include a footnote directing the readers to the files in the `openai/baselines` (Dhariwal et al. 2017) that implements these optimization.

1. **Normalization of Advantages**³: After calculating the advantages based on GAE, the advantages vector is normalized by subtracting its mean and divided by its standard deviation.
2. **Normalization of Observation**⁴: The observation is pre-processed before feeding to the PPO agent. The raw observation was normalized by subtracting its running mean and divided by its variance; then the raw observation is clipped to a range, usually $[-10, 10]$.
3. **Rewards Scaling**⁵: Similarly, the reward is pre-processed by dividing the running variance of the discounted the returns, following by clipping it to a range, usually $[-10, 10]$.
4. **Value Function Loss Clipping**⁶: The PPO implementation of `openai/baselines` clips the value function loss in a manner that is similar to the PPO’s clipped surrogate objective:

$$V_{loss} = \max \left[(V_{\theta_t} - V_{target})^2, (V_{\theta_{t-1}} + \text{clip}(V_{\theta_t} - V_{\theta_{t-1}}, -\epsilon, \epsilon))^2 \right]$$

where V_{target} is calculated by adding $V_{\theta_{t-1}}$ and the A calculated by General Advantage Estimation(Schulman et al. 2016).

³<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L139>

⁴https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/common/vec_env/vec_normalize.py#L4

⁵https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/common/vec_env/vec_normalize.py#L4

⁶<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L68-L75>

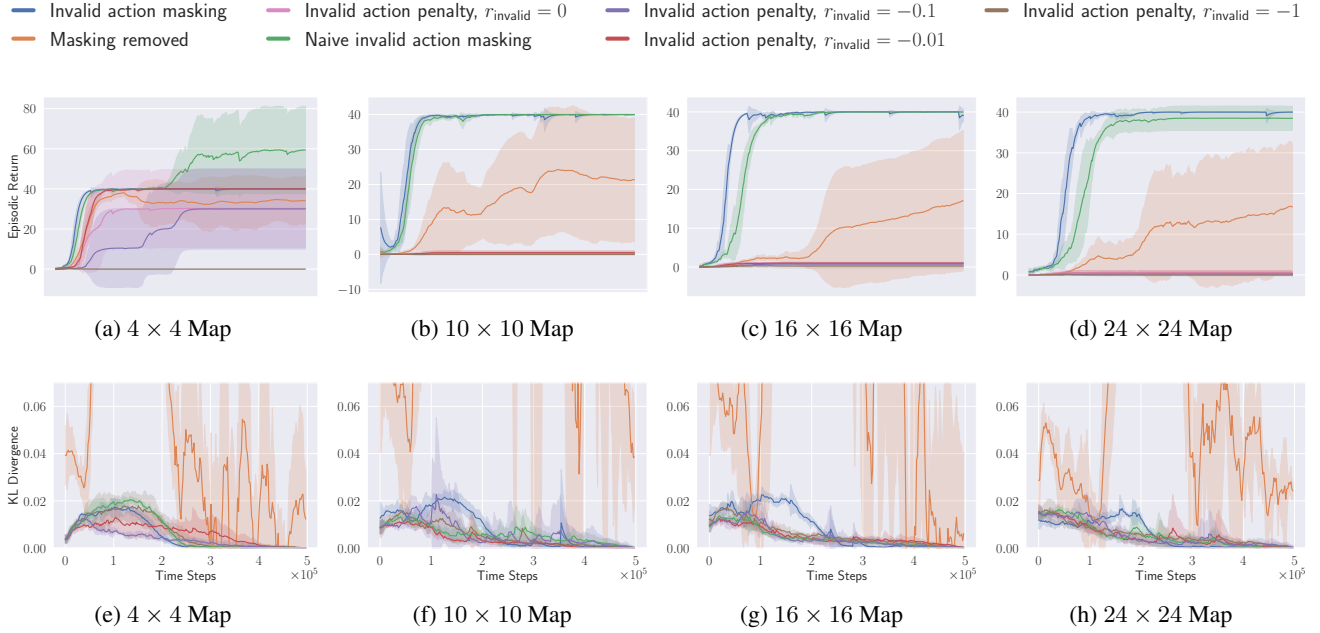


Figure 3: The first row shows the episodic return over the time steps, and the second row shows the Kullback–Leibler (KL) divergence between the target and current policy of PPO over the time steps. The shaded area represents one standard deviation of the data over 4 random seeds. Curves are exponentially smoothed with a weight of 0.9 for readability.

5. **Adam Learning Rate Annealing**⁷: The Adam (Kingma and Ba 2015) optimizer’s learning rate is set to decay as the number of timesteps agent trained increase.
6. **Mini-batch updates**⁸: The PPO implementation of the *openai/baselines* also uses minibatches to compute the gradient and update the policy instead of the whole batch data such as in *open/spinningup*. The mini-batch sampling scheme, however, still makes sure that every transition is sampled only once, and that the all the transitions sampled are actually for the network update.
7. **Global Gradient Clipping**⁹: For each update iteration in an epoch, the gradients of the policy and value network are clipped so that the “global ℓ_2 norm” (i.e. the norm of the concatenated gradients of all parameters) does not exceed 0.5.
8. **Orthogonal Initialization of weights**¹⁰: The weights and biases of fully connected layers use with orthogonal initialization scheme with different scaling. For our experiments, however, we always use the scaling of 1 for historical reasons.

Additional Details on the μ RTS Environment Setup

Each action in μ RTS takes some internal game time, measured in ticks, for the action to be completed. *gym-microrts* (Huang and Ontaño 2019) sets the time of performing harvest action, return action, and move action to be 10 game ticks. Once an action is issued to a particular unit, the unit would be considered as a “busy” unit and would therefore no longer be able to execute any actions until its current action is finished. To prevent the DRL algorithms from repeatedly issuing actions to “busy” units, *gym-microrts* allows performing frame skipping of 9 frames such that from the agent’s perspective, once it executes the harvest action, return action, or move action given the current observation, those actions would be finished in the next observation. Such frame skipping is used for all of our experiments.

Reproducibility

It is important to for the research work to be reproducible. We now present the list of hyperparameters used in Table 4 and the list of neural network architecture in Table 5. In addition, we provide the source code to reproduce our experiments at GitHub¹¹.

⁷<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/ppo2.py#L135>

⁸<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/ppo2.py#L160-L162>

⁹<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L107>

¹⁰<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/a2c/utils.py#L58>

¹¹<https://github.com/neurips2020submission/invalid-action-masking>

Table 3: The list of feature maps and their descriptions.

Features	Planes	Description
Hit Points	5	0, 1, 2, 3, ≥ 4
Resources	5	0, 1, 2, 3, ≥ 4
Owner	3	player 1, -, player 2
Unit Types	8	-, resource, base, barrack, worker, light, heavy, ranged
Current Action	6	-, move, harvest, return, produce, attack

Table 4: The list of experiment parameters and their values.

Parameter Names	Parameter Values
Total Time steps	500,000 time steps
γ (Discount Factor)	0.99
λ (for GAE)	0.97
ε (PPO’s Clipping Coefficient)	0.2
η (Entropy Regularization Coefficient)	0.01
ω (Gradient Norm Threshold)	0.5
K (Number of PPO Update Iteration Per Epoch)	10
α_π Policy’s Learning Rate	0.0003
α_v Value Function’s Learning Rate	0.0003

Table 5: Neural Network Architecture. To explain the notation, let us provide detailed description of the architecture used in 24×24 map as an example. The input to the neural network is a tensor of shape $(24, 24, 27)$. The first hidden layer convolves 16 3×3 filters with stride 1 with the input tensor followed by a 2×2 max pooling layer (Ranzato et al. 2007) and applies a rectifier nonlinearity (Nair and Hinton 2010). The second hidden layer similarly convolves 32 2×2 filters with stride 1 followed by a 2×2 max pooling layer and applies a rectifier nonlinearity. The final hidden layer is a fully connected linear layer consisting of 128 rectifier units. The output layer is a fully connected linear layer with $2hw + 36 = 1188$ number of output.

4×4	10×10
Conv2d(27, 16, kernel_size=2,), MaxPool2d(1), ReLU() Flatten() Linear(144, 128), ReLU(), Linear(128, 68)	Conv2d(27, 16, kernel_size=3,), MaxPool2d(1), ReLU(), Conv2d(16, 32, kernel_size=3), MaxPool2d(1), ReLU() Flatten() Linear(1152, 128), ReLU(), Linear(128, 236)
16×16	24×24
Conv2d(27, 16, kernel_size=3), MaxPool2d(1), ReLU(), Conv2d(16, 32, kernel_size=3), MaxPool2d(1), ReLU() Flatten() Linear(4608, 128), ReLU(), Linear(128, 548)	Conv2d(27, 16, kernel_size=3, stride=1), MaxPool2d(2), ReLU(), Conv2d(16, 32, kernel_size=2, stride=1), MaxPool2d(2), ReLU() Flatten() Linear(800, 128), ReLU(), Linear(128, 1188)

Mastering Strategy Card Game (Hearthstone) with Improved Techniques

Changnan Xiao*
ByteDance
changnanxiao@gmail.com

Yongxin Zhang*
ByteDance
zhangyongxin.yx@bytedance.com

Xuefeng Huang
ByteDance
wangfuming@bytedance.com

Qinhan Huang
ByteDance
huangqinhan@bytedance.com

Jie Chen
ByteDance
chenjiexjtu@gmail.com

Peng Sun†
ByteDance
pengsun000@gamil.com

Abstract—Strategy card game is a well-known genre that is demanding on the intelligent game-play and can be an ideal test-bench for AI. Previous work combines an end-to-end policy function and an optimistic smooth fictitious play, which shows promising performances on the strategy card game *Legend of Code and Magic*. In this work, we apply such algorithms to *Hearthstone*, a famous commercial game that is more complicated in game rules and mechanisms. We further propose several improved techniques and consequently achieve significant progress. For a machine-vs-human test we invite a *Hearthstone* streamer whose best rank was top 10 of the official league in China region that is estimated to be of millions of players. Our models defeat the human player in all *Best-of-5* tournaments of full games (including both deck building and battle), showing a strong capability of decision making.

I. INTRODUCTION

There are many popular strategy card games, such as *Hearthstone*, *Gwent*, *Magic: The Gathering*, *Yu-Gi-Oh: Master Duel*, *Marvel Snap*, and so on. Most strategy card games include at least two stages, where the player constructs a card deck at stage one and battles with the cards-in-deck at stage two. The multiple stages increase the complexity of the game, which is challenging for the player to make decisions and win the match. The multiple stages also open the possibility of in-transitive policies (i.e., the Rock-Paper-Scissors phenomenon), which characterizes a player’s game-play and makes the game intriguing. Since strategy card game is a finite two-player zero-sum game, the *Nash Equilibrium* exists theoretically if the mixed policy is allowed. However, due to the fact that the game is partially observable (imperfect information), where the opponent’s deck and hand are hidden, it’s difficult to obtain NE by dynamic programming or other planning-based methods as in conventional perfect information game. Due to the gap between NE existence and NE finding, strategy card game is thus an appealing test-bench for academic study, and it’s interesting to achieve the master-level intelligence of the strategy card game.

In this work, we focus on a commercial game, *Hearthstone*. Our results demonstrate that the previous algorithm framework [1], injected with several improved techniques proposed in this

study, is able to produce master-level agents on this complicated strategy card game. We evaluate our models in two ways. One is machine-vs-machine evaluation. Our model has 73.6% winrates versus the baseline, where the baseline algorithm won the double championship in COG2022 *Legend of Code and Magic* competition and outperformed other submissions by a large margin. We further explore the feasibility to improve the performance by cheating on the opponent deck and achieve 80.2% winrates versus the baseline. The other is machine-vs-human evaluation. We invite a human player whose best rank was top 10 of the official league in China region that is estimated to be of millions of players. Our models of both the normal-version and cheat-version win all the *Best-of-5* (Bo5) tournaments against the human player under the *Conquest* (CQ) rule.

The rest of this study is organized as follows. To begin with, we introduce the background of *Hearthstone*, related works and the approach that won the double championship in COG2022 competition. Then we describe how the approach is adapted to *Hearthstone*. After that, we show the drawbacks of the approach and how we improve on it. Finally, we do the machine-vs-human evaluation to verify the capability of our approach.

II. BACKGROUND

A. *Hearthstone*

Hearthstone is a competitive two-player turn-based card game. The entire gameplay contains three stages, which are *picking hero* (PH) stage, cards deck building (CB) stage and *battle* (BT) stage. The standard game mode is to play with a randomly chosen opponent player. The goal is to defeat the opponent hero in the BT stage. During the PH stage, the player has to pick a hero, where each hero has a unique Hero Power to be played in the BT stage. The opponent’s picked hero is hidden to the player until the BT stage (see below). During the CB stage, the player drafts a deck containing 30 cards, which comes from a card pool consisting of both standard cards and hero-specific cards. The deck is hidden information to the opponent. During the BT stage, in each turn, the player draws some hand cards from the deck and plays cards under limited

* Equal contributions, † Corresponding author.

resource constraint, i.e., the mana that gradually increases with the number of turns. The player can play multiple cards or execute multiple activities under the resource constraint. The player can pre-end the turn or exhaust all resources. Once the player’s turn ends, it becomes the opponent’s turn to play. The goal is to defeat the opponent hero by reducing its health point to zero.

Our game environment is modified from the open-sourced implementation *Hearthbreaker*¹. Our code supports three heroes: Mage, Warrior and Hunter. A card pool of 350+ cards in total are developed, where on average 270+ cards, including 240 common cards and 30+ hero-specific cards, are available to each hero during the CB stage. The game rules and mechanisms are approximately equivalent to the commercial *Hearthstone* game of April 2015 (the Blackrock Mountain version).

There exists an official ranking system of Standard Mode to measure the gameplay level of all players in a particular region². By the end of year 2022, there are globally four geographically divided regions: Americas, Europe, Asia-Pacific and China³. Except for New Player Ranks, there are Normal Ranks and Legend Ranks. In Normal Ranks, there are 5 leagues (Bronze, Silver, Gold, Platinum, Diamond) with ranks from 10 to 1. Beyond Diamond 1, it’s the Legend Ranks. The icon of Legend Ranks shows a number, n , for each legendary player, which represents that the player is the best n -th player in the region.

Besides the ranking system, tournament is a more straightforward way to evaluate a player’s level, which is the main game format in typical eSport activities. We adopt the *Best of 5* (Bo5) tournament under *Conquest* (CQ)⁴ rule. In a tournament, each player has to prepare multiple decks and each hero can only have at most one deck. CQ requires that any hero (and the deck it brings) that wins a match cannot be played in the remaining matches.

B. Related Work

Deep Reinforcement Learning (DRL) has shown many impressive results on different kinds of games, including single-agent video games [2], [3], two-player games [4] and also multi-player games [5], [6], [7]. Strategy card games is a genre that has the following features: *Partially-Observable Markov Decision Process* (POMDP) and *two-player zero-sum* (2p0s).

To solve a MDP, the objective of DRL is to

$$\text{maximize } \mathbb{E}_{\pi} \left[\sum_t \gamma^t r_t \right], \quad (1)$$

where the expectation is over all trajectories sampled by interactions between the environment and the target policy π , t is the timestamp, γ is the discount rate and r_t is the reward function. To deal with POMDP, various methods are proposed in the literature: to enlarge the observation space [8],

[3], [9], to learn a latent dynamic of the state [10], or to adopt a centralized critic [11].

A 2p0s game is fundamentally different from a single-player MDP. To solve 2p0s game with RL, a meta strategy is required as wrapper for RL in order to efficiently find *Nash Equilibrium* (NE). Regret minimization are widely studied to solve the imperfect information games in a value-centric manner [12], [13]. The Smooth Fictitious Play (SFP) [14], which is essentially Dual Averaging (DA) method [15]), is yet another promising way to find NE in a policy-centric manner. However, it’s somewhat inconvenient in practice, as SFP/DA is only guaranteed to be *average-iterate* convergence (which means the mixture of the policy checkpoints during training converges), and it is notoriously difficult to learn and maintain the average policy when using neural network as policy function approximator in a multi-step game [16]. Some recent works propose a NE finding algorithm with *last-iterate* convergence, where it is unnecessary to separately learn an average policy and the policy of the last iteration suffices to be optimal with some constraints [17], [18], [19]. In particular, [1] modifies the standard SFP and propose the Optimistic SFP that also shows last-iterate convergence.

Hearthstone AI was studied in previous literature, where tree search based methods [20], [21], [22] and evolutionary methods [23] are proposed, which are further combined with Supervised Learning and Reinforcement Learning [24]. In this work, however, we solely rely on deep reinforcement learning and fictitious play which are trained from scratch and do not require look-ahead search in both training and inference-time.

Meanwhile, to our best knowledge, there were no previous Hearthstone AIs showing the ability to defeat top human players in full-game. In [22] and the *hearthstone-ai project*⁵, there were qualitative descriptions of the AIs being competitive with human players, but the level of the human players and the quantitative evaluation are absent. In [24] human-vs-machine results were reported, but the evaluation was fixed on two pre-defined decks and the AI seemed to achieve a $< 50\%$ winrate against the Legendary level human players (Table III of [24]). While the general Legendary level players are top 0.5%, the exact rank of the human players were not reported in [24]. In this work, the evaluation is in full game where the AI and the human player have the complete freedom in both deck building and battle. Our AI can defeat a top 10 human player in the Legendary league, which is estimated to be at least top 0.0005% human players.

There are also Hearthstone AI competitions [25], but no human players are involved as the benchmark.

C. End-to-end policy and Optimistic Smooth Fictitious play

Legends of Code and Magic (LoCM) is a similar strategy card game that is used for research purpose. Previous researches on LoCM mostly use the alternating training: When training the CB policy, BT policy is fixed. When training the BT policy, CB policy is fixed. The CB policy is optimized by

¹<https://github.com/danielyule/hearthbreaker>

²<https://hearthstone.fandom.com/wiki/Ranked>

³<https://hearthstone.fandom.com/wiki/Region>

⁴Rules can be found at <https://hearthstone.fandom.com/wiki/Tournaments>.

⁵<https://github.com/peter1591/hearthstone-ai>

algorithms such as evolutionary method[26], where the score function of cards is defined by expert rules and data mining. The BT policy is optimized by search based methods such as *Monte-Carlo Tree Search* (MCTS) [27] or reinforcement learning [28].

[1] overcomes the weaknesses of alternating training and propose an *end-to-end* (E2E) policy function. It combines the E2E policy function and an *Optimistic Smooth Fictitious Play* (OSFP). The resultant algorithm won the double championship of LoCM competitions of COG2022.

The E2E policy is a single policy function to estimate CB policy and BT policy. Denote the observation as o and the indicator of stage as

$$\delta = \begin{cases} 1, & \text{at CB stage,} \\ 0, & \text{at BT stage.} \end{cases}$$

The E2E policy is calculated by

$$\pi_{\theta}(\cdot|o) = \delta\pi_{\theta_{CB}}(\cdot|o) + (1 - \delta)\pi_{\theta_{BT}}(\cdot|o), \quad (2)$$

where θ_{CB} and θ_{BT} are parameters of CB policy and BT policy respectively. There may exist overlapping parameters between θ_{CB} and θ_{BT} . The value function is estimated by extracted features from both CB policy and BT policy.

Like the traditional Smooth Fictitious Play [29], [30], OSFP is an iterative algorithm that the current policy is updated by playing against the mixture of historical models as the opponent, a procedure called Smooth Best Response (SBR). However, OSFP puts higher mixing weights for the latest model, which leads to the property of last-iterate convergence [1]. It means that the training can be stopped anytime, and it suffices to simply deploy the last checkpoint model which is guaranteed to be the strongest one so far. Finally, like the Deep Fictitious Play in recent literature[31], the SBR is solved by RL method where the parameters of the policy net and value net are updated.

III. APPLICATION ON HEARTHSTONE

We arrange the remaining contents in the following order. In this section, we introduce the beginning of our work, which is our initial baseline. In the next section, we introduce the problems we met and the techniques to overcome the difficulties.

Hearthstone contains three stages, which are *picking hero* (PH) stage, *cards deck building* (CB) stage and *battle* (BT) stage. The PH stage is simply choosing a hero. Since PH is a new stage compared to LoCM and previous work [1], it's a natural question whether we should include PH into our E2E policy function. The conclusion is **NO** and further discussions are shown later. The observation space of CB stage contains information of all cards, the selected cards and the cards that can be selected now. The action space of CB stage is a categorical distribution to select a card. The observation space and the action space of BT stage is more complicated than that of previous work [1]. All observable information of a human player is included in the observation space, which is

listed in Tab. V and Fig. 2. The action space of each step is a single categorical distribution. Playing one card in the game needs two operations, which is represented by a binary tuple in the form of $(type, target)$. *Type* is one of $\{my\ hand\ card, my\ board\ card, opponent's\ board\ card, my\ hero\ power\ card^6\ and\ end\ turn\ card^7\}$. *Target* is one of $\{my\ hero, opponent's\ hero, my\ board\ card\ and\ opponent's\ board\ card\}$. For instance, the first operation is to select a card as *type* and the decision type is *selecting*. The second operation is selecting a card as *target*. Depending on the selected card of the first operation, the decision type of the second operation may differ. We process the logic and apply the *type* card onto the *target* card, which completes a play of a card.

By such an auto-regressive action space decomposition [6], the action size is at most several tens at each time step. Also, we employ a 0/1 *action mask* to indicate the available actions that varies at each time step. When computing the softmax policy inside the neural net, the probabilities of those unavailable actions are zeroed using this action mask. A complete list of observation space and action space of CB stage and BT stage are shown in Appendix VII-2.

The function approximation follows the E2E policy function [1]. The policy function is constructed by equation 2, where $\pi_{\theta_{CB}}$ and $\pi_{\theta_{BT}}$ are estimated respectively, where θ_{CB} and θ_{BT} share card embedding and hero embedding. The value function is estimated by intermediate features of $\pi_{\theta_{CB}}$ and $\pi_{\theta_{BT}}$. The entire network is shown in Appendix VII-3.

The policy gradient that we have applied is a combination of V-Trace [32] and UPGO [6]. For notational convenience, at step t we denote by V_t the value function predicted by the neural net, by

$$\begin{aligned} \delta_t &= r_t + \gamma V_{t+1} - V_t, \\ \rho_t &= \pi(a_t|o_t)/\mu(a_t|o_t) \end{aligned}$$

the temporal difference and the importance sampling weight, respectively. π is learning policy; μ is behaviour policy that generates the data, which is usually π in previous iteration. V-Trace estimates the value function by off-policy correction:

$$v_t = V_t + \sum_{i=0}^k \gamma^i \left[\prod_{j=0}^{i-1} \min(\rho_{t+j}, \bar{c}) \right] \min(\rho_{t+i}, \bar{\rho}) \delta_{t+i}, \quad (3)$$

where $\bar{c}$ and $\bar{\rho}$ are hyperparameters that reduce the variance but introduce a bias. When $\bar{c} \leq \bar{\rho}$, convergence of equation 3 is guaranteed. The policy gradient of V-Trace is given by

$$\min(\rho_t, \bar{\rho})(r_t + \gamma v_{t+1} - V_t) \nabla_{\theta} \log \pi_{\theta}(a_t|o_t). \quad (4)$$

The value function is trained by minimizing l_2 distance between V_t and v_t .

Since *Hearthstone* is a 2p0s game, a meta algorithm is required as the wrapper of DRL in order to find NE solution. We follow previous work on LoCM and apply OSFP [1]. We provide the pseudo code of OSFP in Appendix VII-4.

⁶We regard *hero power* as a card that can be selected and applied.

⁷We regard *end turn* as a card that works for ending the turn.

IV. IMPROVEMENTS AND EXPERIMENTS

A. Machine-vs-Machine Evaluation Criterion

To evaluate two models A -vs- B , we run $2 \times 3 \times 3 \times 2500$ matches and calculate the average winrate. Both A and B play three heroes (henceforth $3 \times 3 \times$). Each model can play on first turn at BT stage or not (henceforth $2 \times$). For each condition, we run 2500 matches. We say A increases $a\%$ w.r.t. B when the winrate of A -vs- B is $(50+a)\%$. All models are compared after training for 2 days, unless otherwise specified. Tab. I gives the results, where each model is abbreviated as “letter digit” that stands for the type and the version number, e.g., “b4”, “c5”. We also use “letter digit how-many-days” to indicate how long the corresponding model trains, e.g., “b4-23day” means the b4 model checkpoint trained for 23 days. See the running texts in the following contents.

(%)	b0	b1.5	b2	b3	b4	c5
b0	50.0	43.0	37.2	24.3	26.4	19.8
b1.5	57.0	50.0	40.0	30.4	29.7	21.3
b2	62.8	60.0	50.0	34.9	38.3	29.9
b3	75.7	69.6	65.1	50.0	43.5	39.0
b4	73.6	70.3	61.7	56.5	50.0	44.5
c5	80.2	78.7	70.1	61.0	55.5	50.0

TABLE I

WINRATE TABLE OF ALL MODELS. THE ENTITY AT (i, j) REPRESENTS THE WINRATE OF $(i$ -TH ROW MODEL)-VS- $(j$ -TH COLUMN MODEL). ALL MODELS ARE DESCRIBED FROM SEC.IV-B TO SEC.IV-H.

B. Exclusion of PH from E2E

Previous work on LoCM only considers CB stage and BT stage. The E2E policy for the two stages is define as in equation 2. However, *Hearthstone* has three stages. To deal with the additional PH stage, there exist two options. One option is to simply sample a hero for each side of a match. The distribution is the prior distribution of heroes, and we simply let it to be a uniform distribution. The other option is including PH into the E2E policy function. To extend E2E into three stages, we define

$$\delta_X = \begin{cases} 1, & \text{if at } X \text{ stage,} \\ 0, & \text{else.} \end{cases}$$

Then the E2E policy can be defined by

$$\pi_{\theta}(\cdot|o) = \delta_{PH}\pi_{\theta_{PH}}(\cdot|o) + \delta_{CB}\pi_{\theta_{CB}}(\cdot|o) + \delta_{BT}\pi_{\theta_{BT}}(\cdot|o). \quad (5)$$

$\pi_{\theta_{PH}}$ is a single-step policy with “null” observation (just like a Bandit), where each action represents the probability of picking the hero.

We expect that the distribution of picked heroes is similar to the players’ on the Blackrock Mountain version of the game when we include PH into E2E policy. However, our experiment fails to do so. Fig. 1 shows the distribution of picked hero and it obviously converges to a single hero. We have tuned the weight of entropy penalty of $\pi_{\theta_{PH}}$, but it converges to either a single hero or the uniform distribution. The policy function only performs well on hunter vs hunter, but performs

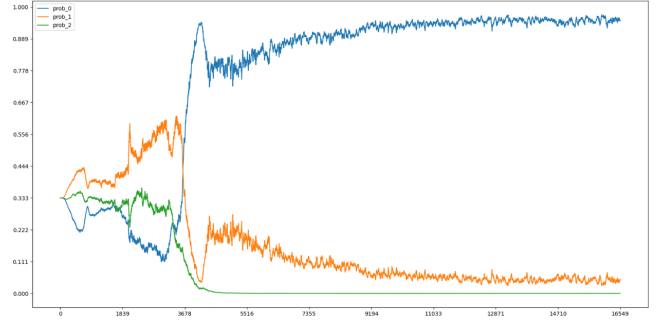


Fig. 1. Distribution of picked hero during training process when including PH into E2E. Blue for hunter, orange for mage and green for warrior.

poor on all the other matches. Hence, we exclude PH from E2E and simply sample a hero from the uniform distribution. We denote the baseline that uniformly samples a hero and is trained with E2E policy as *baseline-0* (b0). Notice that this beginning baseline is actually strong, because the same routine has already won the double championship on LoCM [1].

C. Improvement on γ

Previous work on LoCM [1] trains E2E policy with discount rate $\gamma = 0.99$. In strategy card games, however, each episode has limited steps (~ 100), and a non-zero reward ($+1/-1$) indicating game win/loss is only given at the end of the episode. Hence, the episodic length is limited and the variance of return is bounded. In this setting, $\gamma = 1.0$ should feasibly and faithfully recover the return we are interested in (i.e., the game win/loss), while $\gamma = 0.99$ fails to do so, despite $\gamma = 0.99$ typically leads to faster training convergence. Considering this, we set $\gamma = 1.0$ instead of $\gamma = 0.99$, and find out that the winrate increases around 7% w.r.t. b0. We denote b0 with $\gamma = 1.0$ as *baseline-1* (b1). Note in Tab. I we do not include b1, as it is shortly replaced by b1.5, see below section.

D. Random Initialization via Random-CB

Intuitively, a diversified deck helps the learning of BT policy. To encourage such explorations, we implement random initialization during the CB stage, which is called random-CB. Before each match, we sample a random-CB number $n \in \{0, 1, 2, 4\}$ for each player with probability $(0.5, 0.25, 0.125, 0.125)$. Once n is chosen, the first n steps of CB stage choose cards by uniformly random sampling. This is achieved by multiplying 0 on the logits of $\pi_{\theta_{CB}}$ for the first n steps, shown in Fig. 2. When evaluating the model, we disable random-CB. We find the model is more prone to build a “human-like” deck with random-CB. For example, we observed one of the decks built by the model is much like the so called “Flamewaker Mage” as was invented by and popular in the community back in 2015⁸. Even though the winrates are on-par with b1, we regard it as progress. We denote b1 with random-CB as *baseline-1.5* (b1.5).

⁸<http://games.sina.com.cn/o/z/hs/2015-04-28/1127616179.shtml>

E. Latent Off-Policy Reduction via Balancing

Since a large scale training system is asynchronous, off-policy problem is inevitable. However, we can balance the data production (i.e., rollout during the environment-agents interaction) and consumption (i.e., learning data batch to update Neural Net) to improve the performance. We denote the producing speed as s_p and the consuming speed as s_c . We usually require $s_p \geq s_c$ to fully exploit the utility of GPU resource. We denote $s_p = c \cdot s_c$ with $c \geq 1$. Since it's almost impossible to let $c = 1$ in exact, we only consider $c > 1$.

In a distributed training system that adopts the Actor-Learner architecture [32], [33], there are two options to deal with data overproduction. The first option is a *ring* buffer. The Learner samples uniformly from the buffer. When overproduction happens, the Actor directly overwrites the buffer in a ring order, disregarding whether the existed data have been consumed or not. The second option is a *queue* buffer. The Learner takes the first-in-first-out samples from the buffer. When overproduction happens, the Actor blocks and puts data into the buffer until the buffer is not full. Because the *ring* buffer cannot control how many times each data is used exactly, it involves additional sampling error compared with the desired stationary distribution. Hence, we adopt the *queue* buffer. Since the *queue* buffer requires the Actor to wait, we expect c to be close to 1, where the off-policyyness is alleviated. We find that b1.5 has $c > 2$, therefore we reduce the Actors by half. This surprisingly increases 10.0% winrates w.r.t. b1.5. We denote b1.5 with only half Actors ($c \sim 1$) as *baseline-2* (b2).

F. Improvement on V-Trace

During the training process, we find the convergence speed is not satisfactory. If we look at V-Trace in equation 3, it only requires $\bar{c} \leq \bar{\rho}$ for convergence rather than $\bar{c} \leq \bar{\rho} \leq 1$. Many previous works take $\bar{\rho} = 1$ [32], [1], but it's not necessary. Hence, we increase the hyperparameter $\bar{\rho}$ to increase the variance and reduce the bias. We also find many ρ s from b2 to be extremely small ($\sim 10^{-4}$ or lower), which eliminates the trace. Hence, we also clip ρ to be larger than a threshold to prevent diminish of the trace. Define

$$\text{clip}(x, \alpha, \beta) = \min(\max(x, \alpha), \beta).$$

We revise the V-Trace and estimate the value function by

$$v_t = V_t + \sum_{i=0}^k \gamma^i \left[\prod_{j=0}^{i-1} \text{clip}(\rho_{t+j}, \underline{c}, \bar{c}) \right] \text{clip}(\rho_{t+i}, \underline{\rho}, \bar{\rho}) \delta_{t+i}, \quad (6)$$

where we use $\underline{c} = \underline{\rho} = 0.001$ and $\bar{c} = \bar{\rho} = 1.007$ in practice.

We also find the policy gradient given by clipped importance sampling in equation 4 is not stable. Hence, we replace it by that of PPO [34], which is

$$\begin{aligned} \nabla_{\theta} \min((r_t + \gamma v_{t+1} - V_t) \frac{\pi_{\theta}(a_t|o_t)}{\mu(a_t|o_t)}, \\ (r_t + \gamma v_{t+1} - V_t) \text{clip}(\frac{\pi_{\theta}(a_t|o_t)}{\mu(a_t|o_t)}, 1 - \epsilon, 1 + \epsilon)), \end{aligned} \quad (7)$$

where we use $\epsilon = 0.2$ in practice. Note in equation 7 the value estimate v_{t+1} is still V-Trace as in equation 3, which differs from conventional PPO implementation.

With these improvements on V-Trace, the winrate increases 15.1% w.r.t. b2. We denote b2 with improvements on V-Trace as *baseline-3* (b3).

G. Model Isolation by Hero

All baselines so far are trained with 8 GPUs accompanied with CPUs that ensures data are properly produced and consumed. It's a natural question that whether we can achieve better performance with more computational resources. Unfortunately, when we limit the capacity of neural network, we find that the performance wouldn't be better when we double/triple the batch size/GPUs/CPUs. Considering the fact that each hero has a unique play style, we isolate the model for each hero. This triples the computational resources, but it also promotes the performance. The hero isolation increases 6.5% winrates w.r.t. b3. We denote b3 with hero isolation as *baseline-4* (b4).

H. Cheat

Up to now, b4 is strong enough to play with the top players, shown later. Noticing the fact that *Hearthstone* is a POMDP, we wonder whether cheating by peeking the hidden information leads to a stronger policy. We include the opponent's first n chosen cards in deck into the observation spaces of CB stage and BT stage (Note the opponent hand cards are still unobservable during BT stage.). Hence, the cheating policy is expected to build adversarial deck and to play adversarial strategy against the opponent. However, if both the target policy and the opponent's policy in a game cheat during the training process, it's inconsistent when the target policy plays with the no cheating policies.

To mitigate the gap, we use an asymmetric training strategy. At the beginning of each game, we sample $n_1, n_2 \sim \text{Uniform}(\{0, 1, \dots, 30\})$. Then we let $n_2 = \min(n_1, n_2)$. Let the target policy, which sends training data to Learner, to observe the first n_1 chosen cards of the opponent's deck. Let the opponent's policy, which doesn't send training data to Learner, to observe the first n_2 chosen cards of the opponent's deck. In this way, we guarantee the target policy can always observe more information than the opponent's. When evaluating the cheating policy, we sample $n \sim \text{Uniform}(\{0, 1, \dots, 30\})$ and let the cheating policy to observe the first n chosen cards of the opponent's deck. It increases 5.5% winrates w.r.t. b4. We denote it as *cheat-5* (c5).

V. MACHINE-VS-HUMAN EVALUATION

We develop a dedicated web GUI for machine-vs-human test, see Appendix VII-6.

By the end of November 2022, our models (some intermediate versions between b3 and b4, not reported in Tab. I) were able to stably defeat two legendary-level human players who best ranked around 20000 and 1000, respectively, in the official league of China region. In December 2022, we invited another much stronger human player for the machine-vs-human test.

The human player is a *Hearthstone* streamer, whose best rank was top 10 of the official League in China region which is estimated to be of millions of players. The human player ranked 60 to 70 when participating in the test. Before the serious tournament, we deployed a b4-2day model for the human player to play around and warm up. Such a casual playing is designed to help the human player getting familiar with the Blackrock Mountain version of the game, which is different from the latest commercial *Hearthstone* version. The human player can also build and save decks that are used in the later serious tournaments. The human player can play warm-up matches as many times as wanted. We have told the human player that the official agent is much stronger than the warm-up agent. Once the human player notified us that he was ready, we started four official CQ tournaments. We deployed two different models, b4-23day (the best normal-version) and c5-16day (the best cheat-version that peeks the deck). Each model played two tournaments. The model picks hero by uniformly sampling, when it is required to change hero by the rule. All tournaments are Bo5. The human player doesn't know that we deploy different models. We also set monetary award for the evaluation: For each formal match played, the human player earns xx bucks; But if she/he wins that match, 5 times bucks are earned.

	tournament 1	tournament 2
b4-23day-vs-human	3:0	3:0
c5-16day-vs-human	3:1	3:2

TABLE II
MACHINE-VS-HUMAN EVALUATION RESULT.

The result is shown in Tab. II. It's surprising that the models win all the four tournaments. According to the online interview of the human player after the test and by reviewing the game replay, we have the following conclusions. *Firstly*, the E2E policy and the OSFP training can yield very strong card game agent. The human player says that (English translation from Chinese, the same below): "Generally speaking, the AI's performance is not bad. It knows when to clear the minions. The one turn kill is also accurate." *Secondly*, the agent learns to build human-like deck. The human player says that: "There is one match against AI's Flamewaker Mage. I'm killed when I have 21 hp. It's amazing." However, there is no prior *domain knowledge* or *expert data* during the training process and the models learn to build deck absolutely from-scratch, but the human player regards the AI's deck as Flamewaker Mage which is a popular choice in the *Hearthstone* community. In Sec.VII-7 we show these two decks. *Thirdly*, training time matters. Though c5-2day-vs-b4-2day has winrate 55% in Tab. I, c5-16day performs worse than b4-23day when playing with human players. For instance, c5-16day uses Antique Healbot at full health at one match, which is an obvious mistake. But this has never happened on b4-23day. *Fourthly*, cheat-on-deck can generate specific strategy against the human player. There is one match the AI takes two Antique Healbots that well limit

the human player's Freeze Mage. The human player says that: "AI takes so many healbots. It's almost killed and two healbots heal it!". *Fifthly*, cheat-on-deck encourages diversity. There is one match that the human player says that: "The AI's warrior has Kel'Thuzad. It's unexpected."

Our machine-vs-human evaluation shows that the combination of DRL and OSFP with carefully finetuned techniques is able to produce a master-level agent on card games. The learned agent is also human-like without any prior knowledge of the game, except that the observation space and the action space of the agent are carefully aligned with human players'. Cheat leads to adversarial policies against the opponent's.

VI. CONCLUSION

In this work, we follow existed work on strategy card games and apply the algorithms to the famous game *Hearthstone*. We further improve several techniques, which leads to better performance on *Hearthstone*. We also do machine-vs-human evaluation, where our agents show master-level ability and human-like strategy. Our work show that the algorithm framework of combining E2E policy and OSFP is a promising way to train a master-level agent from scratch on strategy card games.

Acknowledgements

Our colleagues Ze Chen and Wei Xi also contributed to the development and maintenance of the web GUI. We thank the CoG 2023 anonymous reviewers for their helpful feedback that improves the quality of this paper.

VII. APPENDIX

1) *Hyperparameters*: We follow most hyperparameters of [1], but a few are adjusted. The adjustments are highlighted in color.

Parameter	Value
Weight of policy gradient from PPO	1.0
Weight of policy gradient from UPGO	1.0
Weight of value function loss	1.0
Weight of entropy penalty	0.01
Learning rate	7e-5
Batch size	1e+4 * 8gpu
Discount	1.0
LSTM states	256
Sample reuse	2
V-Trace c clip	[0.001, 1.007]
V-Trace ρ clip	[0.001, 1.007]

TABLE III
REINFORCEMENT LEARNING HYPERPARAMETERS.

Parameter	Value
Self-play Probability, p	0.6
Add to historical model threshold, ξ	0.55
Add to historical model max LP, c	6
Num samples of each LP	3.2e+8

TABLE IV
OSFP HYPERPARAMETERS. THE NOTATIONS ARE CORRESPONDING TO ALG. 1.

2) *Observation Space and Action Space*: The observation space and action space are detailed in Tab. V.

3) *End-to-End Network*: The E2E neural net architecture is given in Fig. 2.

4) *Optimistic Smooth Fictitious Play*: Our optimistic smooth fictitious play strategy follows [1]. For completeness, we describe it here. We define a *learning period* (LP) to be a fixed number of training samples. During each LP, we sample opponents according to the payoff table between current policy and historical policies. After each LP, we decide whether current policy should be added to historical policies. The complete codes are shown in Alg. 1 and the corresponding hyperparameters are listed in Tab. IV.

5) *Compute Resource*: We rely on distributed RL training for all the experiments reported in this study. To do so, we use an internal framework which is essentially an Actor-Learner architecture [32] that requires both GPUs and CPU cores for a single RL training. For the best non-cheat training b4 reported in Tab. I, we employ 24 GPUs (V100) and 5856 CPU cores.

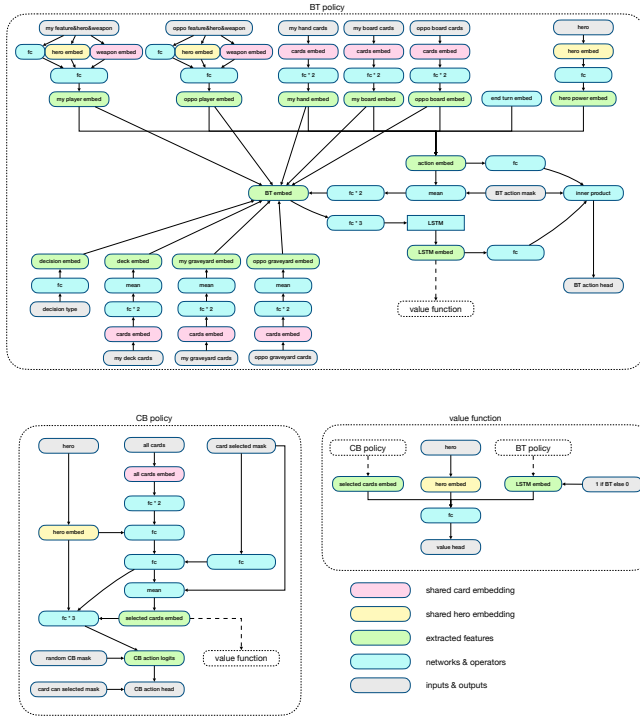


Fig. 2. Neural Network Structure.

Algorithm 1 Optimistic Smooth Fictitious Play.

```

Init  $H = []$ .
Init  $0 < p < 1, 0 < \xi < 1$ .
Init  $c > 0, count = 0$ .
Init probability function  $f$ .
for  $LP = 0, 1, 2, \dots$  do
  Init  $G = C = [0] * \text{len}(H)$ .
  while LP not end do
    for each actor do
      if  $\text{len}(H) = 0$  or  $\text{Unif}(0, 1) < p$  then
        Opponent player = current learner.
        Finish game.
      else
        Sample  $i \sim f(\dots, (G[i], C[i]), \dots)$ .
        Opponent Player =  $H[i]$ .
        Finish game and get  $g \in \{+1, -1\}$ .
         $G[i] = G[i] + g, C[i] = C[i] + 1$ .
      end if
    end for
  end while
  if  $(G[i]/C[i] > \xi, \forall i)$  or  $(count > c)$  then
    Add current learner to  $H$ .
     $count = 0$ .
  else
     $count = count + 1$ .
  end if
end for

```

Observations	Descriptions
hero	my hero, one of (mage, hunter, warrior)
card set	all cards, including each hero's specific cards
card selected mask	1 if the card has been selected else 0
card can selected mask	1 if the card can be selected else 0
hero	my hero, one of (mage, hunter, warrior)
oppo hero	opponent's hero, one of (mage, hunter, warrior)
my deck	cards in my deck
decision type	one of (construct, select, minion battlecry, spell card, minion/hero attack, hero power, end turn)
my board	minions on my board and their scalar features
oppo board	minions on opponent's board and their scalar features
my hand	cards in my hand and their scalar features
my graveyard	cards in my graveyard and their scalar features
oppo graveyard	cards in opponent's graveyard and their scalar features
my player	my features, such as number of hands and minions, mana, weapon
oppo player	opponent's features, such as number of hands and minions, mana, weapon
BT action mask	1 if the action can be done else 0

TABLE V
OBSERVATION SPACE. THE TOP BLOCK IS FOR CB AND THE BOTTOM BLOCK IS FOR BT.

Action	Description
selected card	card that is selected into the deck, corresponding to <i>card set</i> in observation space
selected card	card that is selected as one of (<i>type, target</i>), one step for <i>type</i> and one step for <i>target</i> , $type \in \{\text{my hand card, my board card, opponent's board card, my hero power card, end turn card}\}$, $target \in \{\text{my hero, opponent's hero, my board card and opponent's board card}\}$

TABLE VI
ACTION SPACE. THE TOP LINE IS FOR CB AND THE BOTTOM LINE IS FOR BT.

6) *Web GUI*: For the machine-vs-human test, we develop a dedicated web GUI where we can deploy models that play against a human player in remote, see Fig. 3. The human players can build their own decks and save, and play against AI models in a tournament.

7) *Decks*: As mentioned in Sec.IV-D and Sec.V, we provide the AI deck and the human deck in Fig.4.

8) *Game-Play Videos*: We provide videos of two selected matches in the human-vs-AI evaluation, named as “mmt1”⁹ and “mmt2”¹⁰, respectively. In both videos, top screen is AI player with the b4-23day model (best normal version), bottom screen is human player who is the Hearthstone streamer, as described in Sec.V of the main texts.

The videos are made by checking the replay files and recording the game-play, therefore the hand cards and secrets of both players are visible to the viewer. During the human-vs-AI test, however, such information is hidden from each other player, as in the real Hearthstone games.

Here are the brief descriptions of the two videos.

⁹https://youtu.be/ht_EZbMJmJA

¹⁰<https://youtu.be/YnW2ypZL4HY>

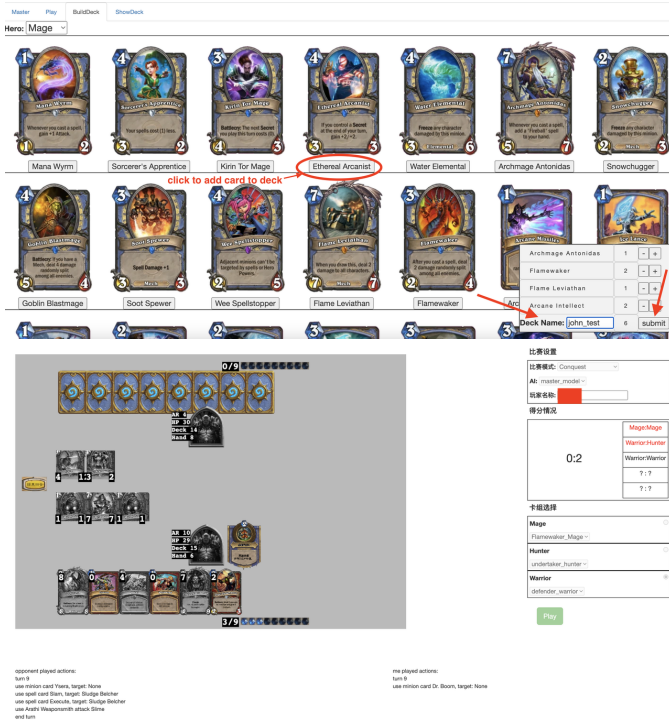


Fig. 3. Web GUI. Upper: deck building. The human player can browse all cards available to a particular hero (Mage, Warrior or Hunter) and select the 30 cards. The deck can be saved, and be retrieved in the later battle stage. Lower: battle. The screen top is AI and the bottom is human player. The actions in previous turn taken by AI (denoted as “opponent player”) and the human player (denoted as “me”) are displayed as texts in the bottom screen.



Fig. 4. The complete 30-card decks. (a): a Mage deck learnt by AI; (b): the human deck “Flamewaker Mage” that was popular in around April 2015.

“mmt1”. AI uses the learnt deck which is much like the “Flamewaker Mage” in real world (See main texts), while the human player uses the so-called “Freeze Mage” deck. Early in the match, both AI and human player choose to draw more cards by using the effect of the cards from their deck, intending for gaining the upper hand. In the mid-match, both players place the card “Emperor Thaurissan” to reduce mana; However, AI manages to accumulate more advantages by successfully playing several combos, and preserves more hand cards during the course. Late in the match, the human player still has 21 Hit Points (out of the full 30 HPs), and henceforth does not choose to place the secret “Ice Block” by considering that 21 HPs should be safe. On the other hand, AI has, however, obtained the card “Flamewaker” and many other spell cards by drawing sufficient cards in the early- to mid-match, resulting to a potential damage that reaches 31 HPs. Consequently, AI performs a One-Turn-Kill (OTK) that ends the match. AI seems to adopt a typical “accumulate-hand-cards-and-outbreak” strategy.

“mmt2”. In regards of deck, this match is still the AI’s learnt Flamewaker Mage vs the human player’s Freeze Mage. In this match, AI constantly pushes the human player by maximizing the mid- to short-term benefit at each turn and gaining the upper hand gradually. Finally, AI wins the match. Compared to “mmt1”, the strategy that AI adopts in this match seems more deliberate and less risky.